

Kriptografija — napredne teme

Aleksandra Labović (mi241025)

Ana Knežević (mi241052)

Marijana Čupović (mi251018)

Marko Lazarević (mi251005)

Matija Stanković (mi21207)

Nikola Milošević (mi241063)

Staša Đorđević (mi251007)

Tanja Gavrić (mr251065)

Tijana Jakšić (mr241087)

Vukašin Marković (mi241051)

8. septembar 2026.

Sadržaj

1	Uvod	1
1.1	Motivacija	1
1.1.1	Osnovni pojmovi	1
1.2	Ilustracija	1
2	Bezbedno izračunavanje sa više učesnika	2
2.1	Uvod	2
2.2	Slučaj dva učesnika	4
2.2.1	Maskirana kola	5
2.2.2	Konstrukcija maskiranog kola	5
2.2.3	Evaluacija maskiranog kola	8
2.2.4	Jaov protokol	9
2.3	Slučaj više učesnika: <i>pošteni ali radoznali</i> učesnici	10
2.3.1	Sabirači	12
2.3.2	Množači	13
2.4	Model više učesnika: zlonamerni učesnici	15
2.4.1	Faza pripreme	16
2.4.2	Glavna faza	16
2.5	Zaključak	17
2.6	Pitanja	18
3	Certificates	19
3.1	Uvod	19

3.2	Sertifikati i sertifikaciona tela	20
3.3	Generisanje efemernih simetričnih ključeva iz statičkih simetričnih ključeva	22
3.3.1	Wide-Mouth Frog protokol	22
3.3.2	Needham–Schroeder protokol	23
3.3.3	Kerberos	25
3.4	Generisanje efemernih simetričnih ključeva iz statičkih javnih ključeva	25
3.4.1	Prenos ključa pomoću javnog ključa	26
3.4.2	Diffie–Hellman protokol	28
3.4.3	Potpisani Diffie–Hellman protokol	29
3.4.4	Station-to-Station protokol	29
3.4.5	Blake–Wilson–Menezes protokol	30
3.4.6	MQV protokol	31
3.5	Simbolička analiza protokola	32
3.6	Zaključak	33
3.7	Pitanja	34
4	Commitments	35
4.1	Motivacija i intuicija	35
4.2	Formalna definicija	36
4.3	Konkretna konstrukcije	37
4.3.1	Heš-zasnovana šema posvete	37
4.3.2	Pedersenova šema posvete	38
4.3.3	KZG polinomijalne posvete	39
4.4	Primene u blockchain ekosistemu	41
4.4.1	Šablon posveti-pa-otkrij	41
4.4.2	Merkle stabla kao vektorske posvete	42
4.5	Šeme posvete kao osnova sistema za dokazivanje	43
5	Uvod u dokaze nultog znanja	45
5.1	Motivacija i osnovna terminologija	46

5.2	Protokol za izomorfizam grafova	46
5.3	Nulto znanje i klase složenosti	50
5.4	Sigma protokoli	52
5.4.1	Fiat-Shamir heuristika i neinteraktivni dokazi	54
5.5	Primena u elektronskom glasanju	55
5.6	Zaključak	56
6	ZK-snark	57
6.1	Uvod u zk-SNARK	57
6.1.1	Definicija i osnovna ideja	57
6.1.2	Od interaktivnih ka neinteraktivnim dokazima	58
6.1.3	Trusted setup	59
6.1.4	Setup, Prove i Verify algoritmi	59
6.2	Primena zk-SNARK sistema u mreži Zcash	61
6.2.1	Problem privatnosti javnih blokčejn mreža	61
6.2.2	Kako izgleda jedna transakcija u blokčejn mreži	62
6.2.3	Šta je javno, a šta Zcash skriva	63
6.2.4	Validacija transakcija pomoću zk-SNARK dokaza	63
6.2.5	Tok jedne Zcash transakcije	65
6.3	Prednosti i nedostaci zk-SNARK sistema	66
6.3.1	Prednosti	66
6.3.2	Nedostaci	67
6.4	Zaključak	67
7	ZK-stark	69
7.1	Motivacija i računarski integritet	70
7.2	Transparentnost i odsustvo poverljivog podešavanja	71
7.3	Postkvantna sigurnost	72
7.4	Od izvršavanja programa do polinomskih ograničenja	73
7.5	Polinomi traga i proširenje domena	74
7.6	Posvećivanje vrednostima pomoću Merklovog stabla	75

7.7	Polinom kompozicije	76
7.8	FRI protokol	77
7.9	DEEP-FRI	78
7.10	Pretvaranje u neinteraktivni dokaz	78
7.11	Pojednostavljen primer zk-STARK postupka	79
7.12	Skalabilnost	80
7.13	Primene zk-STARK sistema	80
7.14	Poređenje zk-SNARK i zk-STARK sistema	81
7.15	Aurora i drugi transparentni sistemi	82
7.16	Prednosti i nedostaci zk-STARK sistema	83
7.17	Zaključak	83
8	Kriptografija zasnovana na rešetkama	84
8.1	Uvod	84
8.2	Kvantni računari	85
8.3	Rešetke	86
8.3.1	Teški problemi na rešetkama	88
8.4	NTRU	90
8.4.1	Generisanje ključa	90
8.4.2	Šifrovanje	91
8.4.3	Dešifrovanje	91
8.4.4	Interpretacija pomoću rešetaka	91
8.5	Zaključak	93
8.6	Pitanja	94
9	Učenje sa greškama	95
9.1	Uvod	95
9.2	Učenje sa greškama	96
9.2.1	Protokol javnog ključa zasnovan na problemu učenja sa greškama	98
9.3	Učenje sa greškama u prstenu	100
9.3.1	Protokol javnog ključa zasnovan na učenju sa greškama u prstenu	101

9.4	Kyber	103
9.4.1	Modulno učenje sa greškama	104
9.4.2	Konstrukcija ML-KEM šeme	104
9.4.3	Praktična upotreba Kyber-a	104
9.5	Pitanja	105
10	Potpuno homomorfna enkripcija	106
10.1	Uvod	106
10.2	Istorija	106
10.3	Multiplikativno ili aditivno homomorfna enkripcija	107
10.4	Notacija i pojmovi	107
10.5	DGHV	109
10.5.1	Uvodni primer	109
10.5.2	DGHV	110
10.6	Bootstrapping	111
10.7	Primene	112
10.7.1	Podaci u oblaku	112
10.7.2	Mašinsko učenje	112
10.7.3	Internet stvari	113
10.8	Otvorena pitanja	113
10.9	Pitanja	114
11	Secret sharing	115
11.1	Uvod	115
11.1.1	Način predstavljanja struktura	117
11.2	Opšte metode deljenja tajni	117
11.2.1	Ito-Niizeki-Saito šema	117
11.2.2	Replicirano deljenje tajni	118
11.3	Rid-Solomonovo kodiranje	119
11.3.1	Rekonstrukcija podataka - Lagranžova interpolacija	120
11.3.2	Detekcija greške	120

11.3.3	Ispravljanje greške: Berlekemp-Velč algoritam	121
11.4	Šamirova šema deljenja tajni	121
11.4.1	Algoritam deljenja	122
11.4.2	Algoritam rekonstrukcije	122
11.4.3	Korekcija pogrešnih udela	123
11.4.4	Pseudoslučajno deljenje tajni (PRSS)	123
11.5	Zaključak	124
11.6	Pitanja	124
12	Zaključak	125
12.1	Rezime rezultata	125
12.1.1	Dalji rad	125

Glava 1

Uvod

1.1 Motivacija

Kriptografija obezbeđuje poverljivost, integritet i autenticnost podataka. U ovom radu koristimo standardni zapis i okruženja za matematičke izraze.

1.1.1 Osnovni pojmovi

Definicija 1.1.1. Kriptosistem je uređeni skup (P, C, K, E, D) gde su P skup otvorenih tekstova, C skup sifrovanih tekstova, K skup ključeva, a E i D funkcije enkripcije i dekripcije.

Teorema 1.1.2. Za svako $m \in P$ i $k \in K$ važi $D_k(E_k(m)) = m$.

Dokaz. Sledi direktno iz definicije korektnosti kriptosistema. □

Primer 1.1.3. Za Cezarovu sifru sa pomerajem $k = 3$, slovo A prelazi u slovo D .

1.2 Ilustracija

Slika 1.1 prikazuje mesto za ilustraciju. Ako fajl ne postoji, slika se preskace bez prekida kompilacije.

Slika 1.1: Primer slike

Knjiga [31] je dobar uvod u kriptografiju.

Glava 2

Bezbedno izračunavanje sa više učesnika

2.1 Uvod

Osnovni problem kriptografije je kako bezbedno preneti poruku od pošiljaoca do primaoca, a da pritom ne otkrijemo sadržaj poruke neovlašćenim trećim licima. Na primer u Difi-Helmanovoj razmeni ključeva, dva učesnika (Alice i Bob) žele da izračunaju zajednički tajni ključ, a da pritom ne otkriju svoje privatne ključeve. Nakon razmene Alice i Bob mogu da izračunaju zajednički tajni ključ, dok njihovi privatni ključevi ostaju skriveni kako jedan od drugog, tako i od potencijalnog prisluškivača. Postavlja se pitanje da li je moguće ovaj proces uopštiti na više učesnika, i na izračunavanje proizvoljnih funkcija nad privatnim podacima učesnika.

Bezbedno izračunavanje sa više učesnika (eng. *Secure Multi-Party Computation - MPC*) je oblast kriptografije koja se bavi problemom kako dva ili više učesnika mogu da izračunaju funkciju nad svojim privatnim ulazima, a da pritom ti ulazi ostanu privatni. U zavisnosti od funkcije koja se računa, može doći do određenog curenja informacija o ulazima, ali cilj MPC-a je da se to curenje svede na minimum, odnosno da se ne otkrije ništa više od onoga što bi se otkrilo da su učesnici poslali svoje ulaze pouzdanom trećem licu koje bi izračunalo funkciju i vratilo rezultat [31]. U primeru 2.1.1 ćemo ilustrovati osnovnu ideju MPC-a [36]¹.

Primer 2.1.1. Zamislimo da imamo grupu prijatelja, koji žele da saznaju ko od njih ima najviše novca, ali ne žele da otkriju koliko tačno novca imaju. Prijatelji mogu pronaći neko pouzdano treće lice kome će poslati svoje privatne informacije

¹str. 1, paragraf **Introduction**

o bogatstvu, a koje će izračunati ko je najbogatiji i vratiti rezultat. Ovaj problem se može rešiti koristeći MPC protokol. Učesnici mogu koristiti MPC protokol da izračunaju ko je najbogatiji, a da pritom ne otkriju svoje tačne iznose novca. U ovom slučaju, funkcija f koju žele da izračunaju je "ko ima najviše novca", a ulazi su njihove privatne informacije o bogatstvu (x_i). Funkcija f izračunava indeks najbogatijeg učesnika, odnosno $f(x_1, x_2, \dots, x_n) = i$ takav da $x_i > x_j, \forall j \neq i, 1 \leq j \leq n, j \neq i$.

Jasno je da će informacija o tome ko je najbogatiji biti otkrivena. MPC protokol će osigurati da se ne otkrije ništa više od toga, kao na primer tačni iznosi novca koje imaju ostali učesnici [31].

Neki kriptografski protokoli koji se mogu posmatrati kao primer MPC-a su [31]:

- Protokoli za elektronsko glasanje, gde se računa konačan rezultat izbora bez otkrivanja pojedinačnih glasova učesnika.
- Protokoli šifrovanja, u kojima pošiljalac i primalac izvršavaju zajedničku računsku proceduru tako da poruka ostane skrivena od protivnika.

Prilikom projektovanja protokola za sigurno izračunavanje sa više učesnika, pored privatnosti ulaza potrebno je razmotriti i ponašanje učesnika. Najčešće se razmatraju dva modela protivnika. *Pošteni ali radoznali* (eng. *honest-but-curious*) protivnici korektno izvršavaju protokol, ali pokušavaju da iz dostupnih informacija saznaju što više o privatnim podacima drugih učesnika. *Zlonamerni* (eng. *malicious*) protivnici mogu proizvoljno odstupati od protokola i slati netačne podatke, pa protokol mora garantovati i privatnost i ispravnost izračunavanja [31].

Ovi modeli bezbednosti značajno se razlikuju, a protokoli koji su sigurni u jednom modelu nisu nužno sigurni u drugom modelu. Na primer, protokol koji je siguran protiv *poštenih ali radoznalih* protivnika najčešće nije primenljiv u prisustvu zlonamernih protivnika.

Još jedna pretpostavka je da je komunikacija između učesnika asinhrona, tj. da poruke koje se šalju između učesnika mogu biti odložene ili čak izgubljene. Ovakva komunikacija je najpraktičnija i najčešća u realnim scenarijima. U takvom modelu, jedan od učesnika mora biti poslednji. U takvoj situaciji, neki učesnici mogu znati rezultat izračunavanja, dok drugi još uvek nisu dobili sve poruke. Zlonamerni učesnici mogu lažirati situaciju i ne poslati poslednju poruku, čime će onemogućiti druge učesnike da dobiju rezultat. Ipak, pretpostavljamo da zlonamerni učesnici neće izvesti takav napad. Protokol koji je siguran protiv zlonamernih učesnika koji mogu da izbrišu poslednju poruku se naziva fer protokol [31].

Neka je n broj učesnika koji učestvuju u izračunavanju tražene funkcije. Želeli bismo da kreiramo protokole za sigurno izračunavanje sa više učesnika koji su sposobni da tolerišu veliki broj zlonamernih učesnika. Ispostavlja se da postoji teorijsko ograničenje broja učesnika čija korupcija može biti tolerisana, ova ograničenja su predstavljena u tabeli 2.1 [31].

Model protivnika	Računski neograničen	Računski ograničen
<i>Pošten ali radoznao</i>	$t < n/2$	$t \leq n - 1$
<i>Zlonameran</i>	$t < n/3$	$t < n/2$

Tabela 2.1: Broj zlonamernih učesnika koje MPC protokoli mogu tolerisati.

Protokoli za sigurno izračunavanje mogu se podeliti u dve kategorije na osnovu pristupa rešavanju problema. Prvi je zasnovan na Jaovoj (eng. *Yao's*) ideji, nazvanom maskirano kolo (eng. *garbled circuit*) ili Jaovo kolo. U ovom slučaju funkcija koja se računa predstavlja se kao binarno kolo, a zatim se šifruju elementi tog kola da bi se formiralo maskirano kolo. Ovaj pristup je jasno zasnovan na računskoj pretpostavci, tj. da je šifarski sistem siguran. Drugi pristup zasniva se na šemama deljenja tajni: ovde se obično funkcija koja se računa predstavlja kao aritmetičko kolo. U ovom pristupu koristi se savršeno sigurna šema tajnog deljenja da bi se postigla savršena sigurnost [31].

Ispostavlja se da je prvi pristup bolji za komunikaciju između dva učesnika, dok je drugi pristup bolji za komunikaciju između tri ili više učesnika [31]. U nastavku ćemo detaljnije prikazati oba pristupa.

2.2 Slučaj dva učesnika

U slučaju bezbednog izračunavanja sa dva učesnika, pretpostavljamo da postoje dve strane A i B , koje imaju ulaze x i y , redom, i da A želi da izračuna funkciju $f_A(x, y)$, a B želi da izračuna funkciju $f_B(x, y)$. Cilj je da se to postigne bez otkrivanja bilo kakvih informacija o ulazima ili funkcijama osim onoga što se može zaključiti iz izlaza funkcija i sopstvenih ulaza [31].

Bez gubitka opštosti, problem se redukuje na jednu funkciju f čiji je izlaz namenjen B -u. U tom slučaju A poseduje dodatni tajni ulaz k ne duži od najdužeg izlaza funkcije $f_A(x, y)$. Potrebno je kreirati protokol u kojem B razmenom poruka sa A dobija vrednost funkcije $f(x, y, k) = (k \oplus f_A(x, y), f_B(x, y))$ tokom kog ne sazna ništa više o x i k osim onoga što je implicirano izlazom. Nakon što B dobije ovu vrednost, on može da pošalje vrednost $k \oplus f_A(x, y)$ nazad A -u, koji zatim može da dešifruje

ovu vrednost koristeći svoj tajni ulaz k , i tako sazna vrednost $f_A(x, y)$ [31]. Ova apstrakcija funkcije omogućava da se MPC primeni i na scenarije koji nisu striktno numerički. U primeru 2.2.1 ćemo ilustrovati ovaj protokol na primeru pregovaranja između dve strane.

Primer 2.2.1. Nesvesno pregovaranje.

Zamislimo da Ana pokušava da proda Bobanu kuću. U principu, svako od njih ima strategiju pregovaranja na umu. Ako numerišemo sve moguće Anine strategije kao $A_1, A_2, \dots, A_t$, i Bobanove kao $B_1, B_2, \dots, B_u$, onda će ishod (nema dogovora, ili prodaja po x dolara) biti odlučan jednom kada se odrede stvarne strategije A_i i B_j koje se koriste. Napišimo ishod kao $f(i, j)$, onda je moguće obaviti pregovaranje nesvesno, u smislu da Ana neće dobiti nikakve informacije o Bobanovim taktikama pregovaranja osim da su one konzistentne sa ishodom, i obrnuto [36]².

2.2.1 Maskirana kola

Jedan od pristupa rešavanju ovog problema zasniva se na Jaovom protokolu sa maskiranim kolima (eng. *garbled circuit*). Pretpostavimo da funkcija $f(x, y)$ može biti izračunata u polinomijalnom vremenu. To znači da postoji binarno kolo koje takođe može izračunati izlaz funkcije [31].

Logičko kolo može biti predstavljeno skupom žica $W = \{w_1, \dots, w_n\}$ i skupom logičkih kapija $G = \{g_1, \dots, g_m\}$. Svaka kapija je funkcija koja uzima kao ulaz vrednosti dve žice i proizvodi vrednost izlazne žice. Na primer, pretpostavimo da je g_1 OR kapija koja uzima kao ulaz žice w_1 i w_2 i proizvodi izlaznu žicu w_3 . Izlazna vrednost se tada izračunava kao logička disjunkcija ulaznih vrednosti. *Maskirano kolo* je kolo u kojem su vrednosti žica maskirane, a kapije su predstavljene kao tabele koje sadrže maskirane vrednosti izlazne žice za svaku kombinaciju ulaznih žica. Ove tabele se nazivaju maskiranim tabelama (eng. *garbled tables*), a proces kreiranja ovih tabela i maskiranja žica se naziva konstrukcija maskiranog kola (eng. *garbled circuit construction*) [31].

2.2.2 Konstrukcija maskiranog kola

Jaov protokol za konstrukciju maskiranog kola je proces u kom jedna strana (koju ćemo zvati A) kreira maskirano kolo, a druga strana (koju ćemo nazvati B) evaluiira maskirano kolo. Pretpostavimo da je funkcija f čiju vrednost tražimo

²str. 4, paragraf **Oblivious Negotiation**

predstavljena logičkim kolom koje sadrži skup žica $W = \{w_1, \dots, w_n\}$ i skup logičkih kapija $G = \{g_1, \dots, g_m\}$. Proces konstrukcije maskiranog kola se sastoji od sledećih koraka [31]:

- Za svaku žicu $w_i \in W$ se biraju dva nasumična kriptografska ključa, k_i^0 i k_i^1 . Prvi ključ predstavlja šifru za logičku nulu, a drugi predstavlja šifru za logičku jedinicu.
- Za svaku žicu $w_i \in W$ definiše se njena logička vrednost $v_i \in \{0, 1\}$ i bira se nasumična vrednost $\rho_i \in \{0, 1\}$. Vrednost ρ_i se naziva permutacioni bit. Njegova uloga je da randomizuje interpretaciju ključeva na svakoj žici, odnosno da sakrije koja od dve vrednosti k_i^0 i k_i^1 odgovara logičkoj nuli, a koja jedinici. Na taj način evaluator ne može direktno da zaključi semantičko značenje ključa koji poseduje, već samo da ga koristi u daljoj evaluaciji kola. Sa njom konstruišemo maskiranu, ili “spoljašnju” vrednost, žice kao $e_i = v_i \oplus \rho_i$.
- Za svaku logičku kapiju $g_i \in G$ sa ulaznim žicama w_{i_0} i w_{i_1} i izlaznom žicom w_{i_2} , kreira se maskirana tabela koja predstavlja funkciju kapije nad maskiranim vrednostima žica. Za neku funkciju enkripcije sa dva ključa E , maskirana tabela se sastoji od vrednosti:

$$c_{a,b}^{w_{i_2}} = E_{k_{w_{i_0}}^{a \oplus \rho_{i_0}}, k_{w_{i_1}}^{b \oplus \rho_{i_1}}} \left(k_{w_{i_2}}^{o_{a,b}} \parallel (o_{a,b} \oplus \rho_{i_2}) \right), \quad a, b \in \{0, 1\}.$$

gde je $o_{a,b} = g_i(a \oplus \rho_{i_0}, b \oplus \rho_{i_1})$. Ova tabela sadrži četiri maskirane vrednosti, po jednu za svaku kombinaciju ulaza $a, b \in \{0, 1\}$. Svaka vrednost je maskirana tako da se može dešifrovati samo ako evaluator poseduje odgovarajuće ključeve $k_{w_{i_0}}^{a \oplus \rho_{i_0}}$ i $k_{w_{i_1}}^{b \oplus \rho_{i_1}}$. Deo $(o_{a,b} \oplus \rho_{i_2})$ služi za identifikaciju stvarne vrednosti izlazne žice kroz odgovarajući permutacioni bit.

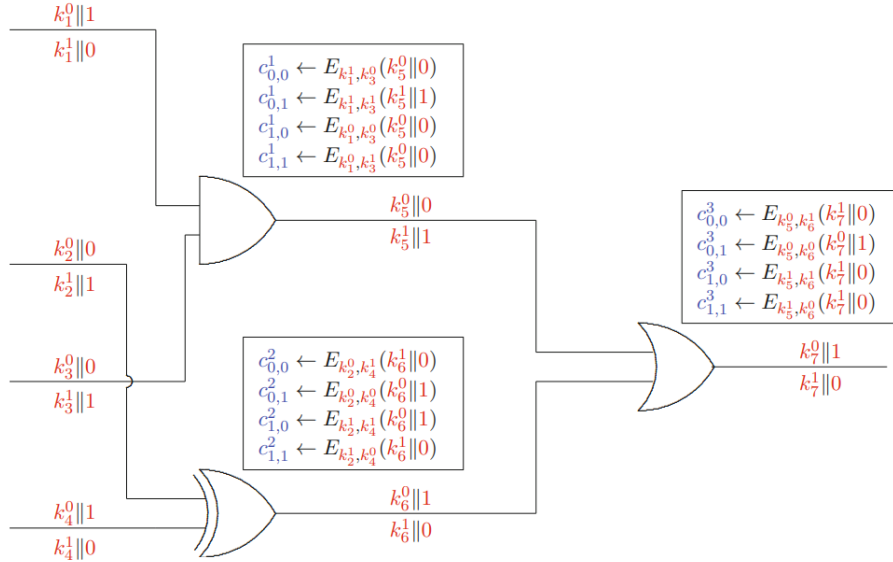
Nećemo precizirati tačnu konstrukciju funkcije enkripcije E , već pretpostavljamo da se radi o simetričnoj enkripciji koja je semantički sigurna, tako da napadač ne može da izvuče informacije o otvorenom tekstu iz šifrata bez poznavanja odgovarajućeg ključa.

Kako bismo bolje ilustrovali proces konstrukcije maskiranog kola, posmatrajmo konkretan primer. Pretpostavimo da učesnici A i B imaju po dva bita ulaza. Ulaze učesnika A označićemo žicama w_1 i w_2 , a ulaze učesnika B žicama w_3 i w_4 . Njihov cilj je da bezbedno izračunaju funkciju

$$f(\{w_1, w_2\}, \{w_3, w_4\}) = (w_1 \wedge w_3) \vee (w_2 \oplus w_4).$$

Na slici 2.1 prikazano je logičko kolo koje realizuje datu funkciju, zajedno sa maskiranim vrednostima žica i maskiranim tabelama pridruženim svakoj logičkoj kapiji. U ovom primeru permutacioni bitovi imaju vrednosti

$$\rho_1 = \rho_4 = \rho_6 = \rho_7 = 1, \quad \rho_2 = \rho_3 = \rho_5 = 0.$$



Slika 2.1: Maskirano kolo za $f(\{w_1, w_2\}, \{w_3, w_4\}) = (w_1 \wedge w_3) \vee (w_2 \oplus w_4)$ [31].

Posmatrajmo najpre žicu w_1 . Njoj su pridružena dva ključa, k_1^0 i k_1^1 , koji predstavljaju unutrašnje logičke vrednosti 0 i 1. Pošto je za ovu žicu izabrano $\rho_1 = 1$, spoljašnja reprezentacija unutrašnjih vrednosti 0 i 1 iznosi

$$0 \oplus \rho_1 = 1, \quad 1 \oplus \rho_1 = 0.$$

Zbog toga se žica w_1 predstavlja parom parova:

$$(k_1^0 || 1, k_1^1 || 0),$$

koji nam govori da ključu k_1^0 odgovara spoljašnja vrednost žice 1 koju koristimo pri izračunavanju logičke kapije u koju žica ulazi [31].

Sada posmatrajmo AND kapiju koja kao ulaz koristi žice w_1 i w_3 , a kao izlaz proizvodi žicu w_5 . Za ovu kapiju konstruiše se maskirana tabela sa četiri reda, po jedan za svaku moguću kombinaciju ulaza $a, b \in \{0, 1\}$, gde a i b predstavljaju moguće eksterne oznake ulaznih žica w_1 i w_3 koje su vezane za odgovarajuće ključeve [31].

Razmotrimo prvi red tabele, koji odgovara slučaju $a = b = 0$. Prema definiciji

konstrukcije, koriste se ključevi $k_1^{a \oplus \rho_1}$ i $k_3^{b \oplus \rho_3}$. Kako je $\rho_1 = 1$ i $\rho_3 = 0$, dobijamo $a \oplus \rho_1 = 0 \oplus 1 = 1$ i $b \oplus \rho_3 = 0 \oplus 0 = 0$, pa se za ovaj red koriste ključevi k_1^1 i k_3^0 [31].

Zatim se određuje izlaz kapije za odgovarajuće ulazne vrednosti. Pošto AND kapija na ulazu $(1, 0)$ daje rezultat $o_{a,b} = 0$, potrebno je maskirati ključ koji predstavlja logičku nulu na izlaznoj žici w_5 , odnosno ključ k_5^0 . Pored njega se čuva i spoljašnja reprezentacija izlaza. Kako je $\rho_5 = 0$, spoljašnja reprezentacija izlazne vrednosti 0 iznosi

$$o_{a,b} \oplus \rho_5 = 0 \oplus 0 = 0.$$

Zbog toga prvi red maskirane tabele sadrži maskirani šifrat $E_{k_1^1, k_3^0}(k_5^0 || 0)$ koji se može otvoriti jedino korišćenjem ključeva k_1^1 i k_3^0 , a njegov sadržaj predstavlja par

$$(k_5^0 || 0).$$

Na potpuno isti način konstruišu se i preostala tri reda tabele, kao i tabele za sve ostale kapije u kolu. Rezultat ovog postupka jeste potpuno maskirano kolo koje može biti poslato evaluatoru, bez otkrivanja stvarnih logičkih vrednosti koje se nalaze na žicama [31].

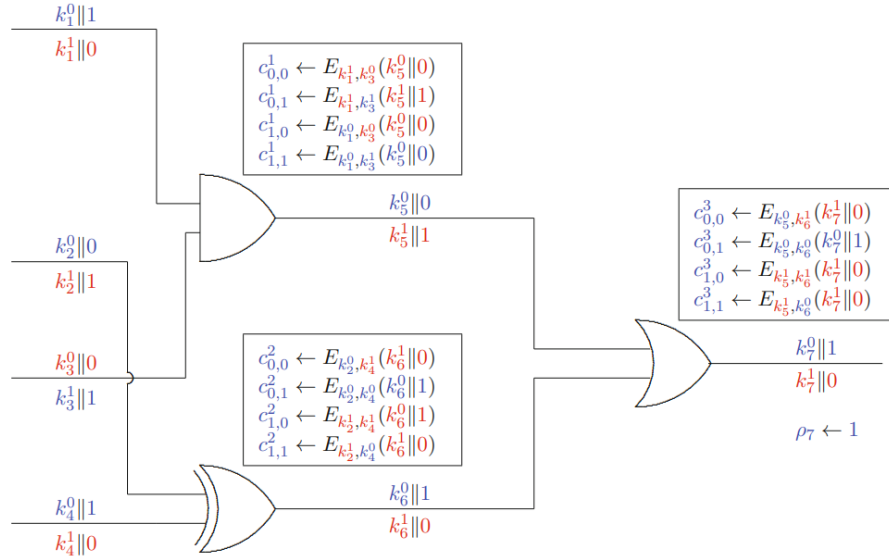
2.2.3 Evaluacija maskiranog kola

Evaluacija maskiranog kola je proces u kojem evaluator (u našem primeru, strana B) koristi maskiranu tabelu i ključeve koje poseduje da bi izračunao izlaz funkcije. Evaluacija se vrši tako što se kreće od ulaznih žica i koristi se maskirana tabela da bi se dobile vrednosti izlaznih žica, sve dok se ne dođe do izlaza funkcije [31].

Na slici 2.2 prikazan je proces evaluacije maskiranog kola. Pretpostavimo da B poseduje maskirane vrednosti ulaznih žica w_1, w_2, w_3, w_4 koje su redom $k_1^0, k_2^0, k_3^1, k_4^0$ (označene plavom bojom), kao i vrednosti ρ_i izlaznih žica, u ovom primeru poseduje vrednost $\rho_7 = 1$ [31].

B prvo evaluiira AND kapiju g_1 . Pošto zna da su spoljašnje vrednosti ulaznih žica w_1 i w_3 redom 1 i 1, u maskiranoj tabeli za AND kapiju pronalazi vrednost $c_{1,1}^1$ i dešifruje je koristeći ključeve k_1^1 i k_3^1 koje poseduje. Dešifrovanjem dobija vrednost $k_5^0 || 0$, pa nema način da sazna da li ova vrednost predstavlja logičku nulu ili jedinicu jer ne zna vrednost ρ_5 [31].

B sada primenjuje isti postupak evaluacije na XOR kapiju g_2 . Pošto zna da su spoljašnje vrednosti ulaznih žica w_2 i w_4 redom 1 i 0, pronalazi vrednost $c_{1,0}^2$ u maskiranoj tabeli za XNOR kapiju i dešifruje je koristeći ključeve k_2^0 i k_4^0 . Dešifrovanjem



Slika 2.2: Evaluacija maskiranog kola za $f(\{w_1, w_2\}, \{w_3, w_4\}) = (w_1 \wedge w_3) \vee (w_2 \oplus w_4)$ [31].

dobija vrednost $k_6^0 || 1$. Opet, on nema način da sazna da li ova vrednost predstavlja logičku nulu ili jedinicu jer ne zna vrednost ρ_6 [31].

Postupak se nastavlja evaluacijom OR kapije g_3 . Pošto zna da su spoljašnje vrednosti ulaznih žica w_5 i w_6 redom 0 i 1, pronalazi vrednost $c_{0,1}^3$ u maskiranoj tabeli za OR kapiju i dešifruje je koristeći ključeve k_5^0 i k_6^0 . Dešifrovanjem dobija vrednost $k_7^1 || 1$. Pošto je B dobio vrednost $\rho_7 = 1$, on izračunava unutrašnju vrednost izlazne žice w_7 kao $1 \oplus \rho_7 = 0$ [31].

2.2.4 Jaov protokol

Kada imamo definisanu konstrukciju maskiranog kola i proces evaluacije, možemo opisati Jaov protokol za sigurno izračunavanje sa dva učesnika. Protokol se sastoji od sledećih faza [31]:

1. Učesnik A konstruiše maskirano kolo i B -u šalje samo vrednosti $c_{a,b}^i$.
2. Učesnik A zatim šalje B -u šifrovane vrednosti komponenti svojih ulaznih žica. Na primer, pretpostavimo da su ulazi učesnika A $w_1 = 0$ i $w_2 = 0$. Tada učesnik A šalje B -u dve vrednosti k_1^0 i k_2^0 . Važno je primetiti da učesnik B ne može saznati stvarne vrednosti žica w_1 i w_2 iz ovih vrednosti jer ne zna ρ_1 i ρ_2 , a ključevi k_1^0 i k_2^0 mu samo izgledaju kao nasumični ključevi.

3. Učesnici A i B zatim izvršavaju protokol ignorantnog prenosa (engl. *oblivious transfer*, OT) za svaku ulaznu žicu učesnika B . Cilj ovog koraka je da učesnik B dobije tačno jedan od dva ključa koji odgovaraju njegovoj ulaznoj vrednosti, pri čemu učesnik A ne saznaje koji je ključ izabran, dok učesnik B ne dobija informaciju o drugom ključu. Na primer, ako su ulazi učesnika B $w_3 = 1$ i $w_4 = 0$, onda se za svaku žicu izvršava zaseban OT protokol nad parovima (k_3^0, k_3^1) i (k_4^0, k_4^1) . Kao rezultat, učesnik B dobija ključeve k_3^1 i k_4^0 .
4. Učesnik A zatim šalje B -u vrednosti ρ_i za sve izlazne žice. U našem primeru on otkriva vrednost $\rho_7 = 1$.
5. Na kraju, učesnik B evaluira kolo koristeći šifrovane vrednosti ulaznih žica koje je dobio, koristeći tehniku opisanu ranije.

Na početku protokola, sve što učesnik B zna o maskiranom kolu su plavi elementi na slici 2.1. Međutim, na kraju protokola on zna plave elemente na slici 2.2. U našem primeru možemo proceniti šta je učesnik B naučio iz izračunavanja. Učesnik B zna da je izlaz finalne OR kapije nula, što implicira da su ulazi te kapije dali posmatrani izlaz; to dalje implicira da je izlaz AND kapije nula i da je izlaz XOR kapije nula. Međutim, učesnik B je već znao da će izlaz AND kapije biti nula, jer njegov sopstveni ulaz bio nula. Stoga, učesnik B je naučio da druga ulazna žica učesnika A predstavlja nulu, jer inače XOR kapija ne bi dala nulu. Dakle, dok prvi ulaz učesnika A ostaje privatn, drugi ulaz ne ostaje, što ilustruje da protokol čuva privatnost ulaza do onoga što se može zaključiti iz izlaza funkcije [31].

2.3 Slučaj više učesnika: *pošteni ali radoznali* učesnici

U slučaju sa više učesnika osnovna ideja je da se funkcija koja se računa postavi kao aritmetičko kolo, a zatim se koristi šema deljenja tajni da bi se postigla sigurnost. Aritmetičko kolo se sastoji od konačnog polja $\mathbb{F}_q$ i polinomijalne funkcije, definisane nad konačnim poljem, koja može imati mnogo ulaza i izlaza. Ideja je da se takva funkcija može izračunati korišćenjem aritmetičkih kola za sabiranje i množenje nad konačnim poljem [31].

Može se pokazati da se aritmetička kola mogu predstaviti kao binarna kola proširivanjem svake sabiračke i množačke kapije u odgovarajuće binarne operacije. Obrnuto, svako binarno kolo može se posmatrati kao aritmetičko kolo nad konačnim

poljem $\mathbb{F}_q$, pri čemu se logičke vrednosti 0 i 1 identifikuju sa elementima tog polja, uz pretpostavku da je karakteristika polja veća od 2.

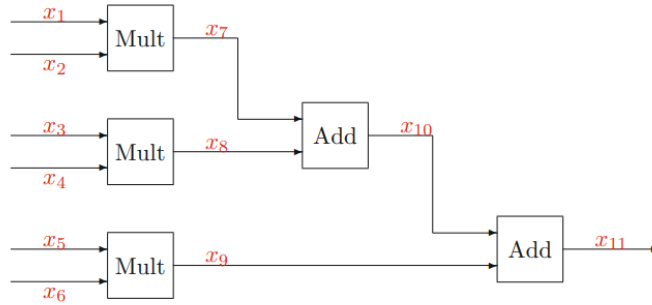
U toj interpretaciji, logičke operacije se mogu izraziti polinomima nad poljem. Na primer, za XOR kapiju važi $x \oplus y = -2xy + x + y$, dok se AND kapija može predstaviti kao $x \wedge y = xy$.

Iako su ove dve reprezentacije ekvivalentne, određene funkcije se prirodnije i efikasnije izražavaju kao aritmetička kola, dok su druge pogodnije za binarnu reprezentaciju [31].

Protokol ćemo predstaviti kroz konkretan primer. Pretpostavimo da imamo šest učesnika $P_1, \dots, P_6$ koji imaju šest tajnih vrednosti $x_1, \dots, x_6$, od kojih svaka pripada polju $\mathbb{F}_p$ za neki dovoljno veliki prost broj p . Na primer, mogli bismo uzeti $p \approx 2^{128}$, ali u našem primeru, da bismo lakše predstavili stvari, uzećemo $p = 101$. Pretpostavimo da učesnici žele da izračunaju vrednost funkcije

$$f(x_1, \dots, x_6) = x_1x_2 + x_3x_4 + x_5x_6 \pmod{p}.$$

Aritmetičko kolo ove funkcije sastoji se od dva sabiračka i tri množačka kola, i predstavljeno je na slici 2.3.



Slika 2.3: Aritmetičko kolo [31].

Oslonićemo se na Šamirovu (eng. *Shamir's*) šemu deljenja tajni da bismo postigli sigurnost. U osnovnom protokolu, vrednost svake žice x_i se deli između svih učesnika, pri čemu svaki učesnik j dobija deo $x_i^{(j)}$. Jasno je da ako se dovoljno učesnika udruži, onda mogu odrediti vrednost žice x_i zbog svojstava šeme deljenja tajni [31].

Svaki učesnik može kreirati delove svojih sopstvenih ulaznih vrednosti na početku protokola i poslati deo ostalim učesnicima. Dakle, potrebno je pokazati kako dobiti delove izlaza kapije, s obzirom na delove ulaza kapije. Podsećamo da se u Šamirovoj šemi deljenja tajni deljena vrednost daje kao konstantni član polinoma f stepena

t , pri čemu su delovi evaluacija polinoma na pozicijama koje odgovaraju svakom učesniku [31].

2.3.1 Sabirači

Pretpostavimo da imamo dve tajne vrednosti a i b koje su deljene između učesnika pomoću polinoma

$$f(X) = a + f_1X + \cdots + f_tX^t, \quad g(X) = b + g_1X + \cdots + g_tX^t.$$

Svaki učesnik i poseduje vrednosti $a^{(i)} = f(i)$ i $b^{(i)} = g(i)$. Cilj je konstruisati deljenje sume $c = a + b$, odnosno pronaći vrednosti $c^{(i)}$ koje odgovaraju nekom polinomu $h(X)$ takvom da je $h(0) = c$.

Razmatramo polinom

$$h(X) = f(X) + g(X).$$

Iz linearnosti polinoma sledi da je $h(X)$ takođe polinom stepena najviše t , jer sabiranje ne povećava stepen. Posebno, u nuli važi

$$h(0) = f(0) + g(0) = a + b = c.$$

Zato je $h(X)$ validan polinom koji enkodira traženu tajnu vrednost c u istom tipu deljenja.

Svaki učesnik i može lokalno izračunati

$$c^{(i)} = h(i) = f(i) + g(i) = a^{(i)} + b^{(i)}.$$

Korektnost postupka zasniva se na činjenici da evaluacija polinoma u tački komutira sa sabiranjem, odnosno da za svaki i važi

$$(f + g)(i) = f(i) + g(i).$$

Zbog toga skup vrednosti $\{c^{(i)}\}$ predstavlja validno deljenje tajne c istom šemom polinomskog deljenja kao i originalne vrednosti a i b , uz očuvanje stepena polinoma i bez potrebe za komunikacijom između učesnika [31].

2.3.2 Množači

Izračunavanje izlaza aritmetičkih kola za operaciju množenja je složenije. Podsećamo se osobine Lagranžove interpolacije polinoma.

Neka je $f(X)$ polinom nad nekim poljem, i neka su poznate njegove vrednosti u tačkama $1, 2, \dots, n$. Tada postoji vektor koeficijenata $(r_1, \dots, r_n)$, koji se naziva vektor rekombinacije, takav da važi

$$f(0) = \sum_{i=1}^n r_i f(i).$$

Ovaj vektor zavisi samo od izabranih tačaka evaluacije, a ne od samog polinoma, i važi za svaki polinom stepena najviše $n - 1$ [31].

Za izračunavanje izlaza množenja koristi se sledeći postupak rekonstrukcije u okviru deljenja zasnovanog na polinomima. Pretpostavimo da svaka strana poseduje delove vrednosti a i b u obliku $a^{(i)} = f(i)$ i $b^{(i)} = g(i)$, gde važi $f(0) = a$ i $g(0) = b$. Cilj je konstruisati deljenje $c^{(i)} = h(i)$ tako da je $h(0) = c = a \cdot b$ [31].

- Svaka strana lokalno računa $d^{(i)} = a^{(i)} \cdot b^{(i)}$.
- Svaka strana konstruiše polinom $\delta_i(X)$ stepena najviše t takav da važi $\delta_i(0) = d^{(i)}$.
- Strana i šalje vrednosti $d_{i,j} = \delta_i(j)$ strani j .
- Svaka strana j rekonstruiše vrednost

$$c^{(j)} = \sum_{i=1}^n r_i d_{i,j}.$$

Ispravnost se zasniva na svojstvu Lagranžove interpolacije. Ako je stepen polinoma ograničen sa $2t \leq n - 1$, tada vektor rekombinacije $r_1, \dots, r_n$ omogućava rekonstrukciju

$$h(0) = \sum_{i=1}^n r_i d^{(i)}.$$

Definišemo

$$h(X) = \sum_{i=1}^n r_i \delta_i(X),$$

pri čemu je $\deg(h) \leq t$ i važi

$$h(0) = \sum_{i=1}^n r_i \delta_i(0) = \sum_{i=1}^n r_i d^{(i)} = c.$$

Za svako j dodatno važi

$$h(j) = \sum_{i=1}^n r_i \delta_i(j) = \sum_{i=1}^n r_i d_{i,j} = c^{(j)}.$$

Primer 2.3.1. Ovaj primer ilustruje kompletan tok verifikovanog aritmetičkog MPC protokola: inicijalno deljenje tajni, evaluaciju multiplikacionih i adicionih kapija, kao i finalnu rekonstrukciju rezultata. Razmatra se polje $\mathbb{F}_{101}$ i šest učesnika sa ulazima

$$x_1 = 20, x_2 = 40, x_3 = 21, x_4 = 31, x_5 = 1, x_6 = 71.$$

Svaki učesnik formira Šamirovo deljenje svog ulaza polinomom stepena $t = 2$ i distribuira evaluacije svim učesnicima, čime se dobija matrica deljenja prikazana u Tabeli 2.2.

j	1	2	3	4	5	6
1	44	2	96	23	86	83
2	26	0	63	13	52	79
3	4	22	75	62	84	40
4	93	48	98	41	79	10
5	28	35	22	90	37	65
6	64	58	53	49	46	44

Tabela 2.2: Delovi ulaznih vrednosti učesnika [31].

Multiplikaciona kapija se evaluira primenom lokalne multiplikacije i rekonstrukcije deljenja, što daje delove promenljive $x_7 = x_1 x_2$. Svaki učesnik dobija odgovarajući deo, što daje rezultate:

$$x_7^{(1)} = 9, \quad x_7^{(2)} = 97, \quad x_7^{(3)} = 54, \quad x_7^{(4)} = 82, \quad x_7^{(5)} = 80, \quad x_7^{(6)} = 48.$$

Analogno se dobijaju deljenja za x_8 i x_9 .

Adicione kapije se evaluiraju lokalno sabiranjem odgovarajućih delova, na primer:

$$x_{11}^{(1)} = x_7^{(1)} + x_8^{(1)} + x_9^{(1)} \bmod 101 = 92,$$

što se ponavlja za sve učesnike, dobijajući:

$$x_{11}^{(1)} = 92, \quad x_{11}^{(2)} = 63, \quad x_{11}^{(3)} = 21, \quad x_{11}^{(4)} = 67, \quad x_{11}^{(5)} = 100, \quad x_{11}^{(6)} = 19.$$

Finalno, učesnici objavljuju svoje delove i rekonstruišu polinom stepena $t = 2$, iz kojeg se dobija konstantni član 7, koji predstavlja rezultat MPC izračunavanja [31].

Pretpostavimo da je broj korumpiranih učesnika veći od t . U tom slučaju, korumpirani učesnici mogu se udružiti i rekonstruisati bilo koju od tajni u šemi, jer su deljenja zasnovana na polinomima stepena najviše t . Međutim, može se pokazati, koristeći savršenu tajnost Šamirove šeme deljenja tajni, da sve dok je broj korumpiranih učesnika manji ili jednak t , gore opisani protokol je savršeno siguran [31].

U slučaju da su svi učesnici *pošteni ali radoznali*, gore opisani protokol je dovoljan za sigurno izračunavanje. Ukoliko učesnici ne slede protokol, mogu namestiti nevalidne rezultate poštenim učesnicima. Da bismo to pokazali, dovoljno je primetiti da bi zlonamerni učesnik mogao jednostavno da proizvede nevalidno deljenje svog proizvoda u drugom delu multiplikacionog protokola opisanog ranije[31].

2.4 Model više učesnika: zlonamerni učesnici

U prethodnom modelu pretpostavili smo da svi učesnici korektno izvršavaju protokol. Međutim, zlonamerni učesnici mogu slati proizvoljne vrednosti i time uticati na rezultat izračunavanja. Posebno je problematičan protokol za množenje, jer učesnik može poslati neispravne delove tajne i tako izazvati pogrešan izlaz.

Zbog toga je potrebno omogućiti otkrivanje i ispravljanje grešaka koje uvode zlonamerni učesnici. Ovo se postiže korišćenjem Rid–Solomonovih kodova (eng. *Reed–Solomon codes*), koji proizlaze iz polinomske strukture Šamirove šeme deljenja tajni, kao i unapred generisanih Biverovih (eng. *Beaver's*) trojki koje omogućavaju bezbedno izvršavanje množenja [31].

Protokol se sastoji iz dve faze. U fazi pripreme (eng. *preprocessing*) generišu se pomoćne vrednosti nezavisne od ulaza učesnika, dok se u glavnoj fazi (eng. *main phase*) evaluira samo kolo.

2.4.1 Faza pripreme

U fazi pripreme, najpre se uspostavljaju pseudoslučajna šema deljenja tajni (eng. *pseudorandom secret sharing scheme* - *PRSS*) i pseudoslučajna šema deljenja nule (eng. *pseudorandom zero-sharing scheme* - *PRZS*), nakon čega se za svaki multiplikacioni čvor generiše Biverova trojka, koja je nasumična trojka deljenja $a^{(i)}, b^{(i)}, c^{(i)}$ takva da je $c = a \cdot b$.

To se postiže pomoću sledećih koraka [31]:

- Koristeći PRSS generišu se dva nasumična deljenja $a^{(i)}$ i $b^{(i)}$ stepena t .
- Koristeći PRSS generiše se još jedno nasumično deljenje $r^{(i)}$ stepena t .
- Koristeći PRZS generiše se deljenje nule $z^{(i)}$ stepena $2t$.
- Svaki učesnik lokalno izračunava vrednost

$$s^{(i)} = a^{(i)} \cdot b^{(i)} - r^{(i)} + z^{(i)}.$$

Ovo lokalno izračunavanje daje deljenje stepena $2t$ vrednosti $s = a \cdot b - r$.

- Zatim učesnici šalju svoje vrednosti $s^{(i)}$ i pokušavaju da rekonstruišu vrednost s . Ovde koristimo svojstva detekcije grešaka Rid–Solomonovih kodova. Ako je broj zlonamernih učesnika ograničen sa $t < n/3$, svaka greška u deljenju stepena $2t$ biće otkrivena. U ovoj fazi protokol se prekida ako se otkrije bilo kakva greška.
- Nakon toga učesnici lokalno izračunavaju delove $c^{(i)}$ po formuli

$$c^{(i)} = s + r^{(i)}.$$

Ukoliko se tokom prve faze otkrije nekonzistentnost, protokol se prekida. U suprotnom nastavlja se na glavnu fazu. Jedina razlika u glavnoj fazi u odnosu na protokol za *poštene ali radoznale* učesnike je u načinu evaluacije množioca [31].

2.4.2 Glavna faza

Pretpostavimo da su ulazi u množiocu predstavljeni delovima $x^{(i)}$ i $y^{(i)}$, i želimo da dobijemo deljenje $z^{(i)}$ proizvoda $z = x \cdot y$. Iz faze prethodne obrade, za svaki čvor učesnici takođe imaju trojku deljenja $a^{(i)}, b^{(i)}, c^{(i)}$ takvu da je $c = a \cdot b$. Protokol množenja glasi [31]:

- Svaki učesnik lokalno izračunava vrednosti

$$d^{(i)} = x^{(i)} - a^{(i)}, \quad e^{(i)} = y^{(i)} - b^{(i)},$$

i šalje ih ostalim učesnicima.

- Na osnovu primljenih vrednosti učesnici rekonstruišu

$$d = x - a, \quad e = y - b.$$

- Svaki učesnik zatim lokalno izračunava svoj deo izlaza

$$z^{(i)} = d \cdot e + d \cdot b^{(i)} + e \cdot a^{(i)} + c^{(i)}.$$

Vredno je napomenuti da se rekonstrukcija u drugom koraku može izvršiti čak i ako korumpirani učesnici pošalju nevažeće vrednosti, pod uslovom da postoji najviše $t < n/3$ zlonamernih učesnika. Zahvaljujući svojstvima ispravljanja grešaka Rid-Solomonovih kodova, možemo se oporaviti od bilo kakvih grešaka koje unesu zlonamerni učesnici. Gornji protokol proizvodi validno deljenje izlaza množioca zato što važi [31]

$$de + db + ea + c = (x - a)(y - b) + (x - a)b + (y - b)a + c = ((x - a) + a)((y - b) + b) = xy = z.$$

Na ovaj način, operacija množenja svodi se na rekonstrukciju dve javne vrednosti d i e , dok se ostatak izračunavanja obavlja nad deljenim vrednostima. Zahvaljujući mogućnosti detekcije i ispravljanja grešaka, protokol ostaje ispravan čak i u prisustvu do $t < n/3$ zlonamernih učesnika.

2.5 Zaključak

Bezbedno izračunavanje sa više učesnika (MPC) predstavlja jednu od najznačajnijih oblasti moderne kriptografije, koja omogućava zajedničku obradu podataka uz strogo očuvanje privatnosti privatnih ulaza učesnika. Kroz ovo poglavlje smo analizirali razvoj od Jaovog protokola za dva učesnika, zasnovanog na binarnim **maskiranim kolima** i ignorantnom prenosu, do skalabilnih sistema za više učesnika koji koriste aritmetička kola i Šamirovo deljenje tajni. Razmatranjem različitih modela protivnika, od poštenih ali radoznalih do zlonamernih MPC protokoli postižu neophodnu ravnotežu između teoretske bezbednosti i praktične efikasnosti, čime se

eliminise potreba za centralnim autoritetom od poverenja.

2.6 Pitanja

1. Šta je osnovni cilj protokola za bezbedno izračunavanje sa više učesnika (MPC)?
2. Šta je i kako se konstruiše šifrovano binarno kolo?
3. Šta je i kako se konstruiše šifrovano aritmetičko kolo?

Glava 3

Certificates

3.1 Uvod

Jedan od osnovnih problema u kriptografiji jeste uspostavljanje zajedničkog ključa između strana koje žele da komuniciraju. U simetričnoj kriptografiji obe strane koriste isti tajni ključ. Taj ključ mora biti poznat samo učesnicima komunikacije. Ako strane nemaju unapred uspostavljen bezbedan kanal, postavlja se pitanje kako tajni ključ mogu bezbedno razmeniti.

Kriptografija javnog ključa ublažava ovaj problem. Svaki korisnik ima par ključeva: javni i privatni ključ. Javni ključ može biti dostupan svima, dok privatni ključ mora ostati poznat samo vlasniku. Pošiljalac zato ne mora unapred da deli tajni ključ sa primaocem. Umesto toga, javni ključ može da se koristi za uspostavljanje novog simetričnog ključa ili za proveru digitalnog potpisa [31].

U ovom kontekstu razlikuju se dugoročni i kratkoročni ključevi. Dugoročni, odnosno statički ključevi, koriste se tokom dužeg vremenskog perioda. Njihovo kompromitovanje može imati ozbiljne posledice, jer može ugroziti poverenje u identitet korisnika i buduću komunikaciju. Kratkoročni, odnosno sesijski ili efemerni ključevi, koriste se za jednu komunikacionu sesiju ili tokom ograničenog vremenskog perioda. Ako je kompromitovan jedan sesijski ključ, posledice bi trebalo da budu ograničene na tu sesiju [31].

Međutim, javni ključ je bezbedno upotrebljiv samo ako postoji pouzdan način da se utvrdi kome pripada. Sama dostupnost javnog ključa nije dovoljna, jer napadač može objaviti svoj javni ključ i predstaviti ga kao ključ drugog korisnika. Ako pošiljalac prihvati takav ključ kao ispravan, bezbednost daljeg protokola može biti narušena [31].

Zbog toga su u sistemima javnog ključa potrebni mehanizmi za proveru autentičnosti javnih ključeva. Tu ulogu imaju digitalni sertifikati. Oni povezuju identitet subjekta sa njegovim javnim ključem, a tu vezu potvrđuje poverljiva treća strana, najčešće sertifikaciono telo, odnosno *Certificate Authority* (CA) [31, 9].

3.2 Sertifikati i sertifikaciona tela

Kriptografija javnog ključa rešava deo problema distribucije ključeva, ali ne rešava sama po sebi pitanje poverenja. Korisnik mora imati pouzdan način da proveri da javni ključ zaista pripada subjektu sa kojim želi da komunicira.

Zbog toga je u sistemima javnog ključa jedan od osnovnih pojmova vezivanje javnog ključa.

Definicija 3.2.1. Vezivanje javnog ključa *binding* je postupak kojim se javni ključ povezuje sa određenim subjektom. Subjekt može biti osoba, računar, server, organizacija ili proces [31].

Tu potvrdu nam daje digitalni sertifikat.

Definicija 3.2.2. Digitalni sertifikat je digitalno potpisana struktura podataka koja sadrži javni ključ subjekta i podatke koji identifikuju vlasnika sertifikata, kao što su ime, naziv organizacije i period važenja, čime se javni ključ povezuje sa određenim subjektom [9, 16].

Vrednost sertifikata potiče iz činjenice da tu vezu potvrđuje strana kojoj se veruje. U klasičnom modelu to je poverljiva treća strana, odnosno *Trusted Third Party* (TTP). U kontekstu digitalnih sertifikata tu ulogu najčešće ima sertifikaciono telo *Certificate Authority* (CA) [31].

Sertifikat sam po sebi ne garantuje bezbednost ukoliko privatni ključ subjekta nije zaštićen. Ako je privatni ključ kompromitovan, napadač može zloupotребiti sertifikat i predstaviti se kao njegov vlasnik [16]. Zbog toga se poverenje u sertifikat ne zasniva samo na njegovom sadržaju, već i na zaštiti privatnih ključeva, pouzdanosti sertifikacionog tela i pravilima po kojima se sertifikati izdaju, proveravaju i opozivaju.

Definicija 3.2.3. Sertifikaciono telo, *Certificate Authority* (CA), jeste poverljivi autoritet koji izdaje digitalne sertifikate i svojim digitalnim potpisom potvrđuje da određeni javni ključ pripada određenom subjektu [31].

Korisnik koji proverava sertifikat ne proverava neposredno identitet subjekta, već proverava potpis sertifikacionog tela. Ako korisnik veruje javnom ključu tog tela, onda može da proveri da li je sertifikat zaista potpisalo to telo i da li su podaci u sertifikatu ostali neizmenjeni. Zbog toga je zaštita privatnog ključa sertifikacionog tela od posebnog značaja. Ako bi taj ključ bio kompromitovan, napadač bi mogao da izdaje sertifikate koji izgledaju kao da dolaze od pouzdanog autoriteta.

U manjim sistemima moguće je da postoji samo jedno sertifikaciono telo kojem svi učesnici veruju. U većim sistemima to nije dovoljno praktično. Različite organizacije, domeni ili države mogu imati sopstvena sertifikaciona tela, pa se postavlja pitanje kako uspostaviti poverenje između njih. Jedan od načina za to je unakrsna sertifikacija.

Definicija 3.2.4. Unakrsna sertifikacija, odnosno *cross-certification*, je postupak u kojem jedno sertifikaciono telo izdaje sertifikat drugom, čime se uspostavlja odnos poverenja između njih [9].

Takav odnos poverenja mora biti jasno određen. Kada jedno sertifikaciono telo prihvati drugo, ono omogućava da se poverenje prenese i na sertifikate koje izdaje to drugo telo. Zbog toga se kod sertifikata sertifikacionih tela koriste pravila i ograničenja koja određuju za koje namene sertifikat može da se koristi i u kojoj meri poverenje može dalje da se prenosi.

Definicija 3.2.5. Lanac sertifikata, *certification path*, je niz sertifikata kojim se poverenje prenosi od unapred poznatog poverljivog javnog ključa do sertifikata krajnjeg subjekta [9].

Početna tačka lanca naziva se sidro poverenja. To je sertifikaciono telo kojem korisnik unapred veruje. Korisnik zatim proverava sertifikate redom, od tog tela do krajnjeg subjekta. Ako su potpisi ispravni, sertifikati važeći i ograničenja ispunjena, javni ključ krajnjeg subjekta može se smatrati autentičnim [9].

Da bi se sertifikati koristili na isti način u različitim sistemima, potreban je zajednički standard. Najznačajniji standard za digitalne sertifikate je X.509, koji određuje njihov format, osnovna polja i način provere [9]. X.509 sertifikat sadrži podatke o subjektu, njegov javni ključ, podatke o izdavaocu, period važenja i digitalni potpis, dok ekstenzije u verziji v3 omogućavaju da se odredi namena ključa i ograničenja upotrebe sertifikata [9].

Iako se sertifikat izdaje za određeni period važenja, on može postati nepouzdan i pre isteka tog perioda. To se dešava, na primer, ako je privatni ključ kompromitovan, ako se promeni veza između subjekta i sertifikacionog tela, ili ako podaci u sertifikatu više nisu tačni [9]. U takvim slučajevima sertifikat se opoziva.

Jedan od osnovnih načina za proveru opoziva jeste lista opozvanih sertifikata, odnosno *Certificate Revocation List* (CRL). CRL je potpisana lista opozvanih sertifikata, pri čemu se svaki sertifikat prepoznaje po svom serijskom broju [9].

3.3 Generisanje efemernih simetričnih ključeva iz statičkih simetričnih ključeva

U ovim protokolima dugoročni simetrični ključevi služe za uspostavljanje novog sesijskog ključa, a ne za zaštitu celokupne komunikacije. Nakon uspostavljanja, sesijski ključ se koristi za jednu komunikacionu sesiju ili tokom ograničenog vremenskog perioda. Time se smanjuju posledice kompromitovanja ključa, jer se napad na jedan sesijski ključ ograničava na komunikaciju koja je tim ključem zaštićena.

Koristimo sledeće oznake:

- A i B označavaju strane koje žele da uspostave zajednički ključ,
- S označava poverljivu treću stranu, odnosno server za distribuciju ključeva,
- K_{AS} , K_{BS} i K_{AB} označavaju tajne simetrične ključeve poznate redom stranama A i S , B i S , odnosno A i B ,
- $\{M\}_K$ označava poruku M šifrovanu ključem K ,
- N_A i N_B označavaju brojeve koji se koriste samo jednom, odnosno *nonce* vrednosti,
- T_A , T_B i T_S označavaju vremenske oznake[31].

3.3.1 Wide-Mouth Frog protokol

Wide-Mouth Frog protokol je jednostavan protokol za raspodelu sesijskog ključa. Strana A bira novi sesijski ključ K_{AB} , a zatim ga preko poverljive treće strane S dostavlja strani B . Protokol se sastoji iz dve poruke:

$$\begin{aligned} A &\rightarrow S : A, \{T_A, B, K_{AB}\}_{K_{AS}} \\ S &\rightarrow B : \{T_S, A, K_{AB}\}_{K_{BS}} \end{aligned}$$

U prvoj poruci A šalje serveru svoj identitet i šifrovani deo koji sadrži vremensku oznaku T_A , identitet strane B i sesijski ključ K_{AB} . Ako server proceni da je vremenska

oznaka sveža, prihvata zahtev i šalje strani B poruku šifrovanu ključem K_{BS} . Kada je B dešifruje, dobija informaciju da strana A želi da koristi ključ K_{AB} za dalju komunikaciju [31].

Prednost ovog protokola je jednostavnost, jer ima samo dve poruke. Ipak, protokol zavisi od vremenskih oznaka, pa satovi učesnika moraju biti dovoljno usklađeni. Osim toga, sesijski ključ bira strana A , dok ga server samo prosleđuje. Zbog toga B mora da veruje da je A izabrala dobar i zaista nov ključ [31].

3.3.2 Needham–Schroeder protokol

Needham–Schroeder protokol se razlikuje od Wide-Mouth Frog protokola po tome što sesijski ključ ne bira strana A , već ga generiše poverljiva treća strana S . Na taj način se poverenje u izbor ključa premešta sa učesnika komunikacije na server [31].

Do konačnog oblika protokola dolazi se postepeno [31]. Najjednostavnija ideja je da A zatraži ključ od servera, da server pošalje ključ A , a da ga zatim A prosledi strani B :

$$\begin{aligned} A &\rightarrow S : A, B \\ S &\rightarrow A : K_{AB} \\ A &\rightarrow B : K_{AB}, A \end{aligned}$$

Ovakav protokol nije bezbedan, jer se ključ K_{AB} šalje otvoreno. Zato se uvodi šifrovanje pomoću dugoročnih ključeva koje učesnici dele sa serverom:

$$\begin{aligned} A &\rightarrow S : A, B \\ S &\rightarrow A : \{K_{AB}\}_{K_{AS}}, \{K_{AB}\}_{K_{BS}} \\ A &\rightarrow B : \{K_{AB}\}_{K_{BS}}, A \end{aligned}$$

Ova verzija čuva tajnost ključa, ali ne sprečava ponavljanje stare poruke. Napadač može sačuvati poruku koju A šalje strani B i kasnije je ponovo poslati. Zbog toga B ne zna da li je ključ nov ili potiče iz stare sesije.

U sledećem koraku se u šifrovane poruke dodaju identiteti učesnika:

$$\begin{aligned} A &\rightarrow S : A, B \\ S &\rightarrow A : \{K_{AB}, A\}_{K_{BS}}, \{K_{AB}, B\}_{K_{AS}} \\ A &\rightarrow B : \{K_{AB}, A\}_{K_{BS}} \end{aligned}$$

Na taj način se određuje kome je ključ namenjen. Strana B vidi da je ključ

namenjen za komunikaciju sa stranom A . Ipak, problem svežine nije potpuno rešen, jer B i dalje ne zna da li je ključ K_{AB} nov.

Zbog toga se uvodi nonce vrednost N_A . Ona omogućava strani A da proveri da odgovor servera pripada njenom trenutnom zahtevu:

$$\begin{aligned} A &\rightarrow S : A, B, N_A \\ S &\rightarrow A : \{N_A, B, K_{AB}, \{K_{AB}, A\}_{K_{BS}}\}_{K_{AS}} \\ A &\rightarrow B : \{K_{AB}, A\}_{K_{BS}} \end{aligned}$$

Pošto se N_A nalazi u odgovoru servera, A zna da odgovor nije preuzet iz stare sesije. Međutim, to ne daje isti dokaz strani B , jer ona dobija samo deo $\{K_{AB}, A\}_{K_{BS}}$.

Konačna verzija protokola dodaje još dva koraka za proveru da obe strane poseduju isti sesijski ključ:

$$\begin{aligned} A &\rightarrow S : A, B, N_A \\ S &\rightarrow A : \{N_A, B, K_{AB}, \{K_{AB}, A\}_{K_{BS}}\}_{K_{AS}} \\ A &\rightarrow B : \{K_{AB}, A\}_{K_{BS}} \\ B &\rightarrow A : \{N_B\}_{K_{AB}} \\ A &\rightarrow B : \{N_B - 1\}_{K_{AB}} \end{aligned}$$

U prvoj poruci A traži od servera ključ za komunikaciju sa B , a N_A povezuje zahtev sa odgovorom. Server zatim šalje novi sesijski ključ K_{AB} , identitet strane B i isti nonce N_A . U okviru odgovora nalazi se i deo $\{K_{AB}, A\}_{K_{BS}}$, koji je namenjen strani B . Strana A ga samo prosleđuje.

Poslednje dve poruke služe za potvrdu posedovanja ključa. Strana B šalje nonce N_B , šifrovan sesijskim ključem K_{AB} , a A vraća vrednost $N_B - 1$. Time pokazuje da poseduje isti sesijski ključ.

Slabost protokola je što strana B ne dobija direktan dokaz da je ključ K_{AB} svež. Ako napadač kasnije sazna stari sesijski ključ i ponovo pošalje staru treću poruku, B može prihvatiti stari ključ kao da je nov. Zato ovaj protokol pokazuje da tajnost ključa nije dovoljna. Svaka strana mora imati dokaz da je ključ namenjen baš toj sesiji [31].

3.3.3 Kerberos

Kerberos polazi od slične ideje kao Needham–Schroeder protokol, ali proveru svežine zasniva na vremenskim oznakama i ograničenom vremenu važenja ključa. Umesto da koristi samo nonce vrednosti, Kerberos uvodi tiket, odnosno šifrovanu potvrdu koju strana A može da pokaže strani B [31].

Pojednostavljen oblik protokola može se zapisati na sledeći način:

$$\begin{aligned} A &\rightarrow S : A, B \\ S &\rightarrow A : \{T_S, L, K_{AB}, B, \{T_S, L, K_{AB}, A\}_{K_{BS}}\}_{K_{AS}} \\ A &\rightarrow B : \{T_S, L, K_{AB}, A\}_{K_{BS}}, \{A, T_A\}_{K_{AB}} \\ B &\rightarrow A : \{T_A + 1\}_{K_{AB}} \end{aligned}$$

U prvoj poruci A traži od servera S pristup strani, odnosno resursu, B . Server zatim šalje sesijski ključ K_{AB} , vremensku oznaku T_S i period važenja L . Ovi podaci određuju koliko dugo se ključ i tiket mogu koristiti.

U odgovoru servera nalazi se i tiket $\{T_S, L, K_{AB}, A\}_{K_{BS}}$, koji je namenjen strani B . Strana A ga ne menja, već ga prosleđuje strani B . Uz tiket šalje i potvrdu $\{A, T_A\}_{K_{AB}}$, kojom pokazuje da zna sesijski ključ i da trenutno učestvuje u komunikaciji.

Ako je tiket važeći i vremenska oznaka nije zastarela, B prihvata ključ K_{AB} . U poslednjoj poruci B vraća vrednost $\{T_A + 1\}_{K_{AB}}$, čime potvrđuje da poseduje isti sesijski ključ.

Kerberos smanjuje mogućnost ponavljanja starih poruka tako što uvodi vremensko ograničenje tiketa. Međutim, zbog oslanjanja na vremenske oznake, zahteva da satovi učesnika budu dovoljno usklađeni. Ako vreme nije sinhronizovano, ispravni tiketi mogu biti odbačeni ili se može pogrešno proceniti njihova svežina [31].

3.4 Generisanje efemernih simetričnih ključeva iz statičkih javnih ključeva

U prethodnim protokolima sesijski ključ se uspostavlja pomoću dugoročnih simetričnih ključeva i poverljive treće strane. U ovom delu razmatra se drugačiji pristup, u kojem strane koriste statičke javne ključeve za uspostavljanje novog simetričnog ključa za jednu sesiju [31].

Ovakav pristup smanjuje potrebu za unapred deljenim tajnama. Međutim, javni

ključ mora biti autentičan, pa se ovi protokoli oslanjaju na digitalne sertifikate i infrastrukturu javnog ključa [31, 9].

U X.509 sertifikatima namena javnog ključa može biti posebno označena. Javni ključ može biti namenjen za prenos ključa, odnosno *key transport*, ili za dogovor ključa, odnosno *key agreement*. Kod prenosa ključa jedna strana bira ključ i šalje ga drugoj strani. Kod dogovora ključa obe strane učestvuju u izračunavanju zajedničkog ključa [31, 9].

Koristimo sledeće oznake:

- A i B označavaju strane koje žele da uspostave zajednički sesijski ključ,
- pk_A i pk_B označavaju javne ključeve strana A i B ,
- sk_A i sk_B označavaju njihove privatne ključeve,
- $Enc_{pk}(M)$ označava šifrovanje poruke M javnim ključem pk ,
- $Sig_{sk}(M)$ označava digitalni potpis poruke M privatnim ključem sk ,
- G označava cikličku grupu reda q , a g njen generator,
- H označava funkciju za izvođenje ključa iz zajedničke tajne.

3.4.1 Prenos ključa pomoću javnog ključa

Najjednostavniji način upotrebe javnog ključa za uspostavljanje sesijskog ključa jeste prenos ključa. Strana A generiše novi simetrični ključ K_{AB} , šifruje ga javnim ključem strane B i šalje ga strani B [31]:

$$A \rightarrow B : Enc_{pk_B}(K_{AB})$$

Poruku može da dešifruje samo strana B , jer samo ona poseduje odgovarajući privatni ključ sk_B . Nakon toga strane A i B imaju zajednički simetrični ključ za dalju komunikaciju. Ovakav pristup podseća na hibridno šifrovanje. Javni ključ se koristi samo za zaštitu sesijskog ključa, dok se podaci kasnije šifruju simetričnim algoritmom [31, 32].

Međutim, ova verzija ne obezbeđuje autentifikaciju pošiljaoca. Strana B može da dešifruje ključ, ali ne zna da li ga je zaista poslala strana A . Napadač E može generisati svoj ključ K_{EB} , šifrovati ga javnim ključem strane B i poslati poruku kao da dolazi od A [31]:

$$E \rightarrow B : \text{Enc}_{pk_B}(K_{EB})$$

U tom slučaju B dobija ispravno šifrovan ključ, ali ga pogrešno povezuje sa stranom A . Zato se uvodi digitalni potpis:

$$A \rightarrow B : c = \text{Enc}_{pk_B}(K_{AB}), \text{Sig}_{sk_A}(c)$$

Strana B proverava potpis pomoću javnog ključa pk_A . Ako je potpis ispravan, B zna da je šifrat poslala strana koja poseduje privatni ključ sk_A . Ipak, ni ova verzija nije dovoljna, jer identitet pošiljaoca nije vezan za sadržaj šifrata. Napadač E može presresti šifrat c , ukloniti potpis strane A , potpisati isti šifrat svojim privatnim ključem i proslediti ga strani B [31]:

$$\begin{aligned} A \rightarrow E : c &= \text{Enc}_{pk_B}(K_{AB}), \text{Sig}_{sk_A}(c) \\ E \rightarrow B : c &= \text{Enc}_{pk_B}(K_{AB}), \text{Sig}_{sk_E}(c) \end{aligned}$$

Strana B tada može zaključiti da ključ deli sa napadačem E , iako je ključ zapravo izabrala strana A . Zbog toga se identitet pošiljaoca uključuje u šifrovani deo poruke:

$$A \rightarrow B : c = \text{Enc}_{pk_B}(A \parallel K_{AB}), \text{Sig}_{sk_A}(c)$$

Nakon dešifrovanja, B vidi da je ključ povezan sa identitetom strane A . Time se sprečava jednostavna zamena potpisa nad istim šifratom. Ipak, protokol i dalje mora da obezbedi svežinu poruke, jer bi stara poruka mogla biti ponovo poslata i prihvaćena kao nova [31].

Glavno ograničenje prenosa ključa jeste to što ključ bira samo jedna strana. Strana B mora da veruje da je A generisala dovoljno slučajan i svež ključ. Osim toga, ovaj pristup nema svojstvo *forward secrecy*. Ako se kasnije kompromituje privatni ključ sk_B , napadač koji je ranije snimio poruke može dešifrovati stare sesijske ključeve i time ugroziti prethodnu komunikaciju [31].

Definicija 3.4.1. *Forward secrecy* je svojstvo protokola po kojem kompromitovanje dugoročnog privatnog ključa ne otkriva ranije uspostavljene sesijske ključeve [31].

Zbog ovih ograničenja uvode se protokoli za dogovor ključa. Kod njih obe strane doprinose nastanku sesijskog ključa. Najpoznatiji primer je Diffie–Hellman protokol.

3.4.2 Diffie–Hellman protokol

Diffie–Hellman protokol predstavlja osnovni primer dogovora ključa. Za razliku od prenosa ključa, nijedna strana ne šalje gotov sesijski ključ. Obe strane učestvuju u njegovom izračunavanju [31, 32].

Neka su javno poznati grupa G , generator g i red grupe q . Strana A bira slučajnu tajnu vrednost a , a strana B bira slučajnu tajnu vrednost b . Zatim razmenjuju javne efemerne vrednosti:

$$A \rightarrow B : g^a$$

$$B \rightarrow A : g^b$$

Nakon razmene, strane nezavisno računaju istu zajedničku tajnu:

$$A : K = (g^b)^a = g^{ab}$$

$$B : K = (g^a)^b = g^{ab}$$

Tajne vrednosti a i b se ne šalju kroz komunikacioni kanal. Iz zajedničke tajne se zatim izvodi simetrični ključ:

$$K_{AB} = H(g^{ab})$$

Bezbednost protokola zasniva se na težini Diffie–Hellman problema. Napadač koji prisluškuje komunikaciju vidi g^a i g^b , ali iz tih vrednosti ne bi trebalo da može efikasno da izračuna g^{ab} . Ova pretpostavka je povezana sa problemom diskretnog logaritma [31, 32].

Prednost ovog protokola je u tome što obe strane doprinose nastanku ključa. Sesijski ključ zavisi od obe efemerne tajne. Ako se vrednosti a i b obrišu nakon završetka sesije, protokol može da obezbedi svojstvo *forward secrecy* [31].

Međutim, osnovni Diffie–Hellman protokol ne obezbeđuje autentifikaciju. Strane mogu izračunati zajednički ključ, ali ne mogu biti sigurne sa kim su ga izračunale. Zbog toga je protokol ranjiv na napad čoveka u sredini, odnosno *man-in-the-middle* napad:

$$A \rightarrow E : g^a$$

$$E \rightarrow A : g^m$$

$$E \rightarrow B : g^n$$

$$B \rightarrow E : g^b$$

Strana A tada računa ključ sa napadačem E , a ne sa stranom B . Isto važi i za stranu B . Napadač može da dešifruje poruke, izmeni ih i ponovo šifruje pre

prosleđivanja. Zbog toga osnovni Diffie–Hellman rešava problem tajnog dogovora ključa, ali ne rešava problem autentifikacije učesnika [31, 32].

3.4.3 Potpisani Diffie–Hellman protokol

Nedostatak autentifikacije u osnovnom Diffie–Hellman protokolu može se ublažiti digitalnim potpisima. Svaka strana potpisuje svoju javnu efemernu vrednost [31]:

$$\begin{aligned} A \rightarrow B : g^a, \text{Sig}_{sk_A}(g^a) \\ B \rightarrow A : g^b, \text{Sig}_{sk_B}(g^b) \end{aligned}$$

Strana B proverava potpis strane A , a strana A proverava potpis strane B . Na taj način se sprečava jednostavna zamena Diffie–Hellman vrednosti, jer napadač ne može da pošalje svoju vrednost kao da dolazi od druge strane bez odgovarajućeg privatnog ključa.

Ipak, potpisivanje samo pojedinačne efemerne vrednosti nije dovoljno. Potpis treba da obuhvati širi kontekst sesije, najčešće obe Diffie–Hellman vrednosti i identitete učesnika. Time se sprečava da se poruke iz jedne sesije iskoriste u drugoj ili da se ključ poveže sa pogrešnim identitetom [31].

3.4.4 Station-to-Station protokol

Station-to-Station (STS) protokol predstavlja autentifikovanu verziju Diffie–Hellman protokola. Cilj protokola je da spreči napad čoveka u sredini. Osnovna ideja je da strane prvo razmene Diffie–Hellman efemerne vrednosti, a zatim da potpišu celu razmenu. Potpisi se štite ključem izvedenim iz zajedničke Diffie–Hellman tajne [31].

Protokol se može zapisati na sledeći način:

$$\begin{aligned} A \rightarrow B : et_A = g^a \\ B \rightarrow A : et_B = g^b, \{\text{Sig}_{sk_B}(et_B, et_A)\}_k \\ A \rightarrow B : \{\text{Sig}_{sk_A}(et_A, et_B)\}_k \end{aligned}$$

Ključ k izvodi se iz zajedničke Diffie–Hellman tajne:

$$k = H(g^{ab}).$$

Strana B računa $k = H(et_A^b)$, dok strana A računa $k = H(et_B^a)$. Pošto važi

$et_A^b = et_B^a = g^{ab}$, obe strane dobijaju isti ključ.

Potpisi obuhvataju obe efemerne vrednosti. Strana B potpisuje par (et_B, et_A) , dok strana A potpisuje par (et_A, et_B) . Redosled vrednosti razlikuje uloge inicijatora i odgovarača. Time se potpis vezuje za celu Diffie–Hellman razmenu, a ne samo za jednu poruku [31].

Zbog toga napadač ne može jednostavno da zameni efemernu vrednost i zadrži ispravan potpis. Svaka promena vrednosti et_A ili et_B menja podatke koji se potpisuju. Na taj način STS protokol obezbeđuje autentifikovani dogovor ključa i štiti osnovni Diffie–Hellman protokol od napada čoveka u sredini.

Postoji i varijanta STS protokola u kojoj se potpisi ne šifruju, već se nad njima računa MAC. Tada se iz zajedničke tajne izvode dva ključa:

$$k \parallel k' = H(g^{ab}).$$

Ključ k koristi se kao dogovoreni sesijski ključ, dok se k' koristi za autentifikaciju poruka tokom protokola. Ovakvo razdvajanje namena ključeva smanjuje rizik od pogrešne upotrebe istog ključa u različite svrhe [31].

3.4.5 Blake–Wilson–Menezes protokol

Blake–Wilson–Menezes protokol omogućava autentifikovan dogovor ključa bez dodatnih potpisa, MAC vrednosti i treće poruke. Ideja je da se efemerne Diffie–Hellman vrednosti kombinuju sa statičkim javnim ključevima učesnika [31].

Pretpostavimo da strane imaju statičke javne ključeve:

$$pk_A = g^a, \quad pk_B = g^b,$$

gde su a i b odgovarajući privatni ključevi. U svakoj sesiji strane biraju nove efemerne tajne. Strana A bira x , a strana B bira y . Zatim razmenjuju efemerne javne vrednosti:

$$\begin{aligned} A \rightarrow B : ek_A &= g^x \\ B \rightarrow A : ek_B &= g^y \end{aligned}$$

Tok poruka je isti kao kod osnovnog Diffie–Hellman protokola. Razlika je u izvođenju ključa. Ključ se ne izvodi samo iz efemernih vrednosti, već i iz statičkih

javnih ključeva [31]:

$$A : k = H(pk_B^x, ek_B^a)$$

$$B : k = H(ek_A^b, pk_A^y)$$

Obe strane dobijaju isti ključ, jer važi:

$$pk_B^x = (g^b)^x = g^{bx} = (g^x)^b = ek_A^b,$$

$$ek_B^a = (g^y)^a = g^{ay} = (g^a)^y = pk_A^y.$$

Autentifikacija se dobija preko statičkih ključeva. Napadač može poslati svoju efemernu vrednost, ali ne može izračunati isti ključ bez odgovarajućeg statičkog privatnog ključa. Zbog toga se izvedeni ključ vezuje za učesnike protokola.

Prednost ovog protokola je jednostavan tok poruka. Potrebne su samo dve poruke, kao kod osnovnog Diffie–Hellman protokola, ali bez dodatnih potpisa kao u STS protokolu. Ograničenje je u tome što izvođenje ključa mora biti pažljivo definisano. U funkciju za izvođenje ključa treba uključiti podatke koji vezuju ključ za učesnike i konkretnu sesiju, kako se ključ ne bi pogrešno povezao sa drugim identitetom [31].

3.4.6 MQV protokol

MQV protokol je autentifikovani protokol za dogovor ključa. Kao i Blake–Wilson–Menezes protokol, koristi statičke i efemerne ključeve. Razlika je u načinu na koji se ti ključevi kombinuju pri izvođenju konačnog sesijskog ključa [31].

Pretpostavimo da strane imaju statičke javne ključeve:

$$pk_A = g^a, \quad pk_B = g^b,$$

gde su a i b odgovarajući privatni ključevi. U svakoj sesiji strane biraju nove efemerne tajne. Strana A bira x , a strana B bira y . Zatim razmenjuju efemerne javne vrednosti:

$$A \rightarrow B : ek_A = g^x$$

$$B \rightarrow A : ek_B = g^y$$

Tok poruka je isti kao kod osnovnog Diffie–Hellman protokola. Razlika je u računanju ključa. MQV iz efernih javnih vrednosti izvodi pomoćne vrednosti koje se zatim kombinuju sa statičkim i efemernim privatnim ključevima [31].

Strana A iz vrednosti ek_A izvodi pomoćnu vrednost s_A , a iz vrednosti ek_B pomoćnu

vrednost t_A . Zatim računa:

$$\begin{aligned} h_A &= x + s_A a \pmod{q}, \\ K_A &= (ek_B \cdot pk_B^{t_A})^{h_A}. \end{aligned}$$

Strana B simetrično izvodi vrednosti s_B iz ek_B i t_B iz ek_A , pa računa:

$$\begin{aligned} h_B &= y + s_B b \pmod{q}, \\ K_B &= (ek_A \cdot pk_A^{t_B})^{h_B}. \end{aligned}$$

Pomoćne vrednosti se određuju tako da obe strane, iako koriste različite privatne podatke, dobijaju istu zajedničku vrednost. Iz te vrednosti se zatim izvodi sesijski ključ [31].

Sušтина MQV protokola je u povezivanju dve vrste ključeva. Efemerni ključevi obezbeđuju svežinu sesije, dok statički ključevi vezuju rezultat za identitete učesnika. Zato se dobija autentifikovan dogovor ključa bez dodatnih potpisa i bez treće poruke.

3.5 Simbolička analiza protokola

Protokoli za uspostavljanje ključa često imaju jednostavan zapis, ali njihova bezbednosna analiza može biti složena. Greška ne mora biti u kriptografskom algoritmu, već u redosledu poruka, značenju poruka ili pretpostavkama koje učesnici imaju. Zbog toga se za analizu protokola koriste formalne metode [31].

Definicija 3.5.1. Simbolička analiza protokola je način analize u kojem se protokol posmatra pomoću formalnih oznaka i logičkih pravila. Cilj je da se proverí šta učesnici mogu da zaključé nakon izvršavanja protokola [31].

U ovom pristupu napadač se obično posmatra kao veoma jak protivnik. On može da presretne poruke, zaustavi ih, promeni im redosled, promeni odredište ili pošalje sopstvene poruke. Zbog toga se često kaže da napadač kontroliše mrežu [31].

Najpoznatiji primer simboličke analize je BAN logika, koju su uveli Burrows, Abadi i Needham. BAN logika se ne bavi neposredno bitovima poruka, već verovanjima učesnika. Ona pokušava da formalno opiše šta strana A , strana B ili poverljiva treća strana S mogu da smatraju istinitim nakon prijema određene poruke [7].

U BAN logici koriste se sledeće osnovne oznake:

- $P \models X$ znači da strana P veruje da je iskaz X tačan,

- $P \triangleleft X$ znači da strana P vidi poruku X ,
- $P \mid \sim X$ znači da je strana P nekada rekla X ,
- $P \Rightarrow X$ znači da strana P ima nadležnost nad iskazom X ,
- $\#(X)$ znači da je poruka ili vrednost X sveža,
- $P \xleftrightarrow{K} Q$ znači da strane P i Q dele tajni ključ K [7].

Za protokole uspostavljanja ključa osnovni cilj je da obe strane veruju da dele isti tajni sesijski ključ:

$$\begin{aligned} A &\mid \equiv A \xleftrightarrow{K_{AB}} B, \\ B &\mid \equiv A \xleftrightarrow{K_{AB}} B. \end{aligned}$$

Jači cilj je potvrda ključa. Tada strana A ne samo da veruje da deli ključ sa B , već veruje i da B to zna. Isto važi i u suprotnom smeru [31].

BAN logika koristi pravila zaključivanja. Jedno od osnovnih pravila je pravilo značenja poruke. Ako A veruje da deli ključ K sa B , i ako vidi poruku šifrovanu tim ključem, tada A može da zaključi da je B nekada rekao sadržaj te poruke:

$$\frac{A \mid \equiv A \xleftrightarrow{K} B, \quad A \triangleleft \{X\}_K}{A \mid \equiv B \mid \sim X}$$

Drugo važno pravilo je pravilo provere svežine. Ako A veruje da je X sveže i ako veruje da je B nekada rekao X , tada A može da zaključi da B i dalje veruje u X :

$$\frac{A \mid \equiv \#(X), \quad A \mid \equiv B \mid \sim X}{A \mid \equiv B \mid \equiv X}$$

Treće važno pravilo je pravilo nadležnosti. Ako A veruje da je B nadležan za X , i ako A veruje da B veruje u X , tada A može prihvatiti X :

$$\frac{A \mid \equiv B \Rightarrow X, \quad A \mid \equiv B \mid \equiv X}{A \mid \equiv X}$$

3.6 Zaključak

Ovaj rad pokazuje da uspostavljanje ključa nije samo pitanje izbora jakog kriptografskog algoritma, već i pitanje poverenja, autentifikacije i pravilnog toka poruka. Iz analiziranih protokola može se zaključiti da tajnost ključa sama po sebi nije dovoljna

ukoliko učesnici ne mogu pouzdano da utvrde sa kim ga dele i da li je namenjen baš trenutnoj sesiji. Digitalni sertifikati i sertifikaciona tela imaju važnu ulogu u povezivanju javnih ključeva sa identitetima učesnika, dok autentifikovani protokoli za dogovor ključa umanjuju mogućnost napada, kao što je napad čoveka u sredini. Zbog toga je ova oblast posebno značajna za razumevanje bezbedne komunikacije u savremenim sistemima, kao što su internet servisi, elektronsko bankarstvo, VPN mreže i drugi sistemi u kojima je potrebno uspostaviti poverljiv komunikacioni kanal. Bezbednost se ne postiže samo primenom šifrovanja, već pažljivim projektovanjem protokola koji istovremeno obezbeđuje tajnost, autentifikaciju, svežinu i ispravno povezivanje ključa sa učesnicima komunikacije.

3.7 Pitanja

1. Zašto javni ključ mora biti povezan sa identitetom korisnika i koju ulogu u tome imaju digitalni sertifikati?
2. Koja je razlika između dugoročnog, odnosno statičkog, i sesijskog, odnosno efemernog ključa?
3. Koja je osnovna razlika između prenosa ključa i dogovora ključa?
4. Zašto osnovni Diffie–Hellman protokol nije dovoljan bez autentifikacije učesnika?
5. Šta znači svežina poruke ili ključa i zašto je važna zaštita od ponavljanja starih poruka?

Glava 4

Commitments

Šeme posvete (engl. *commitment schemes*) su kriptografska primitiva koja jednoj strani omogućava da se obaveže na izabranu vrednost i javno objavi njen kriptografski otisak, a da sama vrednost ostane skrivena i da je nakon toga više nije moguće promeniti. Ova jednostavna ideja pojavljuje se u iznenađujuće širokom spektru kriptografskih protokola - od klasičnih dokaznih sistema, kroz savremene blockchain platforme, pa sve do protokola nultog znanja koji su tema narednih poglavlja.

4.1 Motivacija i intuicija

Zamislamo takmičenje u kome više učesnika istovremeno treba da predaju svoje odgovore. Ako svaki učesnik vidi tuđe odgovore pre nego što preda sopstveni, može ih jednostavno prepisati ili prilagoditi. Da bismo to sprečili u fizičkom svetu, svako bi odgovor stavio u zapečaćenu kovertu i predao je organizatoru - koverta se otvaraju tek kada svi predaju svoje.

Zapečaćena koverta obavlja dve uloge istovremeno: sakriva sadržaj do otkrivanja, i sprečava promenu sadržaja nakon predaje. Upravo ove dve garancije - skrivanje i nepromenljivost - čine suštinu šeme posvete.

U digitalnom svetu ne postoji fizička koverta, ali šema posvete obezbeđuje isti efekat. Umesto koverta, šaljemo kriptografski otisak vrednosti. Iz otiska nije moguće rekonstruisati originalnu vrednost, ali kada se vrednost naknadno otkrije, svako može proveriti da odgovara ranijem otisku.

Ovaj obrazac formalizovao je Manuel Blum 1981. godine kroz problem bacanja novčića telefonom [6]: dve strane žele da se dogovore oko slučajnog ishoda bez međusobnog poverenja. Jedna strana se najpre obavezuje na svoju vrednost, a

tek potom otkriva. Rešenje je identično - šema posvete. Faze se nazivaju faza posvećivanja (engl. *commit phase*) i faza otkrivanja (engl. *reveal phase*), a obe garancije formalizujemo u narednoj sekciji.

4.2 Formalna definicija

Formalna definicija šeme posvete koju koristimo prati pristup iz [31].

Definicija 4.2.1 (Šema posvete). Šema posvete je par algoritama (Posveti , Otvori) nad prostorom poruka $\mathcal{M}$ i prostorom slučajnosti $\mathcal{R}$:

- $\text{Posveti}(m; r) \rightarrow c$: prima poruku $m \in \mathcal{M}$ i slučajnost $r \xleftarrow{\$} \mathcal{R}$ te vraća posvetu c (engl. *commitment*).
- $\text{Otvori}(c, m, r) \rightarrow \{0, 1\}$: proverava da li je c ispravna posveta na poruku m sa slučajnošću r .

Zahtevamo *ispravnost*: za sve $m \in \mathcal{M}$ i $r \in \mathcal{R}$ važi $\text{Otvori}(\text{Posveti}(m; r), m, r) = 1$.

Protokol se odvija u dve faze. U fazi *posvećivanja* pošiljalac bira $r \xleftarrow{\$} \mathcal{R}$, izračunava $c = \text{Posveti}(m; r)$ i šalje c primaocu, zadržavajući (m, r) za sebe. U fazi *otkrivanja* objavljuje par (m, r) , a primalac prihvata samo ako $\text{Otvori}(c, m, r) = 1$.

Vrednost r zove se *oslepljivač* (engl. *blinding factor*) - bez nje, onaj koji zna skup mogućih poruka može jednostavno izračunati posvetu za svaku od njih i porediti sa c , čime tajnost nestaje bez obzira na snagu heš funkcije.

Bezbednosna svojstva

Definicija 4.2.2 (Skrivanje i obavezivanje). Šema posvete zadovoljava:

- *Savršeno skrivanje* (engl. *perfectly hiding*): raspodele posveta identične su za sve poruke - tj. za sve $m_0, m_1 \in \mathcal{M}$ važi $\text{Posveti}(m_0; r) \stackrel{d}{=} \text{Posveti}(m_1; r)$ pri uniformnom r . *Računsko skrivanje* (engl. *computationally hiding*) je slabiji zahtev: raspodele nisu nužno identične, ali ih nijedan efikasan algoritam ne može razlikovati.¹

¹Precizno: za svakog napadača koji radi u polinomijalnom vremenu, njegova prednost u razlikovanju posveta je manja od $1/p(\lambda)$ za svaki polinom p , gde je λ bezbednosni parametar - što se smatra zanemarljivim.

- *Savršeno obavezivanje* (engl. *perfectly binding*): ne postoje $m \neq m'$ i r, r' takvi da $\text{Posveti}(m; r) = \text{Posveti}(m'; r')$ - svaka posveta ima jedinstveno otvaranje. *Računsko obavezivanje* (engl. *computationally binding*) zahteva samo da je pronalazak takvog para računski neizvodljiv.

Granica između dva svojstva

Propozicija 4.2.3. *Nijedna šema posvete ne može biti istovremeno savršeno sakrivajuća i savršeno obavezujuća.*

Dokaz. Pretpostavimo da je šema savršeno sakrivajuća i fiksiramo bilo koje $m_0 \neq m_1 \in \mathcal{M}$. Savršeno skrivanje znači da su raspodele $\text{Posveti}(m_0; r)$ i $\text{Posveti}(m_1; r)$ pri uniformnom r identične, pa imaju isti nosač. To znači da svaka vrednost c koja se može pojaviti kao posveta poruke m_0 mora se moći pojaviti i kao posveta poruke m_1 - u suprotnom bi sama vrednost c odala da poruka nije bila m_1 , što bi narušilo savršeno skrivanje. Dakle, za svako c iz nosača postoji r_0 takvo da $c = \text{Posveti}(m_0; r_0)$, i r_1 takvo da $c = \text{Posveti}(m_1; r_1)$. Ista posveta tako ima dva različita otvaranja na dve različite poruke, što direktno narušava savršeno obavezivanje. $\square$

Iz ove propozicije sledi da svaka šema mora da napravi kompromis. U narednoj sekciji videćemo dva tipa:

- Heš-zasnovana šema - računski sakrivajuća i računski obavezujuća jer heš funkcija daje pseudoslučajne raspodele koje ne garantuju savršena svojstva.
- Pedersenova šema je savršeno sakrivajuća i računski obavezujuća.

4.3 Konkretnе konstrukcije

4.3.1 Heš-zasnovana šema posvete

Najjednostavnija šema posvete zasniva se na kriptografskim heš funkcijama.

Definicija 4.3.1 (Heš-zasnovana šema posvete). Neka je $H: \{0, 1\}^* \rightarrow \{0, 1\}^\ell$ kriptografska heš funkcija otporna na nalaženje sudara (engl. *collision-resistant hash function*). Šema je definisana sa:

- **Postavi:** odabira se kriptografska heš funkcija H , prostor poruka je $\mathcal{M} = \{0, 1\}^*$, a prostor slučajnosti $\mathcal{R} = \{0, 1\}^\lambda$ gde je λ bezbednosni parametar.²
- $\text{Posveti}(H, m; r) = H(m \parallel r)$, gde $\parallel$ označava konkatenciju.
- $\text{Otvori}(H, c, m, r) = 1$ ako i samo ako $c = H(m \parallel r)$.

Obavezujuće svojstvo neposredno sledi iz otpornosti heš funkcije na nalaženje sudara: pronalaženje $m \neq m'$ i r, r' takvih da $H(m \parallel r) = H(m' \parallel r')$ direktno je sudar funkcije H . Formalno, ovo je računsko obavezivanje - sudari između različitih otvaranja postoje u teoriji, jer heš funkcija slika beskonačan skup ulaza u konačan skup izlaza. Međutim, u praksi, nijedan poznati algoritam ne može da ih pronađe za kriptografski jake heš funkcije.

Svojstvo skrivanja je samo računsko. Raspodela posvete $c = H(m \parallel r)$ pri uniformnom r nije savršeno identična za sve poruke, jer heš funkcija daje pseudoslučajne a ne uniformne izlaze. Nijedan efikasan algoritam ne može razlikovati raspodele, ali teorijski neograničen protivnik to može, pa savršeno skrivanje nije postignuto.

Napomena 4.3.2. Uključivanje slučajnog oslepljivača r je neophodno. Bez njega, napadač koji zna da je poruka jedna od konačno mnogo mogućih vrednosti može jednostavno da izračuna heš svake od njih i poredi sa c . Ovo je naročito važno u kontekstu glasanja, gde su ishodi $\{0, 1\}$, a bez oslepljivača bi svaka posveta odmah otkrivala glas.

4.3.2 Pedersenova šema posvete

Heš-zasnovana šema je računski obavezujuća i računski sakrivajuća. Pedersenova šema [26] postiže kompromis: savršeno skrivanje uz računsko obavezivanje, oslanjajući se na algebarsku strukturu grupe umesto na heš funkciju.

Definicija 4.3.3 (Pedersenova šema posvete). Neka je $\mathbb{G}$ ciklična grupa prostog reda q , i neka su $g, h \in \mathbb{G}$ generatori takvi da $\log_g(h)$ nije poznat nijednoj strani. Šema je definisana sa:

- **Postavi:** javni parametri su $(\mathbb{G}, q, g, h)$, prostor poruka $\mathcal{M} = \mathbb{Z}_q$, prostor slučajnosti $\mathcal{R} = \mathbb{Z}_q$.

²Bezbednosni parametar λ određuje koliko operacija napadač mora da izvrši da bi razbio šemu - potrebno mu je 2^λ operacija. Za $\lambda = 128$ to iznosi $2^{128} \approx 3,4 \cdot 10^{38}$ operacija, što prevazilazi računске mogućnosti svih kompjutera na svetu zajedno. Svaki dodatni bit udvostručuje potreban posao.

- $\text{Posveti}(m; r) = g^m h^r$, gde se $r \xleftarrow{\$} \mathbb{Z}_q$ bira uniformno.
- $\text{Otvori}(c, m, r) = 1$ ako i samo ako $c = g^m h^r$.

Savršeno skrivanje. Pošto se r bira uniformno iz $\mathbb{Z}_q$, vrednost h^r ravnomerno pokriva celu grupu $\mathbb{G}$. Za svaku moguću fiksiranu vrednost m_0 , posveta $g^{m_0} h^r$ prolazi kroz sve elemente grupe kako r varira - dakle raspodela posvete ne zavisi od m . Ni teorijski neograničen protivnik ne može iz c naučiti ništa o m .

Računsko obavezivanje. Pretpostavimo da protivnik može da nađe $m \neq m'$ i r, r' takve da $g^m h^r = g^{m'} h^{r'}$. Iz toga sledi $g^{m-m'} = h^{r'-r}$, odakle se može izračunati

$$\log_g(h) = \frac{m - m'}{r' - r} \pmod{q}.$$

Dakle, pronalaženje dva različita otvaranja iste posvete ekvivalentno je rešavanju problema diskretnog logaritma u $\mathbb{G}$, koji se smatra računski neizvodljivim za pravilno izabrane grupe.

Ograničenje prostora poruka. Za razliku od heš-zasnovane šeme koja prima poruke proizvoljnog oblika, Pedersenova šema zahteva da poruka m bude element $\mathbb{Z}_q$ - dakle broj, zbog toga Pedersen nije zamena za heš šemu u opštem slučaju.

Aditivno svojstvo. Pedersenove posvete imaju važnu algebarsku osobinu: proizvod dve posvete je posveta na zbir posvedočenih vrednosti,

$$g^{m_1} h^{r_1} \cdot g^{m_2} h^{r_2} = g^{m_1+m_2} h^{r_1+r_2}.$$

Ovo homomorfno svojstvo čini Pedersenove posvete posebno korisnim u protokolima nultog znanja, gde je potrebno dokazivati relacije između skrivenih vrednosti bez njihovog otkrivanja.

4.3.3 KZG polinomijalne posvete

Heš-zasnovana i Pedersenova šema obavezuju se na *skalarne* vrednosti. U savremenim ZK sistemima često je potrebno obavezati se na čitav *polinom* i naknadno dokazivati njegove vrednosti u izabranim tačkama, bez otkrivanja samog polinoma. Upravo to je zadatak polinomijalnih šema posvete, od kojih je najpoznatija KZG šema (Kate, Zaverucha i Goldberg) [18].

Osnovna ideja. Ideja KZG je jednostavna: obavežemo se na evaluaciju polinoma u jednoj *tajnoj* tački τ , i tu evaluaciju koristimo kao posvetu na čitav polinom. Tako dobijamo posvetu koja je jedan element grupe eliptičke krive, a koja omogućava efikasno dokazivanje vrednosti polinoma u proizvoljnim tačkama. Ovde g označava generator te grupe - fiksirani javni element krive koji svi učesnici protokola dele, analogan generatoru g kod Pedersena.

Poverljivo podešavanje. KZG zahteva jednokratnu fazu poverljivog podešavanja (engl. *trusted setup*). Tokom te faze bira se tajna vrednost $\tau \xleftarrow{\$} \mathbb{Z}_q$, a zatim se objavljuje strukturisana referentna niska (engl. *structured reference string*, SRS):

$$\text{SRS} = (g, \tau \cdot g, \tau^2 \cdot g, \dots, \tau^d \cdot g).$$

Ovi javni parametri su enkodovani stepeni od τ , ali ne i τ sama. Tajna τ mora biti bezbedno uništena nakon generisanja SRS-a - ko god je zna može da falsifikuje dokaze za proizvoljne vrednosti polinoma.

Posveta na polinom. Neka je polinom $p(X) = \sum_{i=0}^d p_i X^i$. Dokazivač izračunava posvetu kao:

$$C = \sum_{i=0}^d p_i \cdot (\tau^i \cdot g) = p(\tau) \cdot g.$$

Iako τ nije poznata, SRS omogućava izračunavanje $p(\tau) \cdot g$ bez njenog eksplicitnog poznavanja. Posveta je *jedan element grupe*, bez obzira na stepen polinoma.

Dokaz otvaranja. Da bi dokazao da $p(x_0) = y_0$ za neku konkretnu tačku x_0 , dokazivač koristi činjenicu da polinom $p(X) - y_0$ mora biti deljiv sa $(X - x_0)$. Količnik

$$q(X) = \frac{p(X) - y_0}{X - x_0}$$

je polinom stepena $d - 1$. Dokazivač izračunava svedoka provlačenjem q kroz isti proces kao pri posveti - koristeći SRS, bez poznavanja τ :

$$\pi = q(\tau) \cdot g = \sum_{i=0}^{d-1} q_i \cdot (\tau^i \cdot g).$$

Pošto q ima manji stepen od p , potrebni su samo prvih d elemenata SRS-a, koji su dostupni. Svedok π je *jedan element grupe* - konstantne veličine bez obzira na stepen polinoma.

Verifikacija. Bilinearno preslikavanje e je funkcija koja prima dva elementa grupe i vraća broj, uz svojstvo $e(a \cdot g, b \cdot g) = e(g, g)^{ab}$ - što omogućava proveru jednakosti između eksponenata bez njihovog otkrivanja. Verifikator proverava:

$$e(C - y_0 \cdot g, g) \stackrel{?}{=} e(\pi, \tau \cdot g - x_0 \cdot g).$$

Raspisivanjem obe strane postaje jasno šta se proverava:

- Leva strana: $C - y_0 \cdot g = p(\tau) \cdot g - y_0 \cdot g = (p(\tau) - y_0) \cdot g$, pa je $e(C - y_0 \cdot g, g) = e(g, g)^{p(\tau) - y_0}$.
- Desna strana: $\pi = q(\tau) \cdot g$ i $\tau \cdot g - x_0 \cdot g = (\tau - x_0) \cdot g$, pa je $e(\pi, \tau \cdot g - x_0 \cdot g) = e(g, g)^{q(\tau) \cdot (\tau - x_0)}$.

Jednakost važi kada su eksponenti jednaki, tj. kada $p(\tau) - y_0 = q(\tau) \cdot (\tau - x_0)$. Po definiciji količnika $q(X) = \frac{p(X) - y_0}{X - x_0}$, ova jednakost važi za svako X samo ako $(X - x_0)$ zaista deli $p(X) - y_0$ bez ostatka - što je tačno jedino kada $p(x_0) = y_0$. Verifikator je tako potvrdio traženu vrednost polinoma bez poznavanja ostatka polinoma.

Bezbednost i primene. Obavezivanje počiva na d -jakoj Diffie–Hellmanovoj pretpostavci. Glavna prednost KZG šeme nije hiding svojstvo, već *konstantna veličina dokaza* - i posveta i svedok su uvek jedan element grupe, bez obzira na stepen polinoma. KZG posvete koriste se u Ethereum protokolu od 2024. godine kao deo standarda EIP-4844, koji uvodi blob transakcije za efikasno skladištenje podataka slojeva za skaliranje [8]. Ista šema stoji u osnovi modernih ZK-SNARK sistema koji su tema narednih poglavlja.

4.4 Primene u blockchain ekosistemu

Šeme posvete nisu samo teorijska apstrakcija - u ovom poglavlju razmatramo dva konkretna oblika njihove primene u blockchain infrastrukturi. U prvom, posveta se koristi da bi se jedna vrednost sakrila dok se ne ispune određeni uslovi. U drugom, koristi se da bi se efikasno i proverljivo predstavio čitav skup podataka.

4.4.1 Šablon posveti-pa-otkrij

Šablon posveti-pa-otkrij (engl. *commit-reveal*) direktna je primena heš-zasnovane šeme posvete na javnim mrežama gde sve poruke vide svi učesnici čim budu poslate.

Konkretna primena je registracija decentralizovanih domenskih imena u sistemu ENS (engl. *Ethereum Name Service*). Kada korisnik želi da registruje `.eth` domen, zahtev kratko vreme provodi u javnom redu čekanja transakcija (engl. *mempool*) vidljivom svim učesnicima mreže. Napadač koji uoči zahtev može odmah da registruje isti domen pre korisnika - ovaj napad poznat je kao napad pretrcavanjem (engl. *front-running*). ENS rešava ovo uvođenjem dve odvojene faze:

1. **Faza posvećivanja:** korisnik bira slučajnost r , izračunava $c = H(\text{ime} \parallel r)$ i objavljuje samo c na lancu. Iz heša nije moguće saznati koji domen korisnik registruje.
2. **Faza otkrivanja:** nakon obaveznog čekanja od najmanje 60 sekundi, korisnik objavljuje pravo ime i vrednost r . Pametni ugovor (engl. *smart contract*) proverava podudarnost sa ranije poslatim hešom i završava registraciju.

Bezbednost protokola počiva direktno na dva svojstva šeme posvete: *skrivanje* onemogućava napadaču da sazna ime iz heša, a *obavezivanje* sprečava korisnika da naknadno promeni zahtev na popularniji domen koji je u međuvremenu primetio.

Isti obrazac primenjuje se i izvan registracije domena. Sistemi za glasanje na blockchainu koriste commit-reveal da bi sprečili da glasači menjaju odluku na osnovu vidljivog rasporeda glasova.

4.4.2 Merkle stabla kao vektorske posvete

Heš-zasnovana šema iz sekcije 4.3.1 obavezuje na *jednu* poruku. U praksi često treba da se obavežemo na *skup* vrednosti - na primer, na sve transakcije u bloku - i da naknadno efikasno dokazujemo da određena vrednost pripada tom skupu. Merkle stablo (engl. *Merkle tree*) je struktura koja to omogućava.

Definicija 4.4.1 (Merkle stablo). Neka je $\{d_1, d_2, \dots, d_n\}$ skup podataka i H kriptografska heš funkcija. Merkle stablo nad ovim skupom je binarno stablo u kome:

- svaki list sadrži heš jednog podatka: $\ell_i = H(d_i)$,
- svaki unutrašnji čvor sadrži heš svojih potomaka: $v = H(\text{levo} \parallel \text{desno})$,
- koren stabla (engl. *Merkle root*) R je jednoznačno određen celim skupom.

Koren R je posveta na čitav skup - promena bilo kojeg podatka d_i propagira se kroz stablo i menja R , što je direktna posledica otpornosti heš funkcije na nalaženje sudara.

Ključna prednost Merkle stabla je efikasan dokaz uključenosti (engl. *Merkle proof*): da bi dokazao da d_i pripada skupu posvedočenom korenom R , dovoljno je priložiti *put* od lista ℓ_i do korena - niz od $\lceil \log_2 n \rceil$ heš vrednosti. Primalac može da proveri dokaz reizračunavanjem puta i poređenjem sa R , bez poznavanja ijednog drugog podatka iz skupa. U praksi se koriste i varijante veće arnosti - Ethereum koristi strukturu sa arnosti 16 [10] - ali princip posvete i dokaza uključenosti ostaje isti.

U Ethereum protokolu svako zaglavlje bloka sadrži tri Merkle korena: koren stabla transakcija, koren stabla stanja (engl. *state root*) i koren stabla priznanica (engl. *receipts root*) [35]. Na taj način, stanje celog protokola - svi nalozi, stanja ugovora, salda - kompaktno je posvedočeno jednim hešem od 32 bajta, što je direktna primena Merkle korena kao posvete na skup.

4.5 Šeme posvete kao osnova sistema za dokazivanje

Šeme posvete nisu samo samostalni kriptografski alati - one su gradivni elementi savremenih sistema za dokazivanje nultog znanja koji su tema narednih poglavlja.

U zk-STARK sistemima dokazivač se tokom protokola mora obavezati na veliki skup vrednosti, a verifikator nasumično bira pozicije i proverava ih. Za taj korak STARK koristi upravo Merkle stablo: listovi sadrže heševe evaluacija polinoma, a koren se objavljuje kao posveta. Svaki zahtev verifikatora dobija Merkle dokaz - niz heš vrednosti duž putanje do korena, čija veličina raste logaritamski sa brojem posvedočenih vrednosti.

U zk-SNARK sistemima zasnovanim na bilinearnim preslikavanjima koriste se KZG polinomijalne posvete. Posveta na polinom je jedan element grupe eliptičke krive, a dokaz da taj polinom u nekoj konkretnoj tački uzima određenu vrednost je ponovo jedan element grupe, iste veličine, bez obzira na stepen polinoma. Upravo ta konstantna veličina dokaza otvaranja presudno utiče na kompaktnost celog SNARK dokaza.

Ovaj kontrast leži u osnovi ključne razlike između dve porodice ZK sistema: zk-SNARK dokazi su obično svega nekoliko stotina bajtova, dok su zk-STARK dokazi znatno veći, jer Merkle putanje rastu logaritamski.

Veličina dokaza nije samo teorijsko pitanje. Ethereum planira prelaz na besstanu validaciju (engl. *stateless validation*), u kojoj verifikator proverava ispravnost bloka bez čuvanja celokupnog stanja lokalno. Uz svaki blok mora se preneti svedok (engl.

witness) - skup vrednosti i kriptografskih dokaza koji pokrivaju sve pristupe stanju u tom bloku. Trenutno Ethereum čuva stanje u stablu sa arnosti 16 [10]: da bi se dokazao jedan pristup stanju, na svakom nivou stabla potrebno je priložiti svih 15 pratećih čvorova. Za stotine pristupa stanju po bloku, svedok naraste na nekoliko megabajta i ne može se pouzdano preneti unutar 12-sekundnog slot-a.

Rešenje koje Ethereum razrađuje jeste zamena heš-zasnovane posvete unutar čvorova stabla *vektorskom posvetom sa dokazima konstantne veličine*. Pošto dokaz otvaranja ne zavisi od broja elemenata u čvoru, stablo može imati znatno veću arnost bez kazne na veličinu svedoka. Prema merenjima na istorijskim blokovima, svedok za tipičan blok sa Verkle stablom iznosi oko 380 KB u medijani [34], što je značajno manje od nekoliko megabajta kod trenutnog rešenja. Ova evolucija ilustruje kako teorijska svojstva šema posvete - a posebno veličina dokaza otvaranja - imaju direktne praktične posledice za skalabilnost i decentralizaciju blockchain sistema.

Glava 5

Uvod u dokaze nultog znanja

Dokazi nultog znanja (engl. *zero-knowledge proofs*, ZKP) predstavljaju jednu od ključnih kriptografskih primitiva, koje omogućavaju jednoj strani da dokaže istinitost neke tvrdnje drugoj strani, a da pri tome ne otkrije ništa osim same istinitosti. Ovaj pojam uveli su Šefi Goldvoser (engl. *Shafi Goldwasser*) i Silvio Mikali (engl. *Silvio Micali*) sa Tehnološkog instituta Masačusets (engl. *Massachusetts Institute of Technology*) zajedno sa Čarlsom Rekoфом (engl. *Charles Rackoff*) sa Univerziteta u Torontu (engl. *University of Toronto*) u radu *The Knowledge Complexity of Interactive Proof-Systems* [14], predstavljenom na konferenciji STOC¹ 1985. godine. U tom radu autori su pokazali da se istinitost tvrdnje može dokazati tako da dokazivač (engl. *prover*), strana koja dokazuje tvrdnju, uveri verifikatora (engl. *verifier*), stranu koja proverava dokaz, u njenu istinitost, a da pri tome ne otkrije ništa osim same činjenice da je tvrdnja tačna.

Značaj dokaza nultog znanja proizilazi upravo iz ove prividno paradoksalne mogućnosti: jedna strana može dokazati da poseduje određeno znanje, a da pri tome drugoj strani ne otkrije njegov sadržaj. Zbog toga se dokazi nultog znanja koriste kao teorijska osnova i praktičan alat u autentifikaciji, zaštiti privatnosti, digitalnim potpisima, neinteraktivnim dokazima i verifikabilnim sistemima elektronskog glasanja [20]. U ovom poglavlju predstavljamo osnovne ideje i formalne koncepte dokaza nultog znanja, od klasičnih primera do veze sa teorijom složenosti.

¹STOC (engl. *Symposium on Theory of Computing*) jedna je od najuglednijih godišnjih konferencija iz oblasti teorije računarstva.

5.1 Motivacija i osnovna terminologija

Pretpostavimo da Alisa poseduje lozinku i želi da se autentifikuje na veb-sajtu koji administrira Bob, ali ne želi da Bobu, odnosno serveru, otkrije samu lozinku. Direktno slanje lozinke omogućava proveru identiteta, ali istovremeno izlaže lozinku riziku kompromitacije. Jedan od praktičnih pristupa ovom problemu jeste metod delimičnog otkrivanja lozinke, gde sistem u različitim sesijama traži samo pojedine karaktere, na primer prvi, četvrti i deseti karakter. Takav pristup smanjuje količinu informacije koja curi u jednoj sesiji, ali ne rešava osnovni problem: verifikator i dalje uči delove tajne.

Idealni cilj je da verifikator bude uveren da dokazivač zaista poznaje tajnu, ali da iz samog protokola ne sazna ništa o njoj. Dokazi nultog znanja formalizuju upravo tu mogućnost. U opštem kontekstu kriptografskih protokola učesnici se tradicionalno označavaju imenima Alisa i Bob. Međutim, u literaturi o dokazima znanja i dokazima nultog znanja često se koriste imena Pegi (engl. *Peggy*) i Viktor (engl. *Victor*), izvedena iz naziva njihovih uloga dokazivača i verifikatora. Ova konvencija biće korišćena i u nastavku rada.

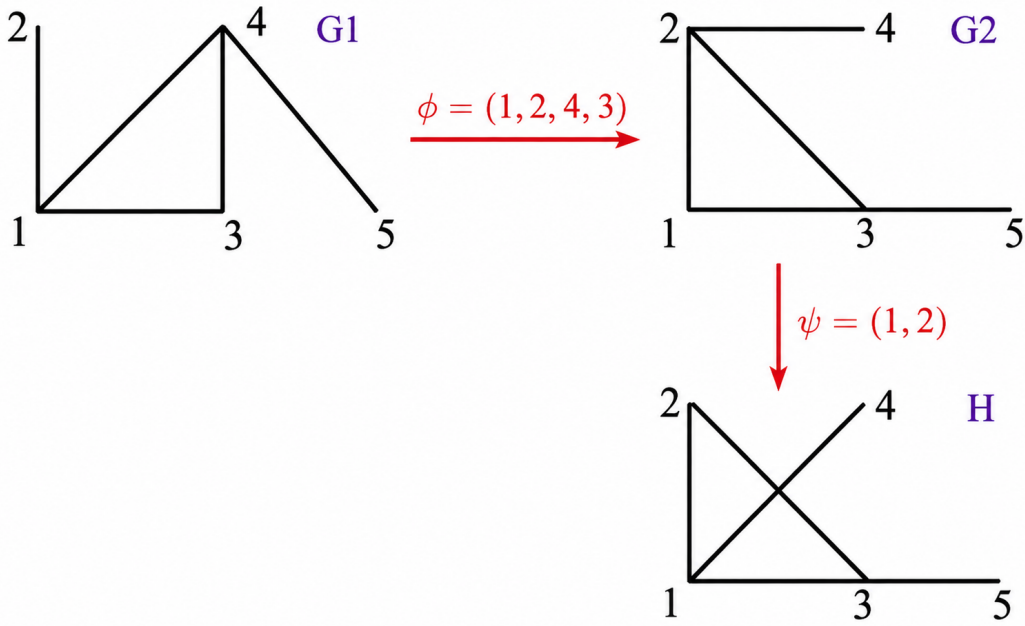
5.2 Protokol za izomorfizam grafova

Kao klasičan primer dokaza nultog znanja razmatra se protokol zasnovan na problemu izomorfizma grafova.

Definicija 5.2.1. Za dva grafa kažemo da su izomorfni ukoliko postoji bijektivno preslikavanje između skupova njihovih čvorova pri kojem se čuva incidentnost sa granama. Takvo preslikavanje naziva se izomorfizam grafova. Ako je φ izomorfizam između grafova G_1 i G_2 , tada pišemo

$$G_2 = \varphi(G_1)$$

Primer takvog preslikavanja prikazan je na slici 5.1. Grafovi G_1 i G_2 imaju identičnu strukturu povezanosti, ali se razlikuju u označavanju čvorova. Izomorfizam između njih dat je permutacijom $\varphi = (1, 2, 4, 3)$, čijom se primenom na graf G_1 dobija graf G_2 .



Slika 5.1: Primer izomorfizma grafova i konstrukcije posvećenog grafa

Pretpostavimo sada da Pegi poznaje izomorfizam φ između javnih grafova G_1 i G_2 , dok Viktor zna samo grafove. Cilj protokola jeste da Pegi ubedi Viktora u to da ima tu informaciju, a da pri tome ne otkrije samu permutaciju. Protokol se odvija u tri koraka. Posveta (engl. *commitment*) je vrednost koju dokazivač šalje verifikatoru kako bi se vezao za određeni izbor pre nego što sazna izazov, čime se onemogućava naknadno menjanje tog izbora. Zatim verifikator šalje izazov (engl. *challenge*), nasumično izabranu vrednost kojom ispituje dokazivačevo znanje. Na osnovu izazova dokazivač formira odgovor (engl. *response*), vrednost koju verifikator proverava kako bi se uverio u istinitost tvrdnje. U tu svrhu koristi se dodatni graf H , prikazan na slici 5.1, čija će uloga biti objašnjena kroz korake protokola.

1. **Posveta** Pegi bira slučajnu permutaciju ψ i konstruiše graf

$$H \leftarrow \psi(G_2).$$

Graf H šalje Viktoru kao posvetu. Pri tome važi

$$H = \psi(G_2) = \psi(\varphi(G_1)),$$

2. **Izazov** Viktor bira slučajnu vrednost $b \in \{1, 2\}$ i zahteva od Pegi izomorfizam između grafova G_b i H .

3. **Odgovor** Ukoliko je $b = 2$, Pegi šalje permutaciju $\chi = \psi$. U suprotnom, za $b = 1$, šalje permutaciju

$$\chi = \psi \circ \varphi.$$

Viktor prihvata dokaz ukoliko proverom utvrdi da važi

$$\chi(G_b) = H.$$

Primetimo da je graf H konstruisan tako da bude izomorfan i grafu G_2 i grafu G_1 . Zaista, po konstrukciji važi

$$H = \psi(G_2),$$

pa je permutacija ψ izomorfizam između grafova G_2 i H . Takođe,

$$H = \psi(\varphi(G_1)) = (\psi \circ \varphi)(G_1),$$

odakle sledi da je kompozicija $\psi \circ \varphi$ izomorfizam između grafova G_1 i H . Zbog toga Pegi može da odgovori na oba moguća Viktorova izazova: ukoliko se zahteva izomorfizam između G_2 i H , dovoljno je da otkrije permutaciju ψ , dok u slučaju da se zahteva izomorfizam između G_1 i H , otkriva kompoziciju $\psi \circ \varphi$.

Pod transkriptom protokola podrazumevamo niz svih poruka razmenjenih između dokazivača i verifikatora tokom izvršavanja protokola, pa jedna runda ima oblik

$$P \rightarrow V : H, \quad V \rightarrow P : b, \quad P \rightarrow V : \chi.$$

Na primer, ako su $\varphi = (1, 2, 4, 3)$ i $\psi = (1, 2)$, tada za izazov $b = 1$ Pegi šalje

$$\chi = \psi \circ \varphi = (1, 2) \circ (1, 2, 4, 3) = (2, 4, 3).$$

U slučaju da Pegi ne zna izomorfizam φ , mora unapred da pogodi koji će izazov Viktor postaviti. Verovatnoća ispravnog pogađanja u jednoj rundi iznosi $1/2$, a ponavljanjem protokola n puta verovatnoća da nepošteni dokazivač prođe sve runde opada na $(1/2)^n$ — vrednost koja eksponencijalno brzo opada i postaje zanemarljiva.

Važan uslov bezbednosti jeste da Pegi u svakoj rundi koristi novu slučajnu permutaciju i novi graf H . Ako bi isti graf H bio upotrebljen više puta, Viktor bi u jednoj rundi mogao zatražiti odgovor za $b = 2$ i dobiti ψ , a u drugoj odgovor za $b = 1$ i dobiti $\psi \circ \varphi$. Tada bi mogao izračunati

$$\varphi = (\psi \circ \varphi) \circ \psi^{-1},$$

čime bi tajni izomorfizam bio otkriven.

Postavlja se pitanje da li Viktor uopšte uči nešto iz izvođenja protokola. Odgovor je negativan, što se vidi iz sledećeg argumenta. Viktor može sam, bez ikakve interakcije sa Pegi, da konstruiše transkript (H, b, χ) koji prolazi svaku proveru. Dovoljno je da nasumično izabere b i χ , a zatim izračuna $H = \chi(G_b)$. Pošto takav transkript ne može da se razlikuje od transkripta nastalog u stvarnom razgovoru, Viktor njime ne može ubediti treću stranu niti da on poznaje φ , niti da je Pegi uopšte učestvovala u protokolu.

Ovo svojstvo predstavlja formalizaciju zahteva nultog znanja. Sve što verifikator vidi tokom izvršavanja protokola može se simulirati bez poznavanja tajne, pa transkript ne sadrži dodatnu informaciju o samoj tajni. Drugim rečima, protokol ne otkriva ništa što verifikator ne bi mogao da proizvede samostalno.

Protokol koji smo opisali primer je interaktivnog dokaznog sistema, konstrukcije u kojoj dokazivač i verifikator razmenjuju poruke kako bi verifikator stekao uverenje u istinitost neke tvrdnje. Od takvog sistema očekujemo da zadovolji tri osnovna svojstva.

- **Potpunost** (engl. *completeness*): ako Pegi zaista poznaje tajnu i pošteno prati protokol, Viktor će prihvatiti dokaz sa verovatnoćom jedan.
- **Tačnost** (engl. *soundness*): ako Pegi ne poznaje tajnu, verovatnoća da Viktor prihvati lažni dokaz je zanemarljivo mala, bez obzira na strategiju kojom se Pegi služi.
- **Nulto znanje** (engl. *zero-knowledge*): transkript protokola ne otkriva verifikatoru nikakvu informaciju osim same istinitosti tvrdnje, što se formalizuje zahtevom da se transkript može simulirati bez poznavanja tajne.

U protokolu za izomorfizam grafova sva tri svojstva su zadovoljena. Potpunost je očigledna. Poštena Pegi uvek može da odgovori na oba moguća izazova. Tačnost sledi iz ranije analize. Verovatnoća da nepošteni dokazivač prođe n rundi iznosi $(1/2)^n$ i eksponencijalno opada sa brojem rundi. Već za $n = 20$, ona je manja od 0.000001. Svojstvo nultog znanja sledi iz simulacionog argumenta, budući da Viktor može sam da konstruiše transkript koji je nerazlučiv od stvarnog, pa protokol ne otkriva nikakvu informaciju o tajnom izomorfizmu.

5.3 Nulto znanje i klase složenosti

Pre nego što razmotrimo koje se tvrdnje mogu dokazati uz nulto znanje, potrebno je objasniti kako se formalizuje zahtev da transkript protokola ne otkriva nikakvu dodatnu informaciju. Osnovna ideja zasniva se na poređenju stvarnih transkripata protokola sa transkriptima koje je moguće generisati simulacijom bez poznavanja tajne. U analizi se uobičajeno pretpostavlja da je Viktor računski ograničen, dok se Pegi tretira kao računski neograničena strana, ako protokol ostaje bezbedan čak i protiv neograničeno moćnog dokazivača, utoliko pre je bezbedan u praksi. U zavisnosti od toga koliko su raspodele stvarnih i simuliranih transkripata bliske, razlikuju se tri nivoa nultog znanja.

Neka je V skup stvarnih transkripata, a S skup simuliranih.

- **Savršeno nulto znanje** ($V = S$): raspodele su identične, pa ni računski neograničen protivnik ne može razlikovati simulirani transkript od stvarnog. Protokol za izomorfizam grafova iz prethodne sekcije primer je ovog nivoa.
- **Statističko nulto znanje** ($V \approx_s S$): raspodele se razlikuju za statistički zanemarljivu vrednost, što ih čini nerazlučivim čak i za računski neograničenog protivnika.
- **Računsko nulto znanje** ($V \approx_c S$): raspodele su nerazlučive za računski ograničenog protivnika. Većina praktičnih protokola spada u ovu kategoriju, jer je takav nivo bezbednosti dovoljan u odnosu na realne pretnje.

Nakon što smo definisali svojstvo nultog znanja, prirodno se nameće pitanje koje se tvrdnje uopšte mogu dokazivati na ovaj način. Već smo pokazali da problem izomorfizma grafova leži u ovoj klasi, ali taj problem zauzima posebno mesto u teoriji složenosti: veruje se da leži između klase **P** i **NP**-kompletnih problema, dakle nije rešiv u polinomskom vremenu, ali verovatno nije ni **NP**-kompletna. Zbog toga on sam po sebi ne govori mnogo o tome koliko su dokazi nultog znanja moćni, moglo bi se pomisliti da su ograničeni upravo na ovakve probleme koji ne spadaju ni u jednu dobro poznatu klasu.

Ispostavlja se, međutim, da je doseg dokaza nultog znanja daleko širi. Kada govorimo o dokazivanju tvrdnji, pod svedokom (engl. *witness*) podrazumevamo dodatnu informaciju uz pomoć koje verifikator može efikasno proveriti ispravnost nekog rešenja. Ključna klasa u ovom kontekstu jeste **NP**, koja obuhvata probleme za koje postoji kratak svedok čiju ispravnost računski ograničen verifikator može

proveriti u polinomskom vremenu. U klasičnom, neinteraktivnom modelu dokazivač jednostavno dostavlja svedoka, a verifikator proverava njegovu ispravnost bez ikakve razmene poruka.

Ako se dozvoli interakcija između računski neograničenog dokazivača i računski ograničenog verifikatora, dobija se klasa interaktivnih dokaza **IP**. Za razliku od klase **NP**, gde dokazivač jednostavno dostavlja svedoka bez ikakve razmene poruka, u okviru ovog modela, dokazivač i verifikator aktivno razmenjuju poruke kroz veći broj rundi

$$\mathbf{IP} = \mathbf{PSPACE},$$

gde je **PSPACE** klasa problema rešivih uz polinomsku količinu memorije. Pošto se široko veruje da je $\mathbf{NP} \subsetneq \mathbf{PSPACE}$, ovaj rezultat pokazuje da interakcija značajno proširuje klasu problema koja se mogu dokazivati.

Šta se dešava kada interaktivnim dokazima dodamo i zahtev nultog znanja? Definišemo klasu **CZK** (engl. *Computational Zero-Knowledge*) kao skup svih problema čija se rešenja mogu verifikovati putem računskog dokaza nultog znanja. Već smo pokazali da problem izomorfizma grafova leži u **CZK**, ali postavlja se pitanje da li tu leže i svi **NP** problemi. Odgovor daje sledeća teorema.

Teorema 5.3.1. *Problem 3-oboјivosti grafa leži u CZK, pod pretpostavkom da postoji računski sakrivajuća šema posvete.*

Problem 3-oboјivosti postavlja pitanje, može li se svaki čvor grafa oboјiti jednom od tri boje tako da nijedna grana ne spaja dva čvora iste boje? U protokolu koji dokazuje ovu teoremu, Pegi zna ispravno bojenje i želi da to dokaže bez otkrivanja samih boja. U fazi posvete, Pegi bira slučajnu permutaciju boja i za svaki čvor objavljuje posvetu na (permutovanu) boju tog čvora. Viktor zatim nasumično bira jednu granu i traži da se otkriju boje njenih krajeva. Pegi otvara odgovarajuće posvete, a Viktor proverava da su boje različite. Permutacija boja menja se u svakoj rundi, pa Viktor ni nakon mnogo rundi ne dobija informaciju o kompletnom bojenju, saznaje samo da su boje krajeva izabrane grane različite.

Pošto je 3-oboјivost **NP**-kompletna, iz teoreme neposredno sledi $\mathbf{NP} \subseteq \mathbf{CZK}$. Svaka tvrdnja koja ima kratak dokaz može se dokazati i uz nulto znanje. Ovaj rezultat ima snažne posledice po kriptografiju. Praktično svaka tvrdnja koja se javlja u kriptografskim protokolima može se dokazati uz nulto znanje, bez obzira na njenu strukturu.

Pod pretpostavkom postojanja jednosmernih funkcija, odnosno funkcija koje se

moгу efikasno izračunati, ali ih je računski neizvodljivo invertovati, važi jednakost

$$\mathbf{CZK} = \mathbf{IP} = \mathbf{PSPACE},$$

Kao ilustracija jednosmerne funkcije često se navodi množenje velikih prostih brojeva: lako je izračunati proizvod $n = p \cdot q$, dok se njegovom faktORIZACIJOM ne može efikasno odrediti početni par prostih brojeva. Iz navedene jednakosti sledi da se svaki problem koji ima interaktivni dokaz može dokazati i uz očuvanje svojstva nultog znanja.

5.4 Sigma protokoli

Protokol izomorfizma grafova ima jasnu teorijsku vrednost, ali u praksi nije najefikasniji. Sigma protokoli (engl. *Sigma protocols*) predstavljaju efikasniju klasu dokaza nultog znanja, zasnovanu na istoj trokoračnoj strukturi:

$$\text{posveta} \longrightarrow \text{izazov} \longrightarrow \text{odgovor}$$

Kod Sigma protokola uobičajeno se pretpostavlja pošten verifikator (engl. *honest verifier*), verifikator koji izazov bira u skladu sa specifikacijom protokola, što omogućava jednostavniju analizu svojstava protokola.

Jedan od najpoznatijih predstavnika ove klase jeste Šnorov identifikacioni protokol (engl. *Schnorr identification protocol*), koji se zasniva na problemu rešavanja diskretnog logaritma. Neka je g generator konačne grupe G reda q , gde je q prost broj. Takva grupa naziva se grupom prostog reda². Neka je $y = g^x$ javna vrednost. Pegi želi da ubedi Viktora da zna tajni eksponent x , a da ga pri tome ne otkrije. Protokol se odvija u tri koraka:

$$P \rightarrow V : r \leftarrow g^k, \quad k \leftarrow \mathbb{Z}_q,$$

$$V \rightarrow P : e \leftarrow \mathbb{Z}_q,$$

$$P \rightarrow V : s \leftarrow k + x \cdot e \pmod{q}.$$

Viktor prihvata dokaz ako važi $r = g^s \cdot y^{-e}$. Ispravnost ove provere sledi direktno iz

²Grupe prostog reda imaju jednostavnu algebarsku strukturu. Svaki element različit od neutralnog generiše celu grupu, pa ne postoje netrivialne podgrupe koje bi dodatno komplikovale analizu protokola. Zbog toga se često koriste u konstrukcijama zasnovanim na problemu diskretnog logaritma.

konstrukcije: ako Pegi zaista zna x , tada

$$g^s \cdot y^{-e} = g^{k+x \cdot e} \cdot g^{-x \cdot e} = g^k = r.$$

Ako Pegi ne zna x , mora unapred da pogodi vrednost izazova e pre nego što ga Viktor pošalje. Pošto se e bira uniformno iz skupa od q elemenata, verovatnoća ispravnog pogađanja iznosi $1/q$, za razliku od protokola za izomorfizam grafova, gde je izazov bio binaran i ta verovatnoća iznosila $1/2$. Zbog toga Šnorov protokol pruža jaku bezbednost već u jednoj rundi.

Šnorov protokol može se proširiti kako bi se dokazalo ne samo poznavanje diskretnog logaritma, već i jednakost dva diskretna logaritma. Tu ideju realizuje Chaum-Pedersen protokol (engl. *Chaum-Pedersen protocol*), koji je prvobitno razvijen za potrebe sistema elektronskog novca, ali danas ima široku primenu. Pegi želi da dokaže da za javne vrednosti $y_1 = g^{x_1}$ i $y_2 = h^{x_2}$ važi $x_1 = x_2$, pri čemu g i h generišu grupu prostog reda q . Zajednički diskretni logaritmi označavamo sa x .

Protokol se može posmatrati kao dva istovremena izvođenja Šnorovog protokola nad različitim bazama, ali sa jednim zajedničkim izazovom:

$$P \rightarrow V : (r_1, r_2) \leftarrow (g^k, h^k),$$

$$V \rightarrow P : e \leftarrow \mathbb{Z}_q,$$

$$P \rightarrow V : s \leftarrow k - e \cdot x \pmod{q}$$

Viktor prihvata dokaz ako važi $r_1 = g^s \cdot y_1^e$ i $r_2 = h^s \cdot y_2^e$.

Potpunost sledi direktno iz konstrukcije: ako Pegi zaista zna x , verifikacioni uslov je uvek zadovoljen, jer

$$g^s \cdot y_1^e = g^{k-ex} \cdot g^{ex} = g^k = r_1,$$

i analogno $h^s \cdot y_2^e = h^k = r_2$. Svojstvo nultog znanja sledi iz toga što simulator $S'(e, s) := (g^s \cdot y_1^e, h^s \cdot y_2^e)$ proizvodi transkripte nerazlučive od stvarnih. Tačnost se pokazuje argumentom specijalne tačnosti: pretpostavimo da Pegi može da prođe verifikaciju za dva različita izazova $e \neq e'$ sa istom posvetom (r_1, r_2) i odgovorima s i s' . Tada važi

$$r_1 = g^s \cdot y_1^e \quad \text{i} \quad r_1 = g^{s'} \cdot y_1^{e'}.$$

Izjednačavanjem dobijamo $g^s \cdot y_1^e = g^{s'} \cdot y_1^{e'}$, odakle sledi

$$y_1^{e-e'} = g^{s'-s}.$$

Analogno, iz uslova za r_2 dobijamo $y_2^{e-e'} = h^{s'-s}$. Iz ove dve jednačine sledi

$$(e - e') \cdot \log_g y_1 = s' - s = (e - e') \cdot \log_h y_2,$$

što potvrđuje da su diskretni logaritmi jednaki i da se zajednička vrednost može izračunati kao

$$x = \frac{s - s'}{e - e'} \pmod{q}.$$

Dakle, Pegi koja ne zna x ne može da konstruiše posvetu koja će proći verifikaciju za više od jednog izazova, što osigurava tačnost protokola. Primetimo da je dobijena vrednost zaista traženi diskretni logaritam, jer iz $y_1 = g^x$ direktno sledi $x = \log_g y_1$.

5.4.1 Fiat-Shamir heuristika i neinteraktivni dokazi

Dokazi nultog znanja koje smo do sada razmatrali su interaktivni. Dokazivač i verifikator razmenjuju poruke u realnom vremenu, pri čemu je upravo ta interakcija ono što obezbeđuje bezbednost. Međutim, u praksi je interakcija često nepoželjna ili neizvodljiva, na primer kada jedan entitet želi da dokaže ispravnost podataka svima koji ih budu čitali, bez mogućnosti direktne komunikacije sa svakim verifikatorom posebno. Zbog toga se uvode neinteraktivni dokazi nultog znanja (engl. *non-interactive zero-knowledge proofs*, NIZK), u kojima dokazivač samostalno konstruiše dokaz koji bilo ko može naknadno proveriti.

Standardan način pretvaranja interaktivnog Sigma protokola u neinteraktivni dokaz jeste Fiat-Shamir heuristika (engl. *Fiat-Shamir heuristic*). Ključna ideja je da se verifikatorov nasumični izazov e zameni heš-vrednošću javnih podataka. Na primeru Šnorovog protokola, gde je g generator grupe, $y = g^x$ javna vrednost čiji se diskretni logaritam dokazuje, a $r = g^k$ posveta koju dokazivač šalje verifikatoru, izazov se definiše kao

$$e = H(r \parallel g \parallel y).$$

Uključivanje g i y u heš je važno. Time se osigurava da je dokaz vezan za konkretnu tvrdnju i konkretne javne parametre, a ne samo za posvetu r . Bez toga, isti dokaz mogao bi se ponovo upotrebiti u drugom kontekstu.

Ista transformacija može se primeniti i za konstrukciju šeme digitalnog potpisa.

Umesto javnih parametara, u heš se uključuje poruka m koja se potpisuje. Javni ključ je tvrdnja X , tajni ključ je svedok w , a potpis poruke m dobija se računanjem

$$r \leftarrow R(w, k), \quad e \leftarrow H(r \parallel m), \quad s \leftarrow S(e, w, k).$$

Verifikator koji želi da proveriti potpis (e, s) na poruci m rekonstruiše posvetu kao $r' \leftarrow S'(e, s)$ i proverava da li važi $e = H(r' \parallel m)$. Ako jednakost važi, potpis je ispravan. Bezbednost ovakve konstrukcije standardno se razmatra uz pretpostavku da se heš funkcija ponaša kao idealna nasumična funkcija.

5.5 Primena u elektronskom glasanju

Dokazi nultog znanja prirodno se kombinuju sa drugim kriptografskim primitivama. Kao ilustraciju razmatramo protokol elektronskog glasanja u kome m glasača glasaju između dve opcije, a n centara za prebrojavanje zajednički utvrđuju rezultat. Cilj je da svako može proveriti ispravnost rezultata, a da pri tome nijedan pojedinačni glas ne bude otkriven.

Pre glasanja svaki centar objavljuje javni ključ za šifrovanje, fiksira se grupa i biraju se javni parametri takvi da nijedna strana ne zna njihov međusobni diskretni logaritam. Svaki glasač poseduje par ključeva za digitalno potpisivanje, čime se osigurava da glasaju samo ovlašćene osobe.

Kada glasač želi da glasa, bira jednu od dve vrednosti i objavljuje Pedersen-ovu posvetu³ svog glasa. Sama posveta ne otkriva vrednost glasa, ali glasač uz nju objavljuje i neinteraktivni dokaz nultog znanja kojim dokazuje da je glas zaista iz dozvoljenog skupa vrednosti, bez otkrivanja koje od dve opcije je izabrao. Time se sprečava mogućnost upisivanja nevalidne vrednosti, a anonimnost glasa ostaje očuvana.

Da bi se omogućilo prebrojavanje bez otkrivanja pojedinačnih glasova, svaki glasač svoju tajnu deli između centara koristeći Shamir-ovo tajno deljenje⁴ i svakom centru šalje odgovarajući šifrovani udeo. Javno objavljuje i posvete koeficijentima, što centrima omogućava da provere konzistentnost primljenih udela sa objavljenim glasom.

Na kraju svaki centar javno objavljuje zbir udela koje je primio, a bilo ko može proveriti ispravnost tog izračunavanja. Konačni zbir glasova dobija se interpolacijom

³Pedersen-ova šema posvete obrađena je u poglavlju 4.3.2

⁴Shamir-ovo tajno deljenje obrađeno je u poglavlju 2.3

iz dovoljnog broja objavljenih vrednosti. Ukoliko je zbir negativan, većina je glasala za jednu opciju, a ako je pozitivan, za drugu. Rezultat je tako javno proverljiv, dok individualni glasovi ostaju trajno skriveni.

5.6 Zaključak

Dokazi nultog znanja predstavljaju jedan od najelegantnijih rezultata moderne kriptografije jer omogućavaju da se istinitost tvrdnje dokaže bez otkrivanja ikakve informacije osim same istinitosti. Ova naizgled paradoksalna mogućnost ima duboke teorijske korene, svaka tvrdnja koja ima kratak dokaz može se dokazati uz nulto znanje, a uz odgovarajuće pretpostavke taj rezultat se proteže na celu klasu **PSPACE**.

U praksi, značaj dokaza nultog znanja leži u tome što omogućavaju bezbednu autentifikaciju bez otkrivanja lozinke, digitalne potpise čija se ispravnost može proveriti bez uvida u tajni ključ i sisteme glasanja u kojima se tačnost rezultata može javno verifikovati a da nijedan pojedinačni glas ne bude otkriven. Sigma protokoli, kao efikasna praktična realizacija ove ideje, danas su sastavni deo brojnih kriptografskih standarda i protokola.

Glava 6

ZK-snark

Dokazi nultog znanja (*Zero-Knowledge Proofs*, ZKP) predstavljaju kriptografske protokole koji omogućavaju da dokazivač (engl. *prover*) ubedi verifikatora (engl. *verifier*) u istinitost određene tvrdnje bez otkrivanja dodatnih informacija o samoj tvrdnji. U prethodnom poglavlju predstavljeni su osnovni koncepti dokaza nultog znanja i svojstva koja ovakvi protokoli moraju da zadovolje. Iako su prvobitno razvijeni kao teorijski koncept, potreba za pojačanom zaštitom privatnosti u distribuiranim sistemima dovela je do razvoja efikasnijih i praktičnijih rešenja. Poseban izazov predstavljala je činjenica da su klasični protokoli dokaza nultog znanja uglavnom interaktivni i zahtevaju višestruku razmenu poruka između učesnika. Kao odgovor na ovo ograničenje razvijeni su *zk-SNARK* sistemi, koji omogućavaju neinteraktivno dokazivanje uz kratke dokaze i brzu verifikaciju.

6.1 Uvod u zk-SNARK

Jedan od najznačajnijih koraka ka praktičnoj primeni dokaza nultog znanja predstavlja razvoj zk-SNARK sistema. Naziv zk-SNARK predstavlja skraćenicu od izraza *Zero-Knowledge Succinct Non-Interactive Argument of Knowledge*. Sam naziv opisuje ključne osobine sistema: očuvanje privatnosti, kratke dokaze i mogućnost njihove verifikacije bez potrebe za višestrukom komunikacijom između učesnika.

6.1.1 Definicija i osnovna ideja

Definicija 6.1.1. *zk-SNARK* predstavlja neinteraktivni dokaz nultog znanja kojim **dokazivač** može da ubedi **verifikatora** da poseduje određeno znanje ili da je izvršio određeni proračun na ispravan način, bez otkrivanja samog znanja ili međurezultata

korišćenih tokom izračunavanja.

U formalnom modelu zk-SNARK sistema razlikuju se **javna tvrdnja** (engl. *statement*) i **svedok** (engl. *witness*). Javna tvrdnja predstavlja informaciju koja je dostupna verifikatoru, dok svedok predstavlja dodatne podatke poznate samo dokazivaču. Cilj dokazivača jeste da pokaže da poseduje svedoka koji potvrđuje ispravnost javne tvrdnje, bez njegovog otkrivanja [23].

Ovakav odnos može se formalno predstaviti pomoću **jednakosti**

$$R(x, w) = 1$$

gde je x javna tvrdnja, w svedok, a R relacija koja proverava da li dati par zadovoljava zadate uslove. Formalnije, relacija R određuje jezik

$$L_R = \{x \mid \exists w \text{ takvo da } R(x, w) = 1\}.$$

Drugim rečima, dokazivač želi da ubedi verifikatora da za javnu tvrdnju x postoji odgovarajući svedok w , pri čemu se vrednost w ne otkriva. [23].

Osnovna ideja zk-SNARK sistema jeste da se složen računarski problem predstavi u obliku skupa matematičkih ograničenja. Na osnovu javne tvrdnje i svedoka generiše se kriptografski dokaz kojim se potvrđuje da su sva ograničenja zadovoljena. Verifikator zatim proverava samo validnost dokaza, bez pristupa podacima korišćenim tokom izračunavanja.

Jedna od ključnih osobina zk-SNARK sistema jeste zahtev da dokaz bude kratak i da se može proveriti znatno efikasnije nego što bi bilo ponovno izvršavanje celog proračuna [23]

6.1.2 Od interaktivnih ka neinteraktivnim dokazima

Za dokaz kažemo da je interaktivan ukoliko se njegova provera odvija kroz više uzastopnih razmena poruka između dokazivača i verifikatora. Verifikator u takvom protokolu šalje izazove, a dokazivač na njih odgovara, pri čemu se kroz tok komunikacije proverava istinitost tvrdnje. Takav pristup predstavlja osnovu velikog broja ranih sistema dokaza nultog znanja.

Iako interaktivni protokoli pružaju visok nivo bezbednosti, njihova primena u distribuiranim sistemima može biti otežana. Potreba za višestrukom razmenom poruka povećava komunikacione troškove i zahteva da obe strane učestvuju u protokolu u

isto vreme. Ovakva ograničenja posebno dolaze do izražaja u blokčejn mrežama, gde veliki broj učesnika mora efikasno da proverava validnost transakcija.

Razvoj **neinteraktivnih dokaza nultog znanja** omogućio je prevazilaženje ovog problema. Umesto višestruke komunikacije između učesnika, dokazivač generiše jedinstven dokaz koji se dostavlja verifikatoru. Nakon prijema dokaza, verifikator može samostalno da proveriti njegovu validnost bez dodatne interakcije [23].

Zk-SNARK predstavlja jednu od najuspešnijih realizacija ovog koncepta. Zbog neinteraktivnog načina rada, ovakvi sistemi su posebno pogodni za okruženja u kojima veliki broj učesnika mora samostalno da proverava dokaze.

6.1.3 Trusted setup

Većina tradicionalnih zk-SNARK sistema zahteva inicijalnu fazu poznatu kao **trusted setup**. Tokom ove faze generišu se javni parametri koji će kasnije biti korišćeni za kreiranje i proveru dokaza. Ovi parametri predstavljaju osnovu na kojoj počiva bezbednost celog sistema [23].

Prilikom generisanja parametara koriste se određene tajne vrednosti koje se moraju ukloniti po završetku postupka. Ukoliko bi napadač došao u posed ovih podataka, mogao bi da generiše lažne dokaze koji bi bili prihvaćeni kao validni, čime bi bezbednost sistema bila ozbiljno ugrožena.

Zbog toga se trusted setup često sprovodi kroz **poseban postupak generisanja parametara** (engl. *setup ceremony*) u kome učestvuju više nezavisnih strana. Ideja ovakvog pristupa jeste da bezbednost sistema ostane očuvana ukoliko makar jedan učesnik pravilno uništi svoj deo tajnih podataka nakon završetka postupka.

Potreba za trusted setup procedurom predstavlja jedan od najčešće navođenih nedostataka zk-SNARK sistema.

6.1.4 Setup, Prove i Verify algoritmi

Rad zk-SNARK sistema može se opisati kroz tri osnovne faze: **generisanje parametara** (engl. *Setup*), **kreiranje dokaza** (engl. *Prove*) i **verifikaciju dokaza** (engl. *Verify*) [23].

Tokom prve faze generišu se javni parametri koji će kasnije biti korišćeni za kreiranje i proveru dokaza. Rezultat ove faze predstavlja par ključeva

$$(pk, vk) \leftarrow Setup()$$

gde pk označava ključ za generisanje dokaza (engl. *proving key*), dok vk označava ključ za verifikaciju dokaza (engl. *verification key*).

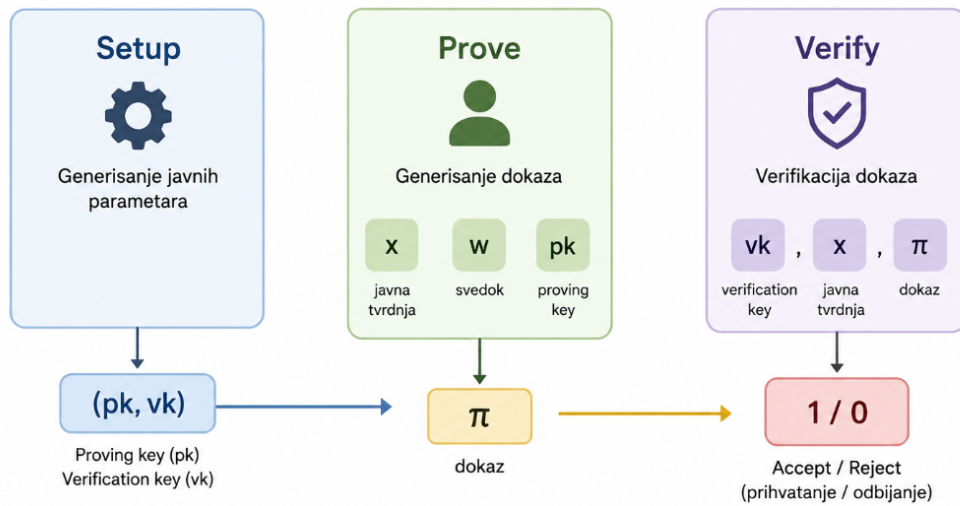
Nakon toga dokazivač koristi javnu tvrdnju x , svedoka w i ključ pk kako bi generisao dokaz

$$\pi \leftarrow Prove(pk, x, w)$$

Generisani dokaz π zatim se dostavlja verifikatoru, koji proverava njegovu ispravnost primenom algoritma

$$Verify(vk, x, \pi) = 1$$

Vrednost 1 označava da je dokaz prihvaćen kao validan, odnosno da dokazivač poseduje odgovarajućeg svedoka za datu javnu tvrdnju. Pri tome verifikator ne dobija nikakve informacije o samom svedoku niti o podacima korišćenim tokom generisanja dokaza.



Slika 6.1: Osnovni tok rada zk-SNARK sistema

Na ovaj način postiže se osnovni cilj dokaza nultog znanja: potvrda ispravnosti određene tvrdnje bez otkrivanja informacija koje su korišćene za njeno dokazivanje. Upravo kombinacija privatnosti, kratkih dokaza i brze verifikacije omogućila je široku primenu zk-SNARK sistema u savremenim distribuiranim okruženjima.

6.2 Primena zk-SNARK sistema u mreži Zcash

Jedan od najpoznatijih primera upotrebe zk-SNARK tehnologije predstavlja kriptovaluta Zcash, koja koristi dokaze nultog znanja kako bi omogućila privatne transakcije uz zadržavanje mogućnosti javne verifikacije njihove ispravnosti.

6.2.1 Problem privatnosti javnih blokčejn mreža

Blokčejn se može posmatrati kao distribuirana i hronološki uređena evidencija transakcija. Transakcije se grupišu u blokove, a svaki novi blok se povezuje sa prethodnim pomoću kriptografskih heš vrednosti. Na taj način nastaje lanac blokova koji je teško naknadno menjati, jer bi promena jednog bloka zahtevala promenu svih narednih blokova i prihvatanje takve izmene od strane mreže.

Jedna od najvažnijih karakteristika javnih blokčejn mreža jeste **transparentnost**. Svi učesnici mreže imaju pristup istoj evidenciji transakcija, što omogućava nezavisnu proveru ispravnosti sistema bez potrebe za centralnim autoritetom. Upravo ova osobina predstavlja osnovu poverenja u mreže poput Bitcoina.

Međutim, potpuna transparentnost sa sobom donosi i određene probleme. Iako se identitet korisnika u Bitcoin mreži ne predstavlja direktno imenom i prezimenom, već **kriptografskim adresama**, sve transakcije ostaju trajno javno dostupne. Zbog toga se Bitcoin često opisuje kao **pseudoanoniman** (engl. *pseudonymous*), a ne kao anoniman sistem.

Svaka transakcija sadrži informacije o adresi pošiljaoca, adresi primaoca i iznosu koji se prenosi. Budući da su svi ovi podaci javno dostupni, moguće je analizirati tokove novca i pratiti kretanje sredstava između različitih adresa. Vremenom se na osnovu obrazaca ponašanja, javno objavljenih adresa ili podataka prikupljenih sa menjačnica kriptovaluta pojedine adrese mogu povezati sa stvarnim identitetima korisnika.

Nakon uspostavljanja takve veze moguće je rekonstruisati značajan deo finansijske aktivnosti korisnika, uključujući podatke o primljenim i poslatim sredstvima, učestalosti transakcija i povezanosti sa drugim učesnicima mreže. Iako ovakva transparentnost doprinosi **proverljivosti sistema**, ona istovremeno može predstavljati **ozbiljno narušavanje privatnosti**.

Potreba za očuvanjem privatnosti uz zadržavanje mogućnosti javne verifikacije transakcija predstavlja jedan od ključnih izazova savremenih blokčejn sistema. Upravo iz tog razloga razvijena su rešenja koja omogućavaju sakrivanje osetljivih podataka,

pri čemu se i dalje može proveriti da li su pravila sistema ispoštovana [11].

6.2.2 Kako izgleda jedna transakcija u blokčejn mreži

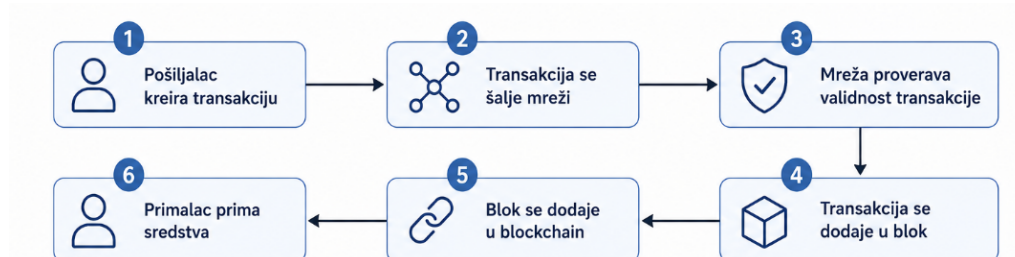
Da bi blokčejn mreža prihvatila transakciju, neophodno je proveriti nekoliko osnovnih uslova:

- da li pošiljalac poseduje sredstva koja želi da prenese;
-
- da ista sredstva nisu već iskorišćena u nekoj prethodnoj transakciji (**dvostruko trošenje**);
- ukupna količina sredstava u sistemu ostaje nepromenjena.

Primer 6.2.1. Posmatrajmo jednostavan primer u kome Ivana (pošiljalac) želi da pošalje Matiji (primalac) 1 BTC.

Ivana kreira transakciju i šalje je mreži na proveru. Nakon toga čvorovi mreže proveravaju da li su ispunjeni svi uslovi potrebni za njeno prihvatanje. Ukoliko je transakcija validna, ona se uključuje u novi blok i postaje deo javne evidencije transakcija.

Nakon potvrde transakcije Matija postaje vlasnik prenetih sredstava. Istovremeno, podaci o transakciji ostaju trajno zapisani u blokčejnu i dostupni svim učesnicima mreže. Svako može da vidi sa koje adrese su sredstva poslata, na koju adresu su preneti i koliki iznos je učestvovao u transakciji.



Slika 6.2: Klasična blokčejn transakcija i proces validacije

Sa aspekta bezbednosti, ovakav model funkcioniše veoma efikasno jer omogućava svakom učesniku mreže da samostalno proveri ispravnost transakcije. Međutim, cena takve proverljivosti jeste potpuna transparentnost podataka o transakcijama. Svaki

učesnik mreže može da vidi ko učestvuje u transakciji i koliki iznos se prenosi, što predstavlja značajan izazov za očuvanje privatnosti korisnika.

6.2.3 Šta je javno, a šta Zcash skriva

Zcash je razvijen sa ciljem da omogući privatne finansijske transakcije bez ugrožavanja bezbednosti mreže. Za razliku od Bitcoina, gde su gotovo svi podaci o transakciji javni, Zcash omogućava sakrivanje ključnih informacija uz istovremeno zadržavanje mogućnosti njihove kriptografske provere.

Osnovu sistema čine dve vrste adresa. Prva grupa su transparentne adrese, koje funkcionišu na sličan način kao adrese u Bitcoin mreži. Drugu grupu čine **zaštićene adrese** (engl. *shielded addresses*), koje omogućavaju realizaciju privatnih transakcija.

Kada se sredstva prenose između zaštićenih adresa, nastaju **zaštićene transakcije** (engl. *shielded transactions*). U tom slučaju podaci o učesnicima transakcije i iznosu koji se prenosi mogu ostati skriveni od javnosti, dok mreža i dalje može da proveri validnost transakcije.

Karakteristika	Bitcoin	Zcash
Pošiljalac	Vidljiv	Može biti skriven
Primalac	Vidljiv	Može biti skriven
Iznos transakcije	Vidljiv	Može biti skriven
Validnost transakcije	Javno proverljiva	Javno proverljiva
Zaštita privatnosti	Ograničena	Visoka

Tabela 6.1: Poređenje privatnosti i verifikacije u Bitcoin i Zcash mrežama

Kod zaštićenih Zcash transakcija mogu biti sakriveni identitet pošiljaoca, identitet primaoca i iznos koji se prenosi. Sa druge strane, sama činjenica da je transakcija validna ostaje proverljiva svim učesnicima mreže.

Na prvi pogled ovakav pristup deluje paradoksalno. Ukoliko su podaci o transakciji skriveni, postavlja se pitanje na koji način mreža može da proveri da korisnik zaista poseduje sredstva koja želi da prenese i da pritom ne pokušava da prevari sistem. Odgovor na ovo pitanje daje primena zk-SNARK dokaza, koji omogućavaju proveru validnosti transakcije bez otkrivanja samih podataka.

6.2.4 Validacija transakcija pomoću zk-SNARK dokaza

Kod klasičnih blokčejn mreža validnost transakcije proverava se direktnim uvidom u podatke o transakciji. Pošto su adrese učesnika i iznos koji se prenosi javno dostupni,

svaki čvor može samostalno da proveriti da li su ispunjena pravila sistema.

Kod zaštićenih Zcash transakcija ovakav pristup nije moguć jer su ključni podaci skriveni. Zbog toga se validacija ne zasniva na proveru samih podataka, već na proveru kriptografskog dokaza koji potvrđuje da su sva pravila sistema ispoštovana [11].

Uslovi koje zaštićena transakcija mora da zadovolji mogu se opisati jednakošću $R(x, w)$, gde javna tvrdnja x sadrži podatke potrebne mreži za proveru dokaza, dok svedok w obuhvata privatne podatke transakcije, kao što su tajni ključevi, informacije o ulaznim sredstvima i vrednosti koje ne treba javno otkriti.

U pojednostavljenom modelu, skup ograničenja može se posmatrati kroz sledeće uslove:

- pošiljalac poseduje odgovarajuće privatne podatke kojima dokazuje pravo da potroši ulazna sredstva;
- za svako ulazno sredstvo proverava se da nije prethodno potrošeno;
- ukupna vrednost ulaza jednaka je ukupnoj vrednosti izlaza, odnosno

$$\sum v_{\text{ulaz}} = \sum v_{\text{izlaz}};$$

- podaci koji se javno objavljuju moraju biti konzistentni sa privatnim podacima korišćenim u dokazu.

Pre generisanja dokaza transakcija se prevodi u skup matematičkih ograničenja koja opisuju pravila sistema. Ta ograničenja proveravaju da li pošiljalac poseduje sredstva koja želi da prenese, da li ista sredstva nisu prethodno potrošena i da li je očuvana ukupna vrednost transakcije. Na osnovu privatnih podataka i javnih parametara sistema korisnik zatim konstruiše zk-SNARK dokaz kojim pokazuje da su sva navedena ograničenja zadovoljena.

Čvorovi mreže proveravaju **isključivo validnost dostavljenog dokaza**, bez pristupa privatnim informacijama. Značajna prednost ovakvog pristupa jeste činjenica da je verifikacija veoma brza i zahteva malu količinu računarskih resursa. Na taj način postiže se ravnoteža između dva naizgled suprotstavljena zahteva: zaštite privatnosti i mogućnosti javne verifikacije transakcija.

6.2.5 Tok jedne Zcash transakcije

Proces izvršavanja zaštićene transakcije u mreži Zcash može se predstaviti kroz nekoliko uzastopnih koraka.

Primer 6.2.2. Posmatrajmo isti scenario kao u prethodnom primeru. Ivana želi da pošalje 1 ZEC Matiji.

Za razliku od klasične blokčejn transakcije, podaci o pošiljaocu, primaocu i iznosu transakcije ne moraju biti javno dostupni. Ivana najpre formira transakciju i na osnovu privatnih podataka generiše zk-SNARK dokaz kojim pokazuje da poseduje sredstva koja želi da prenese i da su zadovoljena sva pravila sistema.

Nakon toga transakcija zajedno sa generisanim dokazom šalje se mreži na proveru. Čvorovi mreže ne dobijaju pristup privatnim podacima transakcije, već proveravaju isključivo validnost dostavljenog dokaza.

Ukoliko je dokaz validan, transakcija se prihvata i uključuje u blok. Nakon potvrde transakcije Matija postaje vlasnik prenetih sredstava, dok podaci o pošiljaocu, primaocu i iznosu ostaju zaštićeni od javnosti.



Slika 6.3: Tok jedne Zcash transakcije sa zk-SNARK dokazom

Tok transakcije prikazan na slici 6.3 može se posmatrati kroz četiri osnovna koraka:

1. Pošiljalac kreira transakciju i generiše zk-SNARK dokaz.
2. Transakcija i dokaz šalju se Zcash mreži.
3. Čvorovi mreže proveravaju validnost dostavljenog dokaza.
4. Nakon uspešne verifikacije transakcija se prihvata i uključuje u blok, a primalac postaje vlasnik sredstava.

Važno je naglasiti da tokom čitavog procesa verifikatori **ne znaju identitet pošiljaoca, identitet primaoca niti iznos koji se prenosi**. Jedina informacija koju dobijaju jeste da su **sva pravila sistema ispoštovana i da je transakcija validna** [11].

6.3 Prednosti i nedostaci zk-SNARK sistema

6.3.1 Prednosti

- **Kompaktnost dokaza**

Jedna od najvažnijih karakteristika zk-SNARK sistema jeste mogućnost generisanja veoma kratkih dokaza. Veličina dokaza ostaje mala čak i kada se dokazuje ispravnost veoma složenih proračuna. Ova osobina omogućava efikasno skladištenje i prenos dokaza u distribuiranim sistemima [23].

- **Brza verifikacija**

Provera validnosti zk-SNARK dokaza zahteva relativno malu količinu računarskih resursa. Zbog toga veliki broj korisnika može nezavisno da proverava dokaze bez značajnog opterećenja sistema. Ovakva osobina posebno je značajna u blokčejn mrežama, gde veliki broj čvorova mora da proverava validnost transakcija [23].

- **Visok nivo privatnosti**

Dokazi nultog znanja omogućavaju proveru ispravnosti određene tvrdnje bez otkrivanja podataka na kojima se dokaz zasniva. Na taj način moguće je očuvati privatnost korisnika uz istovremeno zadržavanje javne proverljivosti sistema. Upravo ova osobina predstavlja osnovu primene zk-SNARK tehnologije u mreži Zcash [11].

- **Pogodnost za blokčejn mreže**

Kombinacija kratkih dokaza i brze verifikacije čini zk-SNARK sisteme posebno pogodnim za primenu u blokčejn mrežama. Na ovaj način moguće je proveravati validnost velikog broja transakcija bez značajnog povećanja komunikacionih i računarskih troškova.

6.3.2 Nedostaci

- **Potreba za trusted setup procedurom**

Većina tradicionalnih zk-SNARK sistema zahteva inicijalnu fazu generisanja javnih parametara poznatu kao trusted setup. Ukoliko bi tajni podaci korišćeni tokom ovog procesa bili kompromitovani, bilo bi moguće generisati lažne dokaze koji prolaze verifikaciju. Zbog toga se trusted setup smatra jednim od najznačajnijih ograničenja tradicionalnih zk-SNARK sistema [23].

- **Visoka cena generisanja dokaza**

Iako je verifikacija dokaza veoma brza, njihovo generisanje može zahtevati značajne računске resurse.

- **Složena implementacija**

Praktična realizacija zk-SNARK sistema zahteva poznavanje naprednih oblasti moderne kriptografije, uključujući aritmetička kola, polinomske reprezentacije i algebarske strukture nad konačnim poljima. Zbog toga razvoj i održavanje ovakvih sistema predstavljaju značajan inženjerski izazov.

6.4 Zaključak

Dokazi nultog znanja predstavljaju jednu od najznačajnijih oblasti savremene kriptografije jer omogućavaju dokazivanje ispravnosti određene tvrdnje **bez otkrivanja informacija** na kojima se dokaz zasniva. Razvoj zk-SNARK sistema omogućio je da se ovaj koncept primeni u praksi kroz generisanje kompaktnih dokaza koji se mogu brzo proveravati i koristiti u sistemima sa velikim brojem učesnika.

Poseban značaj zk-SNARK tehnologije ogleda se u njenoj primeni u mreži **Zcash**. Korišćenjem dokaza nultog znanja moguće je ostvariti privatne transakcije pri čemu se istovremeno zadržava mogućnost javne verifikacije njihove ispravnosti. Na taj način postiže se ravnoteža između **zaštite privatnosti korisnika i očuvanja bezbednosti blokčejn mreže**.

Analizom prednosti i nedostataka može se zaključiti da zk-SNARK sistemi nude visok nivo privatnosti, kompaktne dokaze i brzu verifikaciju, ali da istovremeno donose određena ograničenja, među kojima se posebno izdvajaju potreba za trusted setup procedurom i složenost implementacije.

Dalji razvoj ove oblasti doveo je do pojave novih pristupa, među kojima se posebno izdvajaju zk-STARK sistemi. Za razliku od tradicionalnih zk-SNARK rešenja, **zk-**

STARK ne zahteva trusted setup proceduru i zasniva se na drugačijim kriptografskim pretpostavkama. Ipak, ova prednost dolazi uz određene kompromise, poput veće veličine dokaza i većih komunikacionih zahteva. Zbog toga se danas obe tehnologije aktivno razvijaju i pronalaze primenu u različitim oblastima savremene kriptografije.

Glava 7

ZK-stark

U prethodnom poglavlju, Poglavlju 6, predstavljani su zk-SNARK sistemi, njihove osnovne osobine i način na koji omogućavaju generisanje kratkih neinteraktivnih dokaza i njihovu efikasnu verifikaciju. Posebno je objašnjeno da mnoge zk-SNARK konstrukcije zahtevaju fazu poverljivog podešavanja (engl. *trusted setup*).

Tokom faze poverljivog podešavanja generišu se javni parametri koji se kasnije koriste za formiranje i proveru dokaza. Istovremeno nastaju i tajne pomoćne vrednosti. Te vrednosti moraju biti nepovratno uništene nakon završetka podešavanja. Ukoliko bi ih neko sačuvao ili naknadno kompromitovao, mogao bi da generiše validne dokaze za netačne tvrdnje, čime bi bila narušena ispravnost čitavog sistema.

Potreba za dokaznim sistemom koji zadržava skalabilnost i svojstvo nultog znanja, ali ne zahteva fazu poverljivog podešavanja, dovela je do razvoja zk-STARK sistema. Naziv zk-STARK predstavlja skraćenicu izraza *Zero-Knowledge Scalable Transparent Argument of Knowledge*.

Kao i zk-SNARK, zk-STARK omogućava dokazivaču da ubedi verifikatora u ispravnost određenog izračunavanja bez otkrivanja tajnog svedoka ili poverljivih ulaznih podataka. Međutim, zk-STARK koristi drugačiji pristup. Umesto poverljivo generisanih parametara i konstrukcija zasnovanih na eliptičkim krivama, oslanja se na javnu nasumičnost, kriptografske heš funkcije, polinomske reprezentacije i kodove za korekciju grešaka [2].

U nastavku se izraz STARK koristi kada se govori o osnovnoj arhitekturi dokaznog sistema, dok se izraz zk-STARK koristi kada se posebno naglašava svojstvo nultog znanja.

Glavne osobine zk-STARK sistema sadržane su u samom nazivu:

- **Nulto znanje** (engl. *zero knowledge*) – dokaz ne otkriva tajni svedok niti dodatne informacije o izračunavanju, osim činjenice da je tvrdnja tačna.
- **Skalabilnost** (engl. *scalability*) – vreme rada dokazivača raste približno kvazi-linearano sa veličinom izračunavanja, dok vreme rada verifikatora raste znatno sporije.
- **Transparentnost** (engl. *transparency*) – nije potrebna faza poverljivog podešavanja. Parametri sistema mogu biti javno generisani i provereni.
- **Argument znanja** (engl. *argument of knowledge*) – dokazivač može proizvesti validan dokaz samo ako poseduje odgovarajući svedok, osim sa zanemarljivom verovatnoćom.

zk-STARK sistemi praktično su predstavljeni u radu *Scalable, Transparent, and Post-Quantum Secure Computational Integrity* autora Elija Ben-Sasona i saradnika [2]. Njihov cilj nije bio samo očuvanje privatnosti, već i efikasna provera integriteta velikih izračunavanja nad poverljivim podacima.

7.1 Motivacija i računarski integritet

Posmatrajmo organizaciju koja poseduje veliki skup poverljivih podataka D i izvršava program C nad tim podacima. Organizacija zatim objavljuje rezultat y .

Veza između programa, ulaznih podataka i rezultata može se zapisati kao

$$y = C(D). \quad (7.1)$$

Postavlja se pitanje kako nezavisni verifikator može biti siguran da je rezultat zaista dobijen korektnim izvršavanjem programa C , a ne proizvoljno izabran ili falsifikovan.

Najjednostavniji način provere bio bi da verifikator dobije podatke D , ponovo izvrši program i proveriti da li je dobijeni rezultat jednak y . Ovakav pristup ima dva ozbiljna problema. Prvo, ukoliko su podaci poverljivi, njihovo otkrivanje nije dozvoljeno. Drugo, ponovno izvršavanje programa može biti jednako skupo kao i originalno izračunavanje, pa verifikacija ne donosi nikakvu uštedu.

zk-STARK rešava ovaj problem tako što dokazivač generiše kriptografski dokaz π da postoji skup podataka D za koji važi

$$C(D) = y. \quad (7.2)$$

Verifikator pri tome ne dobija podatke D , niti mora ponovo da izvrši program C . Umesto toga, proverava samo dokaz π , koristeći relativno mali broj računarskih operacija.

U originalnom radu autori razmatraju primer nacionalne baze forenzičkih DNK profila. Policija tvrdi da se DNK profil određene osobe ne nalazi u bazi. Javnost ne želi da slepo veruje toj tvrdnji, dok policija ne sme da objavi celu bazu DNK profila. zk-STARK omogućava policiji da dokaže da je pretraga korektno izvršena i da profil nije pronađen, bez otkrivanja samog profila ili sadržaja baze [2].

7.2 Transparentnost i odsustvo poverljivog podešavanja

Jedna od najvažnijih razlika između mnogih zk-SNARK sistema i zk-STARK sistema odnosi se na način generisanja javnih parametara.

Kod brojnih zk-SNARK konstrukcija neophodno je sprovesti fazu poverljivog podešavanja. U toj fazi generišu se javni parametri koje kasnije koriste dokazivač i verifikator, kao i tajne pomoćne vrednosti koje ne smeju ostati dostupne nakon završetka postupka.

Tajne vrednosti nastale tokom poverljivog podešavanja često se neformalno nazivaju *toxic waste*. Ovaj izraz označava podatke koji su potrebni samo tokom generisanja javnih parametara, ali koji nakon toga moraju biti bezbedno i nepovratno uništeni. Ukoliko ih neko sačuva ili naknadno kompromituje, može steći mogućnost da generiše naizgled validne dokaze za netačne tvrdnje, čime se narušava ispravnost čitavog sistema.

zk-STARK ne zahteva ovakav postupak. Sistem koristi javno dostupnu nasumičnost i kriptografske heš funkcije, pa ne postoji tajni parametar čije bi čuvanje omogućilo falsifikovanje dokaza. Ovakav način inicijalizacije naziva se transparentnim podešavanjem (engl. *transparent setup*) [2, 33].

Transparentnost ne znači da sistem nema parametre. Potrebno je odabrati konačno polje, heš funkciju, parametre kodiranja, faktor proširenja i bezbednosni nivo. Razlika je u tome što su svi ti parametri javni i njihovo poznavanje ne omogućava generisanje lažnih dokaza.

7.3 Postkvantna sigurnost

Brojni zk-SNARK sistemi zasnivaju svoju sigurnost na problemu diskretnog logaritma nad eliptičkim krivama i na bilinearnim uparivanjima (engl. *bilinear pairings*) [23].

Problem diskretnog logaritma podrazumeva da je, za poznate elemente P i Q u grupi eliptičke krive, gde važi

$$Q = kP, \quad (7.3)$$

računarski teško odrediti tajni skalar k .

Bilinearno uparivanje je preslikavanje oblika

$$e: G_1 \times G_2 \longrightarrow G_T, \quad (7.4)$$

koje za elemente $P \in G_1$, $Q \in G_2$ i skalare a i b zadovoljava svojstvo

$$e(aP, bQ) = e(P, Q)^{ab}. \quad (7.5)$$

Ovo svojstvo omogućava da se određeni algebarski odnosi provere pomoću kratkih jednačina, što doprinosi maloj veličini dokaza i brznoj verifikaciji kod pairing zasnovanih zk-SNARK sistema. Bilinearna uparivanja, međutim, nisu sama po sebi pretpostavka težine. Sigurnost zavisi od problema diskretnog logaritma i srodnih problema u izabranim grupama.

Ovi problemi smatraju se teškim za klasične računare kada su parametri pravilno izabrani, ali nisu otporni na poznate kvantne napade. Šorov algoritam omogućava dovoljno snažnom kvantnom računararu da u polinomijalnom vremenu rešava faktORIZACIJU velikih celih brojeva i problem diskretnog logaritma. Zbog toga bi mogao da ugrozi RSA, kriptografiju eliptičkih krivih i zk-SNARK konstrukcije koje zavise od tih pretpostavki [30].

zk-STARK sistemi se ne zasnivaju na problemu diskretnog logaritma niti na bilinearnim uparivanjima. Njihova kriptografska sigurnost prvenstveno se oslanja na svojstva kriptografskih heš funkcija, dok se proveravanje polinomskih tvrdnji zasniva na Reed–Solomonovim kodovima i testovima bliskosti [2, 33].

Koliziona otpornost heš funkcije znači da je računarski teško pronaći dve različite poruke m_1 i m_2 takve da istovremeno važi

$$m_1 \neq m_2 \tag{7.6}$$

i

$$H(m_1) = H(m_2). \tag{7.7}$$

Takav par poruka naziva se kolizija. Koliziona otpornost ne znači da kolizije ne postoje, već da ih je za pravilno odabranu heš funkciju računarski neizvodljivo pronaći.

Kvantni algoritmi mogu ubrzati generičke napade na heš funkcije, ali ih ne narušavaju na isti način na koji Šorov algoritam narušava diskretni logaritam. Groverov algoritam ubrzava generičku pretragu preimage-a sa približno $O(2^n)$ na $O(2^{n/2})$ za heš izlaz dužine n bitova. Zbog toga se pri izboru parametara mora uzeti u obzir smanjen sigurnosni nivo u kvantnom modelu [5].

zk-STARK sistemi se zato opisuju kao verovatno postkvantno sigurni (engl. *plausibly post-quantum secure*) [2]. Ovaj izraz ne predstavlja apsolutnu garanciju otpornosti na sve buduće kvantne napade, već označava da sistem ne zavisi od matematičkih problema za koje već postoji poznat efikasan kvantni algoritam.

7.4 Od izvršavanja programa do polinomskih ograničenja

Verifikator ne može efikasno proveravati proizvoljno izvršavanje programa samo na osnovu malog broja nasumičnih provera. Zbog toga se izvršenje programa najpre prevodi u matematičku reprezentaciju zasnovanu na elementima konačnog polja i polinomskim relacijama. Ovaj postupak naziva se aritmetizacija (engl. *arithmetization*) [2].

Aritmetizacija ne menja rezultat programa, već njegovo izvršavanje opisuje na način pogodan za kriptografsku proveru. Stanja programa predstavljaju se elementima konačnog polja, dok se pravila prelaska iz jednog stanja u drugo izražavaju polinomskim jednačinama.

Osnovna struktura koja nastaje ovim postupkom naziva se trag izvršavanja (engl. *execution trace*). Trag izvršavanja je tabela koja sadrži međustanja programa. Svaki red predstavlja stanje u jednom koraku izvršavanja, dok kolone predstavljaju registre,

memorijske vrednosti ili druge promenljive koje opisuju stanje sistema.

Posmatrajmo program koji počinje vrednošću x_0 i u svakom koraku računa

$$x_{i+1} = x_i^2 + c, \quad (7.8)$$

gde je c javno poznata konstanta.

Ispravan prelaz između dva uzastopna stanja mora zadovoljavati polinomsku relaciju

$$x_{i+1} - x_i^2 - c = 0. \quad (7.9)$$

Na ovaj način korektnost izvršavanja programa svodi se na proveravanje da određene polinomske jednačine važe u svim odgovarajućim redovima traga.

Skup polinomskih pravila koja opisuju ispravno izvršavanje naziva se algebarska međureprezentacija (engl. *Algebraic Intermediate Representation*, AIR) [2, 33].

AIR obično sadrži dve vrste ograničenja:

- **granična ograničenja** (engl. *boundary constraints*), koja određuju početne i završne vrednosti;
- **tranziciona ograničenja** (engl. *transition constraints*), koja određuju dozvoljene prelaze između susednih redova traga.

7.5 Polinomi traga i proširenje domena

Nakon formiranja traga izvršavanja, svaka njegova kolona posmatra se kao skup vrednosti nekog polinoma nad konačnim poljem $\mathbb{F}$.

Neka se trag sastoji od n redova i neka je domen izvršavanja

$$H = \{\omega^0, \omega^1, \dots, \omega^{n-1}\}. \quad (7.10)$$

Za vrednosti jedne kolone

$$x_0, x_1, \dots, x_{n-1}, \quad (7.11)$$

postoji jedinstveni polinom $f(X)$ stepena manjeg od n takav da za svaki indeks i

važi

$$f(\omega^i) = x_i. \quad (7.12)$$

Dokazivač interpolira ovaj polinom, a zatim ga evaluira na većem domenu L . Ovaj postupak naziva se proširenje niskog stepena (engl. *low-degree extension*, LDE).

Ako je veličina originalnog domena n , a proširenog domena bn , broj b naziva se faktor proširenja.

Proširenje domena zasniva se na Reed–Solomonovim kodovima. Vektor evaluacija polinoma niskog stepena predstavlja kodnu reč Reed–Solomonovog koda. Interpolacija i evaluacija nad odgovarajuće strukturisanim domenima efikasno se izvršavaju FFT¹ algoritmima nad konačnim poljima. Za fiksnu širinu traga i odgovarajuće izabrane parametre, dominantne polinomske operacije imaju kvazilinearnu složenost, koja se u pojednostavljenom obliku često zapisuje kao

$$O(n \log n). \quad (7.13)$$

Ova procena ne obuhvata nužno sve troškove kompletnog dokazivača, jer ukupna složenost zavisi i od širine traga, stepena ograničenja i izabranih parametara protokola [2, 33].

7.6 Posvećivanje vrednostima pomoću Merkvog stabla

Dokazivač nakon proširenja domena poseduje veliki broj evaluacija polinoma. Kada bi sve te vrednosti neposredno poslao verifikatoru, dokaz bi bio veoma veliki i verifikacija ne bi bila skalabilna.

Umesto toga, dokazivač nad nizom vrednosti gradi Merkvlovo stablo (engl. *Merkle tree*). Listovi stabla sadrže heševe pojedinačnih vrednosti ili grupa vrednosti, dok svaki unutrašnji čvor sadrži heš svojih potomaka. Koren stabla predstavlja kratku kriptografsku posvetu celom skupu podataka.

Neka su vrednosti koje dokazivač želi da posveti

$$v_0, v_1, \dots, v_{m-1}. \quad (7.14)$$

¹FFT (engl. *Fast Fourier Transform*) označava brzu Furijeovu transformaciju.

Najpre se računaju listovi

$$h_i = H(v_i), \quad (7.15)$$

gde je H kriptografska heš funkcija.

Zatim se susedne vrednosti kombinuju prema pravilu

$$h_{i,j} = H(h_i \parallel h_j), \quad (7.16)$$

sve dok se ne dobije jedan koren stabla.

Dokazivač prvo objavljuje koren stabla. Nakon toga ne može neprimetno promeniti neku od posvećenih vrednosti bez promene korena. Verifikator zatim bira mali broj slučajnih pozicija i zahteva odgovarajuće vrednosti zajedno sa autentifikacionim putanjama.

Za proveru jednog lista nije potrebno dobiti celo stablo. Dovoljno je dobiti približno

$$\log_2 m \quad (7.17)$$

susednih heševa, gde je m broj listova [33].

7.7 Polinom kompozicije

AIR može sadržati veliki broj različitih ograničenja. Direktno proveravanje svakog ograničenja odvojeno povećalo bi veličinu dokaza. Zato se ograničenja kombinuju u polinom kompozicije (engl. *composition polynomial*).

Pretpostavimo da postoje polinomska ograničenja

$$C_1(X), C_2(X), \dots, C_k(X). \quad (7.18)$$

Verifikator bira nasumične koeficijente

$$\alpha_1, \alpha_2, \dots, \alpha_k \in \mathbb{F}, \quad (7.19)$$

a dokazivač formira njihovu linearnu kombinaciju

$$C(X) = \alpha_1 C_1(X) + \alpha_2 C_2(X) + \dots + \alpha_k C_k(X). \quad (7.20)$$

Nasumični koeficijenti sprečavaju dokazivača da unapred prilagodi greške tako da se one međusobno ponište.

Ograničenja se često dele polinomima koji nestaju na odgovarajućem domenu. Neka je

$$Z_H(X) = \prod_{h \in H} (X - h) \quad (7.21)$$

polinom nestajanja (engl. *vanishing polynomial*) domena H . Ako polinom ograničenja nestaje u svim tačkama domena, tada je deljiv sa $Z_H(X)$, pa odgovarajući količnik mora imati očekivani ograničeni stepen [2, 33].

7.8 FRI protokol

Ključna komponenta STARK sistema je FRI protokol, čiji naziv predstavlja skraćenicu izraza *Fast Reed-Solomon Interactive Oracle Proof of Proximity*. FRI je interaktivni dokaz bliskosti sa pristupom proročištu (engl. *Interactive Oracle Proof of Proximity*, IOPP), kojim verifikator proverava da li je prosledena funkcija bliska Reed–Solomonovoj kodnoj reči, odnosno evaluacijama polinoma ograničenog stepena [2, 33, 4].

Važno je razlikovati dve tvrdnje:

1. vrednosti zadovoljavaju AIR ograničenja;
2. vrednosti predstavljaju evaluacije polinoma dozvoljenog stepena.

Samo proveravanje lokalnih AIR ograničenja nije dovoljno, jer proizvoljan niz vrednosti ne mora predstavljati evaluacije jednog konzistentnog polinoma.

FRI koristi rekursivno presavijanje polinoma. Polinom $f(X)$ može se zapisati u obliku

$$f(X) = f_{\text{even}}(X^2) + X f_{\text{odd}}(X^2). \quad (7.22)$$

Verifikator bira slučajnu vrednost β , a dokazivač formira novi polinom

$$g(Y) = f_{\text{even}}(Y) + \beta f_{\text{odd}}(Y). \quad (7.23)$$

Ovaj postupak ponavlja se kroz više rundi:

$$f_0 \longrightarrow f_1 \longrightarrow f_2 \longrightarrow \cdots \longrightarrow f_r. \quad (7.24)$$

U svakoj rundi dokazivač se pomoću Merkvovog stabla posvećuje evaluacijama novog polinoma. Na kraju se dobija dovoljno mali polinom koji verifikator može neposredno proveriti.

Ako su početne vrednosti daleko od svake evaluacije polinoma dozvoljenog stepena, ponavljanje slučajnih upita smanjuje verovatnoću prihvatanja neispravnog dokaza [4].

7.9 DEEP-FRI

DEEP-FRI predstavlja unapređenje FRI protokola. Skraćenica DEEP potiče od izraza *Domain Extension for Eliminating Pretenders* [4].

Neispravna funkcija može na originalnom domenu biti relativno bliska većem broju različitih polinoma niskog stepena. Takvi polinomi predstavljaju lažne kandidate, odnosno *pretenders*. Verifikator zato bira nasumičnu tačku z izvan osnovnog domena evaluacije i od dokazivača zahteva vrednost

$$f(z). \quad (7.25)$$

Pošto različiti kandidati uglavnom daju različite vrednosti u tački z , odgovor dokazivača ga vezuje za znatno užu skup mogućih polinoma.

Zatim se razmatra polinom

$$g(X) = \frac{f(X) - f(z)}{X - z}. \quad (7.26)$$

Ako je f polinom stepena najviše d , tada je g polinom stepena najviše $d - 1$. FRI se potom primenjuje na odgovarajuću transformisanu funkciju. DEEP-FRI poboljšava granice pouzdanosti testa, uz očuvanje linearne aritmetičke složenosti dokazivača i logaritamske aritmetičke složenosti verifikatora [4].

7.10 Pretvaranje u neinteraktivni dokaz

Osnovni STARK protokol opisuje se kao interaktivni dokaz sa pristupom proročistu (engl. *Interactive Oracle Proof*, IOP), u kojem dokazivač šalje posvete, a verifikator

bira javne slučajne izazove. Za neinteraktivnu upotrebu primenjuje se Fiat–Šamirova heuristika [33].

Ako su prethodne posvete

$$c_1, c_2, \dots, c_i, \quad (7.27)$$

sledeći izazov može se dobiti kao

$$\alpha_i = H(x, c_1, c_2, \dots, c_i), \quad (7.28)$$

gde je x javna tvrdnja. Verifikator kasnije ponavlja isto heširanje i proverava da li su svi izazovi pravilno izvedeni.

7.11 Pojednostavljen primer zk-STARK postupka

Posmatrajmo funkciju koja počinje tajnom vrednošću x_0 i tri puta primenjuje pravilo

$$x_{i+1} = x_i^2 + 1. \quad (7.29)$$

Dokazivač želi da dokaže da je nakon tri koraka dobijena javna vrednost y , ali ne želi da otkrije početnu vrednost x_0 .

AIR sadrži tranziciono ograničenje

$$x_{i+1} - x_i^2 - 1 = 0, \quad i \in \{0, 1, 2\}, \quad (7.30)$$

kao i završno ograničenje

$$x_3 - y = 0. \quad (7.31)$$

Dokazivač interpolira kolonu traga polinomom $f(X)$, računa evaluacije na proširenom domenu, posvećuje se tim evaluacijama Merklovim korenom, formira polinom kompozicije, izvršava FRI i koristi Fiat–Šamirovu heuristiku.

Verifikator za izabranu poziciju proverava relaciju

$$f(\omega^{i+1}) - f(\omega^i)^2 - 1 = 0. \quad (7.32)$$

Opisani primer prikazuje proveru računarskog integriteta i osnovni tok STARK protokola, ali sam po sebi ne obezbeđuje nulto znanje. Da bi sistem bio zk-STARK, potrebno je primeniti dodatne tehnike nasumičnog maskiranja polinoma i traga, tako da mali broj upita verifikatora ne otkriva poverljive vrednosti svedoka [2, 33].

7.12 Skalabilnost

Za trag izvršavanja dužine n , pri fiksnoj širini traga i odgovarajuće izabranim parametrima, dominantne operacije dokazivača, kao što su interpolacija, proširenje domena i izgradnja FRI slojeva, imaju kvazilinearnu složenost. Ona se u pojednostavljenom obliku često zapisuje kao

$$O(n \log n). \quad (7.33)$$

Ukupna složenost kompletnog dokazivača ipak zavisi i od memorijskog prostora programa, širine traga, stepena AIR ograničenja, faktora proširenja i drugih parametara protokola [2, 33].

Verifikator proverava mali broj nasumično odabranih pozicija i Merklvih autentifikacionih putanja, pa njegova složenost raste polilogaritamski u odnosu na dužinu izvršavanja. U pojednostavljenom obliku može se zapisati kao

$$O(\log^k n), \quad (7.34)$$

za neku malu konstantu k , čija vrednost zavisi od konkretne konstrukcije i izbora parametara [2].

U eksperimentu sa DNK bazom originalni STARK rad razmatrao je bazu sa približno 1,14 miliona profila. Za najveći testirani slučaj vreme verifikacije bilo je približno šest puta manje od vremena naivne provere, dok je komunikacija bila približno sto puta manja od veličine svedoka [2]. Ove rezultate treba tumačiti u okviru konkretne implementacije, bezbednosnog nivoa i hardvera korišćenog u eksperimentu.

7.13 Primene zk-STARK sistema

Najpoznatije oblasti primene obuhvataju skaliranje blockchain mreža, proverljivo računanje nad poverljivim podacima i dokazivanje svojstava identiteta.

U sistemima za skaliranje blockchain mreža veliki broj tranzicija stanja može se izvršiti van osnovnog lanca, dok se na osnovni lanac šalju rezultat izvršavanja i kriptografski dokaz njegove ispravnosti. Time se smanjuje potreba da svaki učesnik ponovo izvršava čitavo računanje. Originalni STARK rad ovaj pravac navodi kao jednu od važnih mogućih primena skalabilne transparentne verifikacije [2].

Dokazi nultog znanja mogu se koristiti i za proveru tvrdnji nad medicinskim, finansijskim i identitetskim podacima, bez objavljivanja samih poverljivih vrednosti [20].

Ipak, zbog troška generisanja i relativno velike veličine dokaza, zk-STARK nije uvek pogodan za male ili resursno ograničene uređaje. Eksperimentalno poređenje u IoT okruženju pokazuje da veličina dokaza i vreme generisanja mogu predstavljati značajan praktičan problem [28]. Rezultati tog rada odnose se na konkretnu implementaciju i eksperimentalnu konfiguraciju, pa ih ne treba generalizovati na sve STARK sisteme.

7.14 Poređenje zk-SNARK i zk-STARK sistema

Osobina	zk-SNARK	zk-STARK
Poverljivo podešavanje	Često potrebno	Nije potrebno
Osnovne pretpostavke	Eliptičke krive i bilinearna uparivanja	Heš funkcije i kodovi za korekciju grešaka
Postkvantna sigurnost	Uglavnom ne	Verovatno postkvantno siguran
Veličina dokaza	Veoma mala	Veća
Vreme verifikacije	Veoma kratko	Kratko, ali često veće
Transparentnost	Zavisi od sistema	Osnovna osobina
Pogodnost za velike proračune	Dobra	Veoma dobra

Tabela 7.1: Kvalitativno poređenje tipičnih pairing zasnovanih zk-SNARK konstrukcija i zk-STARK sistema [2, 3, 28].

Konkretno performanse zavise od dokazivane relacije, implementacije, bezbednosnog nivoa i hardvera. Tabelu 7.1 zato treba tumačiti kao kvalitativni pregled tipičnih svojstava, a ne kao univerzalno pravilo za svaku pojedinačnu konstrukciju.

7.15 Aurora i drugi transparentni sistemi

Aurora je primer transparentnog zk-SNARK sistema namenjenog dokazivanju zadovoljivosti R1CS² instanci [3].

Glavna razlika između Aurore i STARK sistema jeste model izračunavanja. Aurora polazi od eksplicitno zadatog aritmetičkog kola ili R1CS sistema, dok STARK prirodnije opisuje uniformno izvršavanje programa kroz trag izvršavanja i AIR.

Za aritmetičko kolo sa N kapija, Aurora rad navodi vreme dokazivača

$$T_{\text{prover}} = O(N \log N), \quad (7.35)$$

vreme verifikatora

$$T_{\text{verifier}} = O(N), \quad (7.36)$$

i veličinu dokaza

$$O(\log^2 N). \quad (7.37)$$

Linearna složenost verifikatora ovde uključuje obradu eksplicitno zadate R1CS instance; samo čitanje instance sa N ograničenja već zahteva linearno vreme. Aurora i STARK zato ne treba posmatrati kao potpuno zamenljiva rešenja, već kao sisteme optimizovane za različite modele izračunavanja [3].

²R1CS (engl. *Rank-1 Constraint Satisfaction*) predstavlja sistem ograničenja oblika $(Az) \circ (Bz) = Cz$, koji se često koristi za predstavljanje aritmetičkih kola.

7.16 Prednosti i nedostaci zk-STARK sistema

U tabeli 7.2 prikazane su najvažnije prednosti i nedostaci zk-STARK sistema.

Prednosti	Nedostaci
Nije potrebna faza poverljivog podešavanja.	Dokazi su obično veći nego kod pairing zasnovanih zk-SNARK sistema.
Sistem je verovatno postkvantno siguran.	Dokazivač može imati visoke memorijske zahteve.
Verifikacija je skalabilna i zahteva proveru samo malog broja pozicija.	Aritmetizacija i projektovanje AIR ograničenja mogu biti složeni.
Dokazivač koristi efikasne FFT algoritme za interpolaciju i evaluaciju polinoma.	Fiksni trošak generisanja dokaza može biti visok za mala izračunavanja.
Dokaz može biti javno proverljiv nakon primene Fiat–Šamirove heuristike.	Neinteraktivna verzija zavisi od modela slučajnog proročišta.
Različiti programi mogu se predstaviti pomoću AIR ograničenja.	Izbor parametara, kao što su faktor proširenja i broj FRI upita, zahteva pažljivu bezbednosnu analizu.

Tabela 7.2: Prednosti i nedostaci zk-STARK sistema

7.17 Zaključak

zk-STARK sistemi predstavljaju važan korak u razvoju praktičnih dokaza nultog znanja. Oni omogućavaju dokazivanje ispravnosti velikih izračunavanja nad poverljivim podacima bez otkrivanja samih podataka i bez oslanjanja na fazu poverljivog podešavanja.

Osnovna ideja jeste prevođenje programskog izvršavanja u matematičku reprezentaciju. Izvršavanje se predstavlja tragom, a njegova korektnost AIR ograničenjima. Kolone traga interpoliraju se polinomima i evaluiraju na proširenom domenu. Merklava stabla omogućavaju dokazivaču da se obaveže na veliki broj vrednosti, dok FRI i DEEP-FRI omogućavaju efikasnu proveru niskog stepena.

Važno je razlikovati osnovnu STARK proveru računarskog integriteta od zk-STARK konstrukcije. Svojstvo nultog znanja zahteva dodatno maskiranje polinoma i drugih podataka koje verifikator otvara tokom protokola [2, 33].

U odnosu na zk-SNARK sisteme, zk-STARK ima veće dokaze i zahtevnijeg dokazivača. Međutim, transparentno podešavanje, skalabilna verifikacija i postkvantna orijentacija čine ga pogodnim za sisteme u kojima su dugoročna sigurnost, decentralizacija i javno poverenje od ključnog značaja.

Glava 8

Kriptografija zasnovana na rešetkama

8.1 Uvod

Bezbednost kriptosistema koji se danas koriste se zasniva na pretpostavci da su jednosmerne funkcije na kojima se zasnivaju teške, to jest da ne postoje algoritmi koji ih rešavaju na klasičnom računaru u polinomijalnom vremenu. Sistemi kao što su RSA i Difi-Helmanova razmena ključeva se zasnivaju na problemima faktORIZACIJE i diskretnog logaritma i za njih se smatra da su teški. Pretpostavka o težini se zasniva na višedecenijskim neuspelim pokušajima da se ovi problemi reše na klasičnom računaru.

Ako bi bio pronađen algoritam koji u polinomijalnom vremenu rešava probleme faktORIZACIJE ili diskretnog logaritma, sve informacije koje su do sada šifrovane bi postale kompromitovane. Pojava kvantnih računara ovu situaciju čini mogućom. Šor je pokazao da postoje efikasni algoritmi za faktORIZACIJU i problem diskretnog logaritma na kvantnim računarima [30]. Ovo znači da se sa realizacijom kvantnih računara bezbednost kriptosistema gubi.

Iako još uvek ne postoje kvantni računari, napor da se oni kreiraju su značajni i verovatno će se pojaviti u budućnosti. Zbog toga treba gledati unapred i razvijati kriptosisteme koji će se odupreti napadima od strane algoritama koji se izvršavaju na kvantnim računarima. Veruje se da su kriptografski sistemi zasnovani na heš funkcijama, kodu, rešetkama, jednačinama sa više promenljivih i kriptografija tajnog ključa kvantnorezistentni [5] ¹. Ovo poglavlje će se fokusirati na sisteme zasnovane

¹Strana 1, pasus 3

na rešetkama.

8.2 Kvantni računari

Trenutno postoje različiti predlozi za arhitekturu kvantnog računara koja će prevazići probleme kvantne mehanike i fizička ograničenja. Iz tog razloga se nećemo osvrnuti na konkretne aspekte pravljenja kvantnog računara već ćemo se pozabaviti konceptualnim prikazom elemenata jedne takve mašine.

Kvantni računar se razlikuje od klasičnog po tome što je u svakom trenutku u superpoziciji ² stanja koja se mogu predstaviti nizom bitova. Jedan bit kvantnog računara se naziva *kubit* (eng. *qubit*) i on opisuje superpoziciju stanja 0 i 1. Ovo se može opisati dvodimenzionim Hilbertovim prostorom: $\mathcal{H} = \mathcal{H}_1 = \mathbb{C} \oplus \mathbb{C} \cong \mathbb{C}^2$, čiju bazu čine vektori $(1, 0) = |0\rangle$ i $(0, 1) = |1\rangle$ [5] ³.

Kako je za opisivanje stanja kvantnog računara sa jednim bitom bio potreban dvodimenzioni prostor, tako će za opisivanje mašine sa n bitova biti potreban prostor dimenzije 2^n . Ovo se jednostavno postiže tenzorskim proizvodom Hilbertovih prostora: $\mathcal{H}_n = \mathcal{H} \otimes \dots \otimes \mathcal{H}$. Baza ovog prostora je tenzorski proizvod pojedinačnih baza, tj $|i_1 \dots i_n\rangle = |i_1\rangle \otimes \dots \otimes |i_n\rangle$, gde je $i_1 \dots i_n \in \mathcal{I}_n = \{0, 1\}^n$ [5].

Primer 8.2.1. Neka je $n = 3$, tada je baza prostora:

$$|000\rangle = |0\rangle \otimes |0\rangle \otimes |0\rangle$$

$$|001\rangle = |0\rangle \otimes |0\rangle \otimes |1\rangle$$

$$|010\rangle = |0\rangle \otimes |1\rangle \otimes |0\rangle$$

$$|011\rangle = |0\rangle \otimes |1\rangle \otimes |1\rangle$$

$$|100\rangle = |1\rangle \otimes |0\rangle \otimes |0\rangle$$

$$|101\rangle = |1\rangle \otimes |0\rangle \otimes |1\rangle$$

$$|110\rangle = |1\rangle \otimes |1\rangle \otimes |0\rangle$$

$$|111\rangle = |1\rangle \otimes |1\rangle \otimes |1\rangle.$$

²Superpozicija predstavlja linearnu kombinaciju baznih stanja sistema. Kada govorimo o sistemu sa jednim bitom, stanje bi se moglo opisati formulom $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, gde su $|0\rangle$ i $|1\rangle$ bazna stanja sistema.

³Strana 20, pasus 2

Kako bi se vršile transformacije stanja, koriste se unitarne transformacije nad Hilbertovim prostorom. One se aproksimiraju kvantnim logičkim kolima (eng. *gates*) iz fiksnog konačnog skupa $\mathcal{G}$. Tada je program na kvantnom računar u sekvenca kola koja vrše izračunavanja nad ulaznim stanjem i daju izlazno. Konkretni skup $\mathcal{G}$ zavisi od implementacije ali treba da bude u stanju da aproksimira transformacije kvantnih stanja uz neku malu grešku [5]. Da bi se više reklo o uzrocima grešaka prilikom menjanja kvantnih stanja mora se ući u prirodu kvantnih čestica, njihovu nestabilnost i lako padanje pod uticaj okolnih čestica, pa se na to nećemo osvrnuti. Treba imati na umu da se greške prilikom izmena stanja dešavaju iz fizičkih razloga.

Kada se izvrši program, kvantno stanje treba prevesti u konkretno stanje reprezentovano nizom bitova. Ovaj proces se zove očitavanje ili merenje (eng. *measuring*) i po prirodi je nedeterministički. Nedeterminizam, takođe, potiče od prirode kvantnih čestica jer se do trenutka očitavanja stanje nalazi u superpoziciji baznih stanja.

Ako je krajnje stanje kvantne mašine $v = \sum_{I \in I_n} \alpha_I |I\rangle$, očitavanje zavisi od raspodele verovatnoća za stanje s . Verovatnoća očitavanja stanja v je $P_v(I) = \frac{|\alpha_I|^2}{\sum_{J \in I_n} |\alpha_J|^2}$. [5] Nakon merenja, sistem kolapsira u izmereno stanje i ponovnim merenjem dobija se isti rezultat.

Na ovako definisanom modelu je Šor [30] definisao algoritam koji efikasno rešavaju probleme faktORIZACIJE i diskretnog logaritma. Kada se uspe sa kreiranjem kvantnog računara dovoljne jačine, kriptosistemi zasnovani na ovim funkcijama će postati ranjivi. Zbog toga je neophodno kreirati sisteme koji su kvantnerezistentni, kao što su sistemi zasnovani na rešetkama.

8.3 Rešetke

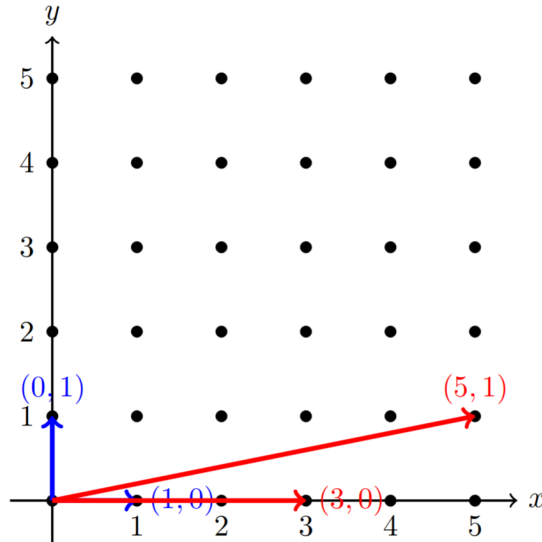
Ovo poglavlje posvećeno je rešetkama (engl. *lattices*), koje su se u kriptografiji prvobitno koristile za kriptanalizu postojećih sistema, dok danas predstavljaju osnovu za konstrukciju novih kriptografskih šema. Bezbednost postkvantnih kriptosistema se zasniva na pretpostavljenoj težini problema na rešetkama kao što su problem najkraćeg vektora i problem najbližeg vektora. Za ove probleme se smatra da ne postoje algoritmi koji ih rešavaju u polinomijalnom vremenu ni na klasičnom ni na kvantnom računaru.

Definicija 8.3.1. Rešetka $\mathcal{L}$ je skup celobrojnih linearnih kombinacija linearno nezavisnih vektora $\{b_1, \dots, b_m\}, b_i \in \mathbb{R}^n$. Drugim rečima, rešetka $\mathcal{L} = \{\sum_{i=1}^m a_i b_i \mid a_i \in \mathbb{Z}\} = \{B\mathbf{a} \mid \mathbf{a} \in \mathbb{Z}^n\}$, gde je $B = [b_1, \dots, b_m]$ bazna matrica rešetke, a vektori $\{b_1, \dots, b_m\}$ čine bazu rešetke.

Kako je rešetka $\mathcal{L}$ diskretan skup, može se definisati minimalna dužina nenula vektora $\lambda_1(\mathcal{L}) := \min\{\|x\| \mid x \in \mathcal{L}, x \neq 0\}$. Element za koji važi da mu je dužina jednaka λ_1 se naziva *najkraći nenula vektor rešetke* i nalaženje ovog elementa se generalno smatra teškim [31] ⁴.

Za rešetku $\mathcal{L}$ i njenu baznu matricu B možemo definisati *diskriminantu* $\Delta(\mathcal{L}) = |\det(B^T B)|^{1/2}$, to jest $\Delta(\mathcal{L}) = |\det(B)|$ ako je B kvadratna matrica. Vrednost diskriminante definiše zapreminu fundamentalnog paralelepipeda koji razapinju bazni vektori [31].

Jedna rešetka ima beskonačan broj baza, pa se može definisati šta je dobra, a šta loša baza za jednu rešetku. *Dobra baza* je ona čiji su vektori kratki i međusobno približno ortogonalni. *Loša baza* se sastoji od relativno dugih vektora između kojih je mali ugao. Na slici 8.1 je prikazana rešetka čija je dobra baza $\{(1, 0), (0, 1)\}$ jer su vektori međusobno ortogonalni i kratki, a loša $\{(3, 0), (5, 1)\}$ [22] ⁵.



Slika 8.1: Struktura rešetke, sa prikazom dobre (plava) i loše (crvena) baze.

Za proizvoljnu rešetku $\mathcal{L}$ čija je baza B se ne može uvek naći potpuno ortogonalna baza, ali postoji *LLL* (Lenstra, Lenstra i Lovász) *algoritam* koji daje redukovanu bazu. On u polinomijalnom vremenu nalazi redukovanu bazu čiji su vektori relativno kratki i približno međusobno ortogonalni, pri čemu je prvi vektor te baze približan najkraćem nenula vektoru rešetke $\mathcal{L}$, sa faktorom aproksimacije $2^{(n-1)/2}$, gde je n dimenzija kvadratne bazne matrice [31].

Poseban tip rešetki su q -arne (eng. *q-ary lattice*). Rešetka $\mathcal{L}$ je q -arna ako važi

⁴Strana 81, pasus 2

⁵Strana 7, pasus 3

$q\mathbb{Z}^n \subset \mathcal{L} \subset \mathbb{Z}^n$ za neko celobrojno q . Za proizvoljnu matricu $A \in \mathbb{Z}_q^{n \times m}$ mogu se definisati dve m -dimenzione q -arne rešetke:

$$\Lambda_q(A) = \{y \in \mathbb{Z}^m \mid y = A^T z \pmod{q}, z \in \mathbb{Z}^n\},$$

$$\Lambda_q^\perp(A) = \{y \in \mathbb{Z}^m \mid Ay = 0 \pmod{q}\}$$

.

Ovaj tip rešetki igra centralnu ulogu u konstrukciji savremenih kriptosistema zasnovanih na rešetkama i biće korišćene u nastavku rada.

8.3.1 Teški problemi na rešetkama

Postoje dva problema na rešetkama koja se smatraju teškim, to su problem najkraćeg vektora i problem najbližeg vektora. Pošto se pretpostavlja da je teško rešiti ove probleme i na klasičnom i na kvantnom računaru, postoje predlozi za postkvantne kriptosisteme koji se zasnivaju na njima. NIST (eng. *National Institute of Standards and Technology*) je 2024. godine objavio standarde za postkvantnu kriptografiju koji se zasnivaju na teškim problemima na rešetkama i dao preporuku da se započne njihova upotreba što pre [?].

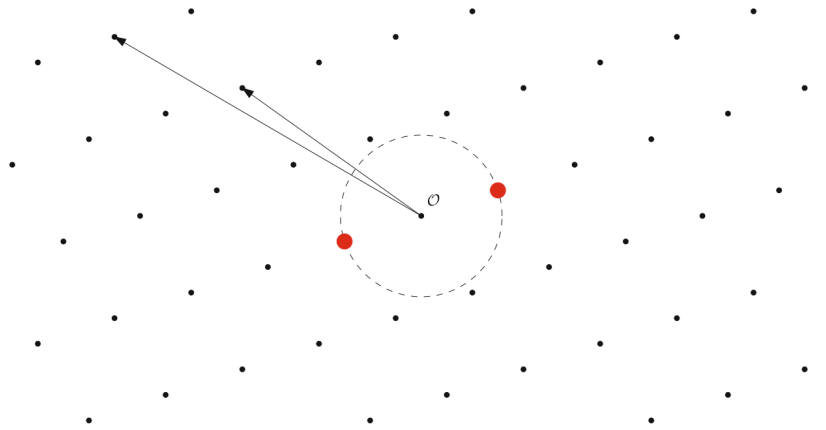
Definicija 8.3.2. Problem najkraćeg vektora (eng. *Shortest Vector Problem - SVP*): Neka je $\mathcal{L}$ rešetka čija je baza B , tada je problem najkraćeg vektora nalaženje nenula vektora $x \in \mathcal{L}$ za koji važi $\|x\| \leq \|y\|$ za svaki nenula vektor $y \in \mathcal{L}$. Drugim rečima, treba naći x takav da je $\|x\| = \lambda_1(\mathcal{L})$.

Postoje i alternativne definicije ovog problema, kao što je **aproksimativni problem najkraćeg vektora**, u oznaci SVP_γ . Ova verzija problema se odnosi na nalaženje vektora $x \in \mathcal{L}$ takvog da je $\|x\| \leq \gamma \cdot \lambda_1(\mathcal{L})$, za neku malu konstantnu γ . LLL algoritam nam omogućava da nađemo ovu vrstu najkraćeg vektora u rešetki, pri čemu je $\gamma = 2^{(n-1)/2}$ [31]⁶.

Rešenje problema najkraćeg vektora nije jedinstveno jer ako je vektor x njegovo rešenje, onda je i vektor $-x$. Na slici 8.2 se vidi primer dva najkraća vektora u dvodimenzionalnoj rešetki pri čemu su oni međusobno suprotni vektori.

Definicija 8.3.3. Problem najbližeg vektora (eng. *Closest Vector Problem - CVP*): Neka je $\mathcal{L}$ n -dimenziona rešetka sa bazom B i $x \in \mathbb{R}^n$ vektor koji ne pripada

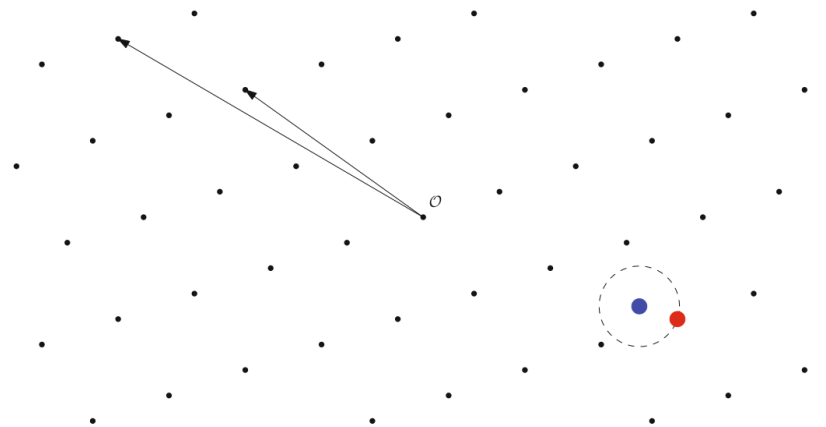
⁶Strana 86, pasus 1



Slika 8.2: Primer rešenja problema najkraćeg vektora u dvodimenzionalnoj rešetki. Crvenom bojom su označeni najkraći vektori, dok su crne strelice bazni vektori.

rešetki, problem najbližeg vektora se sastoji od nalaženja vektora $y \in \mathcal{L}$ takvog da je $\|x - y\| \leq \|x - z\|$ za svako $z \in \mathcal{L}$.

Kao i kod problema najkraćeg vektora, imamo i aproksimativni problem najbližeg vektora, koji se često označava sa CVP_γ . Cilj ovog problema je naći y takvo da je $\|x - y\| \leq \gamma \|x - z\|$ za svaki vektor $z \in \mathcal{L}$, za neku malu konstantnu vrednost γ . Na slici 8.3 se vidi primer jedne dvodimenzionalne rešetke, sa njenim baznim vektorima i vektorom koji je najbliži vektoru koji nije iz rešetke.



Slika 8.3: Primer rešenja problema najbližeg vektora u dvodimenzionalnoj rešetki. Crvenom bojom je označen vektor koji pripada rešetki i najbliži je plavo obeleženom vektoru koji joj ne pripada, dok su crne strelice bazni vektori.

Postoji još jedan problem na rešetkama koji je značajan za oblasti učenja sa greškama i potpuno homomorfne enkripcije. Taj problem je *problem dekodiranja na ograničenoj udaljenosti* koji je sličan problemu najbližeg vektora ali sa dodatom

informacijom o granici za udaljenost između vektora koji pripada rešetki i onome koji ne pripada.

Definicija 8.3.4. Problem dekodiranja na ograničenoj udaljenosti (eng. *Bounded-Distance Decoding Problem - BDD*) Neka je B baza rešetke $\mathcal{L}$ u n -dimenzionalnom realnom prostoru i $x \in \mathbb{R}^n$ vektor koji ne pripada rešetki $\mathcal{L}$. BDD_λ problem se odnosi na pronalaženje vektora $y \in \mathcal{L}$ takvog da je $\|x - y\| \leq \lambda \cdot \lambda_1(\mathcal{L})$, za realan broj λ .

Problemi koji su pomenuti u ovom poglavlju se smatraju teškim za rešavanje i na klasičnom i na kvantnom računaru, to jest ne postoje algoritmi koji ih rešavaju u polinomijalnom vremenu [5]. Zbog ove osobine predstavljaju dobru osnovu za kreiranje kvantnerezistentnih kriptosistema.

8.4 NTRU

NTRU je asimetrični kriptografski sistem koji se zasniva na rešetkama i problemu najkraćeg vektora. Zbog pretpostavljene težine rešavanja ovog problema u višim dimenzijama na klasičnom i kvantnom računaru, ovaj kriptosistem je otporan na kvantne napade. Kao takav, NTRU je dobar kandidat za postkvantni kriptosistem.

NTRU je definisan nad prstenom $R = \mathbb{Z}[X]/(X^N - 1)$, što znači da se svi polinomi sa celobrojnim koeficijentima redukuju po modulu $X^N - 1$. Sa osobinama ovakvog prstena i operacijama nad njegovim elementima smo se upoznali na početku kursa. U nastavku teksta će se množenje polinoma f i g u prstenu R označavati $f \circledast g$.

8.4.1 Generisanje ključa

Kada Alisa želi da pošalje poruku Bobanu, on prvo mora da generiše parametre sistema. On bira celobrojne vrednosti N, p i q takve da je $p < q$ i da važi $\text{gcd}(p, q) = 1$. Nakon ovoga, Boban nasumično generiše dva polinoma f i g stepena manjih od N , čiji su koeficijenti mali, najčešće između -1 i 1 . Polinom f mora biti invertibilan po modulu p i po modulu q , to jest moraju postojati f_p^{-1} i f_q^{-1} , takvi da važi $f_p^{-1} * f \equiv 1 \pmod{p}$ i $f_q^{-1} * f \equiv 1 \pmod{q}$ [22]⁷.

Nakon generisanja potrebnih parametara i polinoma, Boban računa $h \equiv f_q^{-1} \circledast g \pmod{q}$ i objavljuje svoj *javni ključ* (N, p, q, h) , a svoj *tajni ključ* (f) čuva privatno [22].

⁷Strana 9, pasus 4

8.4.2 Šifrovanje

Alisa prvo kodira svoju poruku u polinom m stepena manjeg on N čiji su koeficijenti po modulu p . Koeficijenti u ovom slučaju mogu biti celobrojne vrednosti između $-\frac{p}{2}$ i $\frac{p}{2}$.

Sledeći korak u procesu šifrovanja je nasumični odabir malog polinoma r i računanje šifrata

$$c \equiv pr \circledast h + m \pmod{q}$$

koji šalje Bobanu.

8.4.3 Dešifrovanje

Da bi dešifrovao šifrat c , Boban računa

$$\begin{aligned} a &\equiv f \circledast c \pmod{q} \equiv f \circledast (pr \circledast h + m) \pmod{q} \\ &\equiv pr \circledast f \circledast f_q^{-1} \circledast g + f \circledast m \pmod{q} \\ &\equiv pr \circledast g + f \circledast m \pmod{q} \end{aligned}$$

čiji su koeficijenti iz skupa $(-\frac{q}{2}, \frac{q}{2}]$.

Zatim, da bi otkrio originalnu poruku, Boban računa

$$\begin{aligned} m &\equiv f_p^{-1} \circledast a \pmod{p} \equiv f_p^{-1} \circledast (pr \circledast g + f \circledast m) \pmod{p} \\ &\equiv f_p^{-1} \circledast (pr \circledast g) + f_p^{-1} \circledast (f \circledast m) \pmod{p} \\ &\equiv p * (f_p^{-1} \circledast r \circledast g) + 1 * m \pmod{p} \\ &\equiv m \pmod{p} \end{aligned}$$

8.4.4 Interpretacija pomoću rešetaka

Iako se u prethodnom opisu rešetke ne pojavljuju eksplicitno, NTRU se može prirodno interpretirati u terminima rešetaka. U nastavku ovog poglavlja će se definisati termini i strukture koje NTRU na rešetkama koristi [22]⁸.

Neka je L linearna transformacija koja ciklično pomera elemente vektora. Tada definišemo matricu $L * v = [v, Lv, L^2v, \dots, L^{N-1}v]$ koja sadrži cirkularne pomeraje

⁸Strana 11, pasus 5

vektora v . NTRU takođe koristi konvolucione modularne rešetke $\Lambda(c, q) = \{(a, b) \in \mathbb{Z}^{2N} \mid a \circledast c \equiv b \pmod{q}\}$ [22]⁹.

Generisanje ključa

Pre nego što Alisa pošalje poruku Bobanu, on bira parametre sistema i računa tajni i javni ključ. Parametri sistema su prost broj N , mali ceo broj p i ceo broj q .

Privatni ključ je kratki vektor $(f, g) = (f_0, \dots, f_{N-1}, g_0, \dots, g_{N-1}) \in \mathbb{Z}^{2N}$. Elementi f i g se biraju tako da:

1. $[L * f]$ bude invertibilno po modulu q
2. $f \in e_1 + \{-p, 0, p\}^N$, gde je e_1 prvi standardni bazni vektor rešetke i $g \in \{-p, 0, p\}^N$, gde $f - e_1$ i g imaju tačno $d_f + 1$ pozitivnih elemenata i d_f negativnih. Posledica ovih ograničenja je da je $[L * f] \equiv I \pmod{p}$ i $[L * g] \equiv 0 \pmod{p}$

Javni ključ je Hermitova normalna forma baze rešetke $\Lambda((L * f, L * g)^T, q)$, za koju se smatra da je loša baza rešetke. Dakle, javni ključ je

$$H = \begin{bmatrix} I & 0 \\ L * h & q \cdot I \end{bmatrix}$$

gde je $h \equiv [L * f]^{-1} \pmod{q}$

Šifrovanje

Alisa poruku kodira vektorom $m \in \{-1, 0, 1\}^N$ koji ima tačno $d_f + 1$ pozitivnih elemenata i d_f negativnih. Prvo se nasumično generiše vektor $r \in \{-1, 0, 1\}^N$ koji ima tačno $d_f + 1$ pozitivnih elemenata i d_f negativnih. Ovaj vektor se konkatenuira na poruku m i tako dobijeni vektor se redukuje po modulu H :

$$\begin{bmatrix} r \\ m \end{bmatrix} \bmod \begin{bmatrix} I & 0 \\ L * h & qI \end{bmatrix} = \begin{bmatrix} 0 \\ (m + (L * h)r) \bmod q \end{bmatrix}$$

Dakle šifrat je $c \equiv (m + [L * h]r) \pmod{q}$

⁹Vektori se tumače kao polinomi stepena manjeg od N i to tako da vektor $a = (a_0, \dots, a_{N-1})$ generiše polinom $a(x) = a_0 + a_1x + \dots + a_{N-1}x^{N-1}$

Dešifrovanje

Dešifrovanje šifrata c započinje računanjem izraza:

$$\begin{aligned} [L * f]c &\equiv [L * f](m + [L * h]r) \\ &\equiv [L * f]m + [L * f][L * h]r \\ &\equiv [L * f]m + [L * g]r \pmod{q} \end{aligned}$$

Dobijeni vektor ima elemente koji su između $-\frac{q}{2}$ i $\frac{q}{2}$, pa je dovoljno da se redukuje po modulu p kako bi se dobila originalna poruka:

$$[L * f]m + [L * g]r \equiv I * m + 0 * r \equiv m \pmod{p}$$

8.5 Zaključak

Savremeni kriptografski sistemi svoju bezbednost zasnivaju na problemima za koje se smatra da su teški za rešavanje na klasičnim računarima. Međutim, razvoj kvantnog računarstva je doveo u pitanje tu bezbednost. Kvantni algoritmi koje je Šor pokazao omogućavaju efikasno rešavanje pojedinih problema na kojima se zasnivaju najrasprostranjeniji asimetrični kriptosistemi.

Jedan od najznačajnijih pravaca razvoja postkvantne kriptografije predstavljaju kriptosistemi zasnovani na rešetkama. Njihova bezbednost se oslanja na probleme kao što su problem najkraćeg vektora, problem najbližeg vektora i problem dekodiranja na ograničenoj udaljenosti, za koje trenutno nisu poznati efikasni algoritmi ni na klasičnim ni na kvantnim računarima.

U ovom radu su predstavljeni osnovni pojmovi vezani za rešetke, kao i najvažniji problemi na njima. Takođe je analiziran NTRU, jedan od najpoznatijih kriptosistema zasnovanih na rešetkama, kroz opis generisanja ključeva, procesa šifrovanja i dešifrovanja, kao i kroz njegovu interpretaciju pomoću rešetki. Ovakvi sistemi predstavljaju značajan korak ka obezbeđivanju dugoročne bezbednosti komunikacija u eri kvantnog računarstva.

Iako je oblast postkvantne kriptografije i dalje predmet intenzivnih istraživanja, dosadašnji rezultati ukazuju da kriptografski sistemi zasnovani na rešetkama predstavljaju jedno od najperspektivnijih rešenja za zaštitu podataka u budućnosti.

8.6 Pitanja

1. Na čemu se zasniva bezbednost kriptografskih sistema koji se danas koriste i da li je ona ugrožena? Ako jeste ugrožena, navesti zašto.
2. Kako se definišu rešetke? Koji problemi se smatraju teški na rešetkama?
3. Šta je tajni, a šta javni ključ u osnovnom obliku NTRU kriptosistema?

Glava 9

Učenje sa greškama

9.1 Uvod

U prethodnom poglavlju smo videli zašto su nam potrebni kriptosistemi otporni na napade kvantnih računara, i da su rešetke jedna od osnova za konstrukciju tih kriptosistema. Saznali smo i koji problemi na rešetkama su najrelevantniji, ali nam ostaje da vidimo kako se oni koriste za konstrukcije kriptosistema koji su proglašeni za najnoviji standard u svetu kriptografije. Takođe, dolazi i do potrebe da formalno opravdamo težinu problema nove generacije. Matematički okvir kojim ćemo se služiti da opišemo težinu ovih problema je teorija složenosti.

Centralnu ulogu u teoriji složenosti ima klasa NP , kojoj pripadaju problemi čije se potencijalno rešenje može verifikovati (ne i pronaći!) u polinomijalnom vremenu. Najtežim problemima u ovoj klasi se smatraju *NP-kompletni* problemi, jer bi pronalazjenje polinomijalnog algoritma za rešavanje bilo kog od njih dokazalo jednakost klasa P i NP . [13]¹

Moglo bi se očekivati da bi onda najteži kriptosistemi bili upravo oni zasnovani na NP -kompletnim problemima. Međutim, više pokušaja konstrukcije NP -kompletnih sistema koji su se pojavili u praksi pokazalo se nebezbednim, nađeni su algoritmi koji ih razbijaju. Na primer, Merkle-Helman kriptosistem zasnovan na NP -kompletnom problemu ranca, može se razbiti tehnikama zasnovanim na LLL algoritmu, o kom smo pričali u prethodnom poglavlju [19]².

Problem leži u tome što NP -kompletnost garantuje težinu samo za najgori slučaj datih problema. U kriptografiji se najčešće koriste instance koje nastaju prema nekoj

¹Lep uvod u teoriju složenosti može se pronaći u knjizi Goldreich-a

²Više informacija o ovom problemu i njegovom razbijanju može se pronaći u radu Lagarias-a i Odlyzka.

raspodeli, odnosno prosečni slučajevi. Zato je moguće da je opšti problem težak, a da su specijalne instance koje se pojavljuju u konkretnim kriptosistemima mnogo lakše za rešavanje.

Način da se formalno osiguramo da je naša konstrukcija teška i da nas uspešno brani od napada upravo u prosečnom slučaju jeste *redukcija iz najgoreg u prosečan slučaj* (engl. *worst-case to average-case reduction*). Ideja je da se rešavanje prosečne instance jednog problema poveže sa rešavanjem najgore instance nekog problema za koji verujemo da je težak. Ako takva redukcija postoji, i ako se ona izvodi u polinomijalnom vremenu, tada prosečna težina prvog problema sledi iz težine najgoreg slučaja drugog. Ovakve redukcije postoje za probleme na rešetkama, pa su one posebno pogodne za postkvantnu kriptografiju.

Baš ova redukcija je u osnovi konstrukcije koju ćemo razmatrati u nastavku: problem *učenja sa greškama* (engl. *Learning With Errors*, LWE), čija je prosečna težina povezana sa najgorim slučajem teškog problema na rešetkama. U nastavku će najpre biti definisan problem *učenja sa greškama*, zatim njegova algebarska varijanta *učenje sa greškama u prstenu*, a na kraju i kriptografska šema *Kyber*, koja se zasniva na modulnoj varijanti problema učenja sa greškama.

9.2 Učenje sa greškama

Problem *učenja sa greškama* uveo je Oded Regev u radu *On Lattices, Learning with Errors, Random Linear Codes, and Cryptography* [27], za koji je 2018. godine dobio Gödelovu nagradu.

Pre formalne definicije korisno je prvo objasniti šta u ovom kontekstu znači „učenje“. Pretpostavimo da postoji neka skrivena linearna veza koju želimo da otkrijemo.

Primer 9.2.1. Neka su nepoznate vrednosti s_1 i s_2 , a poznate su jednačine

$$2s_1 + s_2 = 7,$$

$$s_1 + 3s_2 = 11,$$

$$3s_1 + 2s_2 = 12.$$

Ovaj sistem je lako rešiti i dobija se $s_1 = 2, s_2 = 3$.

Dakle, iz tačnih linearnih jednačina možemo direktno da *naučimo*, odnosno odredimo, skrivene vrednosti s_1 i s_2 . Kod problema učenja sa greškama, situacija je slična, ali jednačine nisu potpuno tačne. Svakoju jednačini dodata je mala greška.

Primer 9.2.2. Umesto prethodnog sistema mogli bismo imati

$$\begin{aligned} 2s_1 + s_2 &= 7+1, \\ s_1 + 3s_2 &= 11-1, \\ 3s_1 + 2s_2 &= 12+2. \end{aligned}$$

Ove jednačine više ne odgovaraju tačno istoj linearnoj vezi, već su samo blizu nje.

Zadatak je da se, iz takvih približno tačnih jednačina, rekonstruišu skrivene vrednosti. Upravo dodavanje male greške, koje na prvi pogled deluje bezazleno, čini problem znatno težim.

Da bismo ovo matematički opisali, potrebno je uvesti raspodelu greške.

Definicija 9.2.3. Diskretna Gausova raspodela greške nad $\mathbb{Z}^n$, sa oznakom $D_{\mathbb{Z}^n, \sigma}$, predstavlja raspodelu greške nad celobrojnim vektorima dimenzije n . Raspodela proizvodi vektore čije su koordinate približno raspodeljene kao normalna raspodela sa srednjom vrednošću nula i standardnom devijacijom σ .

Primer 9.2.4. Ako je $n = 8$ i ako je σ malo, jedan mogući izlaz raspodele $D_{\mathbb{Z}^n, \sigma}$ mogao bi biti vektor $e = (0, -1, 2, 0, -3, 1, 0, 2)$.

Definicija 9.2.5. Pretraživački problem učenja sa greškama (engl. *Search-LWE*). Neka je q prirodan, najčešće prost broj, i neka je $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$ skup ostataka po modulu q . Posmatrajmo matricu $A \in \mathbb{Z}_q^{n \times m}$, tajni vektor $s \in \mathbb{Z}_q^m$, i vektor greške $e \leftarrow D_{\mathbb{Z}^n, \sigma}$. Pomoću njih računamo

$$b = A \cdot s + e \pmod{q}$$

Matrica A i vektor b su poznati, dok su tajni vektor s i greška e nepoznati. Dakle, pretraživački LWE je problem pronalaženja tajnog vektora $s \in \mathbb{Z}_q^m$ na osnovu para (A, b) .

Da greške nema, odnosno da je $e = 0$, problem bi se sveo na rešavanje sistema linearnih jednačina $A \cdot s = b \pmod{q}$, što smo videli u primeru 9.2.1. Međutim, umesto tačnog linearnog sistema, dodavanjem greške dobija se sistem približno tačnih linearnih jednačina, iz kojeg treba rekonstruisati tajni vektor.

Veza između problema učenja sa greškama i rešetki leži u posebnoj vrsti rešetki koje smo već pominjali, a to su q -arne rešetke. Za matricu A posmatrajmo rešetku

$$\mathcal{L} = \Lambda_q(A^T) = \{x \in \mathbb{Z}^n \mid x \equiv A \cdot z \pmod{q}, z \in \mathbb{Z}^m\}$$

Vektor $A \cdot s$ pripada ovoj rešetki, dok je poznati vektor b dobijen dodavanjem male greške: $b = A \cdot s + e \pmod{q}$.

Dakle, vektor b se nalazi blizu nekog vektora iz rešetke $\mathcal{L}$. Problem učenja sa greškama može se zato geometrijski posmatrati kao problem u kom treba pronaći rešetkasti vektor koji je blizak datom vektoru b . Možemo čak i proceniti koliko su bliska ova dva vektora, jer znamo da je e dobijeno iz naše raspodele greške $D_{\mathbb{Z}^n, \sigma}$. Sa veoma velikom verovatnoćom, procenjujemo normu vektora e kao $\|e\| \leq \frac{2 \cdot \sigma \cdot \sqrt{m}}{\sqrt{2 \cdot \pi}}$. Ovo je upravo ideja problema dekodiranja na ograničenoj udaljenosti (eng. *Bounded-Distance Decoding Problem* - *BDD*). Tačnije, problem učenja sa greškama se iz ove perspektive može posmatrati kao BDD_α problem, gde je $\alpha = \frac{2\sigma\sqrt{m}}{\sqrt{2\pi}\lambda_1(\mathcal{L})}$.

Ovo pokazuje intuiciju iza redukcije problema učenja sa greškama. Formalna redukcija iz najgoreg u prosečan slučaj je složenija [27], ali upravo ova veza sa problemima na rešetkama daje osnovu za tvrdnju da je *LWE* težak problem u prosečnom slučaju.

9.2.1 Protokol javnog ključa zasnovan na problemu učenja sa greškama

Na osnovu problema učenja sa greškama može se konstruisati jednostavan protokol javnog ključa. Ovaj protokol nije najefikasnija praktična šema, ali je koristan jer jasno pokazuje kako se ideja problema koristi u kriptografiji. Malu grešku ćemo koristiti za sakrivanje tajne vrednosti, dok će dešifrovanje i dalje biti moguće ako greška ne postane prevelika.

Neka su dati parametri n , m i q , gde je q neparan prost broj. Poruka koju šifrujemo biće jedan bit, pa ćemo je označiti sa $M \in \{0, 1\}$.

Generisanje ključa.

Za tajni ključ bira se vektor $s \in \mathbb{Z}_q^n$.

Zatim se za $i = 1, \dots, m$ nasumično biraju vektori javnog ključa $a_i \in \mathbb{Z}_q^n$ i male greške $e_i \leftarrow D_{\mathbb{Z}, \sigma}$.

Za svaki indeks i računa se $b_i = a_i \cdot s - 2 \cdot e_i \pmod{q}$.

Objavljujemo javni ključ: $pk = ((a_1, b_1), \dots, (a_m, b_m))$,

Na primer, neka je $q = 17$, $s = (2, 3)$, $a_1 = (1, 2)$, $a_2 = (4, 1)$, $a_3 = (2, 4)$, i neka

su greške $e_1 = 1$, $e_2 = -1$, $e_3 = 2$. Tada dobijamo:

$$\begin{aligned} b_1 &= (1, 2) \cdot (2, 3) - 2 \cdot 1 = 8 - 2 = 6 \pmod{17}, \\ b_2 &= (4, 1) \cdot (2, 3) - 2 \cdot (-1) = 11 + 2 = 13 \pmod{17}, \\ b_3 &= (2, 4) \cdot (2, 3) - 2 \cdot 2 = 16 - 4 = 12 \pmod{17}. \end{aligned}$$

Enkripcija.

Da bi se šifrovala poruka $M \in \{0, 1\}$, pošiljalac prvo bira nasumične indikatorske vrednosti $\iota_i \in \{0, 1\}$, $i = 1, \dots, m$. Ove vrednosti određuju koji će se elementi javnog ključa koristiti pri formiranju šifrata.

Zatim se računa šifrat:

$$c = \sum_{i=1}^m \iota_i a_i \pmod{q} \quad \text{i} \quad d = M - \sum_{i=1}^m \iota_i b_i \pmod{q}.$$

Dekripcija.

Primalac, koji zna tajni ključ s , računa

$$\begin{aligned} c \cdot s + d \pmod{q} &= \left(\sum_{i=1}^m \iota_i a_i \right) \cdot s + M - \sum_{i=1}^m \iota_i b_i \pmod{q} \\ &= \sum_{i=1}^m \iota_i (a_i \cdot s) + M - \sum_{i=1}^m \iota_i (a_i \cdot s - 2e_i) \pmod{q} \\ &= M + 2 \sum_{i=1}^m \iota_i e_i \pmod{q}. \end{aligned}$$

Vrednost $\eta = 2 \sum_{i=1}^m \iota_i e_i$ naziva se *šum šifrata*. Pošto su greške e_i male, očekujemo da i njihov zbir ostane mali u odnosu na modul q . Ako je šum dovoljno mali, i nakon primene operacije $\pmod{q}$ imaćemo vrednost $M + \eta$.

Kako je η paran broj, poruka se dobija redukcijom po modulu 2:

$$(M + \eta) \pmod{2} = M.$$

Zato dešifrovanje uspeva sve dok je šum dovoljno mali.

9.3 Učenje sa greškama u prstenu

U prethodnom delu videli smo kako se problem učenja sa greškama može iskoristiti za konstrukciju protokola javnog ključa. Međutim, protokol ima jedan praktičan problem: javni ključ i šifrat mogu biti veoma veliki. Zbog toga je memorijsko opterećenje relativno veliko.

Da bi se ono smanjilo, uvodi se varijanta problema zasnovana na prstenovima polinoma, koja se naziva *učenje sa greškama u prstenu*, ili *prstenasti LWE* (engl. *Ring-LWE*). Osnovna ideja je da se umesto matrica koriste polinomi iz prstenova polinoma. Na taj način se dobija kompaktniji zapis, a računanje postaje efikasnije.

Kod običnog LWE problema osnovna operacija je množenje matrice i vektora, $x \mapsto A \cdot x$. Kod Ring-LWE problema, umesto matrica koristimo polinome u odgovarajućem prstenu. Posmatrajmo prsten $R = \mathbb{Z}[X]/(F(X))$, gde je $F(X)$ celobrojni polinom stepena n . Elementi ovog prstena mogu se posmatrati kao polinomi stepena manjeg od n sa celobrojnim koeficijentima. Sabiranje se vrši kao obično sabiranje polinoma, dok se množenje vrši kao množenje polinoma, nakon čega se rezultat redukuje po modulu polinoma $F(X)$.

Pošto u kriptografskim primenama radimo i po modulu q , uvodimo oznaku $R_q = (\mathbb{Z}/q\mathbb{Z})[X]/(F(X))$. Elementi prstena R_q su polinomi stepena manjeg od n , čiji se koeficijenti posmatraju po modulu q .

Veza između običnog LWE i Ring-LWE problema dolazi iz toga što se polinom može posmatrati i kao vektor njegovih koeficijenata, ali i kao matrica. Na primer, polinom

$$a(X) = a_0 + a_1X + \cdots + a_{n-1}X^{n-1} \quad \text{zapisujemo kao} \quad a = (a_0, a_1, \dots, a_{n-1}).$$

Slično tome, množenje nekim fiksiranim polinomom $a(X)$ može se predstaviti kao množenje posebnom matricom. Ako je

$$c(X) = a(X) \cdot b(X) \pmod{F(X)},$$

onda se koeficijenti polinoma $c(X)$ mogu dobiti kao

$$c = M_a \cdot b,$$

gde je M_a matrica koja odgovara množenju polinomom $a(X)$.

Kao i kod običnog LWE problema, potrebno je uvesti raspodelu greške.

Definicija 9.3.1. Diskretna Gausova raspodela greške nad R , sa oznakom $D_{R,\sigma}$, predstavlja raspodelu koja daje polinome iz R čiji su koeficijenti mali, odnosno približno raspodeljeni kao normalna raspodela sa srednjom vrednošću nula i standardnom devijacijom σ .

Drugim rečima, ako je $e(X) = e_0 + e_1X + \dots + e_{n-1}X^{n-1}$, onda su koeficijenti e_i sa velikom verovatnoćom mali.

Definicija 9.3.2. Pretraživački problem učenja sa greškama u prstenu (engl. *Search Ring-LWE*) definiše se na sledeći način. Neka su $a, s \in R_q$, neka je $e \leftarrow D_{R,\sigma}$ i neka je

$$b = a \cdot s + e \pmod{q}.$$

Za dati par (a, b) , cilj je pronaći tajni polinom s .

Ova definicija veoma liči na običan LWE problem. Razlika je u tome što umesto matrice A , vektora s i vektora greške e , sada radimo sa polinomima a , s i e u prstenu R_q . Dakle, izraz

$$b = A \cdot s + e \pmod{q}$$

iz običnog LWE problema zamenjen je izrazom

$$b = a \cdot s + e \pmod{q}$$

u prstenu polinoma.

Prednost Ring-LWE problema je u tome što se pomoću jednog polinoma može predstaviti struktura koja bi kod običnog LWE zahtevala celu matricu. Time se smanjuje veličina javnog ključa i šifrata, a množenje polinoma može se računati efikasnije od opšteg množenja matrice i vektora. Zbog toga Ring-LWE predstavlja važan korak od teorijske LWE konstrukcije ka praktičnijim kriptografskim protokolima.

9.3.1 Protokol javnog ključa zasnovan na učenju sa greškama u prstenu

Protokol javnog ključa koji smo opisali kod običnog učenja sa greškama možemo preneti i u Ring-LWE okruženje. Ali nije nam cilj samo da prepíšemo istu konstrukciju sa polinomima, već i da smanjimo veličinu javnog ključa. Kod protokola javni ključ je sadržao veliki broj vrednosti koje se mogu posmatrati kao šifrovanja nule. Kod Ring-LWE protokola želimo da takva šifrovanja nule pravimo po potrebi, tokom enkripcije, umesto da ih sve unapred čuvamo u javnom ključu.

Neka su p i q uzajamno prosti brojevi, pri čemu je $p \ll q$. Takođe, neka je zadat prsten R , odgovarajući prsten R_q , kao i raspodela greške $D_{R,\sigma}$. Parametri p , q , R i σ predstavljaju javne parametre sistema. Poruke koje šifrujemo pripadaju prstenu R_p .

Generisanje ključa.

Za tajni ključ biramo male polinome $s, e \leftarrow D_{R,\sigma}$.

Zatim biramo i nasumičan element $a \leftarrow R_q$.

Računamo $b = a \cdot s + p \cdot e \pmod{q}$.

Javni ključ: $pk = (a, b)$, tajni ključ: $sk = s$.

Javni ključ se može posmatrati kao nasumično šifrovanje nule.

Enkripcija.

Neka je poruka $M \in R_p$. Da bismo šifrovali poruku, radimo sledeće:

$$\begin{aligned} e_0, e_1, e_2 &\leftarrow D_{R,\sigma} \\ c_0 &= b \cdot e_0 + p \cdot e_1 + M \pmod{q} \\ c_1 &= a \cdot e_0 + p \cdot e_2 \pmod{q} \end{aligned}$$

Šifrat je par (c_0, c_1) .

Ovde polinom e_0 služi za nasumičnost, dok polinomi e_1 i e_2 dodaju mali šum. Na taj način se tokom enkripcije pravi novo nasumično šifrovanje nule, koje se zatim dodaje poruci M .

Dekripcija.

Za dekrpciju koristimo tajni ključ s i računamo

$$\begin{aligned} c_0 - c_1 \cdot s &= (b \cdot e_0 + p \cdot e_1 + M) - (a \cdot e_0 + p \cdot e_2) \cdot s \pmod{q} \\ &= ((a \cdot s + p \cdot e) \cdot e_0 + p \cdot e_1 + M) - (a \cdot e_0 \cdot s + p \cdot e_2 \cdot s) \pmod{q} \\ &= p \cdot e \cdot e_0 + p \cdot e_1 + M - p \cdot e_2 \cdot s \pmod{q} \\ &= M + p \cdot (e \cdot e_0 + e_1 - e_2 \cdot s) \pmod{q}. \end{aligned}$$

Označimo šum šifrata sa

$$\eta = e \cdot e_0 + e_1 - e_2 \cdot s.$$

Tada je rezultat dekrpcije oblika

$$M + p \cdot \eta \pmod{q}.$$

Pošto su polinomi s , e , e_0 , e_1 i e_2 birani iz raspodele greške, njihovi koeficijenti su

mali, pa je i šum η mali. Ako je šum dovoljno mali, ne dolazi do preskakanja preko modula q , pa izraz možemo posmatrati kao

$$M + p \cdot \eta.$$

Na kraju redukujemo rezultat po modulu p :

$$M + p \cdot \eta \equiv M \pmod{p}.$$

Dakle, dobija se originalna poruka M .

Ovaj protokol pokazuje dve glavne prednosti učenja sa greškama u prstenu. Prvo, poruke više nisu samo bitovi iz skupa $\{0, 1\}$, već mogu biti elementi prstena R_p . Drugo, javni ključ je znatno manji nego kod osnovne šeme učenja sa greškama, jer se sastoji samo od dva elementa prstena, a i b .

Takođe, šema se može pokazati i kao IND-CPA (Indistinguishability under chosen-plaintext attack) bezbedna, što znači da je otporna na napade sa izabranim otvorenim tekstom.

Sa druge strane, učenje sa greškama u ovom osnovnom obliku *nije* IND-CCA (Indistinguishability under chosen-ciphertext attack) bezbedno, jer je moguće iz postojećeg šifrata c^* konstruisati novi validan šifrat c , čija je dešifrovana poruka m u određenoj vezi sa originalnom porukom m^* . Iako je ovo slabost iz ugla otpornosti na napade, od posebnog je značaja za potpuno homomorfno šifrovanje.

9.4 Kyber

NSA je 2022. godine objavila CNSA 2.0, skup preporuka za prelazak sistema nacionalne bezbednosti na kvantno-otporne algoritme. U tom skupu, Kyber je naveden kao osnovni algoritam za uspostavljanje ključa. Time Kyber postaje jedna od najvažnijih praktičnih primena ideje učenja sa greškama, a njegova standardizovana verzija je 2024. godine objavljena kao ML-KEM u standardu FIPS 203.

Pre nego što opišemo samu šemu Kyber, potrebno je ukratko objasniti problem na kome se ona zasniva, a to je modulno učenje sa greškama, poznato kao *Module-LWE*.

9.4.1 Modulno učenje sa greškama

Module Learning With Errors, odnosno Module-LWE se može posmatrati kao uopštenje običnog LWE i Ring-LWE pristupa. Prostor $\mathbb{Z}_q^n$ zamenjuje modulom R_q^k , gde je R_q^k k -struki Dekartov proizvod prstena polinoma R_q [24]. Zato tajni ključ s u Module-LWE problemu nije običan vektor nad $\mathbb{Z}_q$, već element modula R_q^k , odnosno vektor čije su komponente polinomi.

Neka je R_q prsten polinoma po modulu q , kao u Ring-LWE delu. Umesto jednog elementa $a \in R_q$, kod Module-LWE problema posmatramo matricu $A \in R_q^{k \times k}$ i vektore $s, e \in R_q^k$. Osnovni izraz tada ima oblik

$$b = A \cdot s + e \pmod{q}.$$

Prednost ovakvog pristupa je u tome što se zadržava veliki deo efikasnosti koju daje rad sa prstenom polinoma, ali se dobija fleksibilnija struktura nego kod čistog Ring-LWE problema. Parametar k određuje dimenziju modula, pa se izborom tog parametra mogu dobiti različiti nivoi bezbednosti bez promene osnovnog prstena.

9.4.2 Konstrukcija ML-KEM šeme

Konstrukcija ML-KEM šeme može se posmatrati u dva koraka. Prvo se, na osnovu Module-LWE problema, konstruiše šema javnog ključa (engl. *Public-Key Encryption*, PKE). Ova šema koristi istu osnovnu ideju koju smo već videli kod LWE i Ring-LWE konstrukcija.

Drugi korak je pretvaranje te PKE šeme u mehanizam enkapsulacije ključa (engl. *Key Encapsulation Mechanism*, KEM). To se postiže pomoću Fujisaki–Okamoto transformacije, koja se često koristi za dobijanje jačeg bezbednosnog modela iz osnovne šeme javnog ključa.

Zbog osobina Fujisaki–Okamoto transformacije, očekuje se da dobijeni ML-KEM zadovoljava IND-CCA2 bezbednost. [24]

9.4.3 Praktična upotreba Kyber-a

Važnost Kyber-a nije samo teorijska. I pre završne standardizacije, Kyber je počeo da se integriše u kriptografske biblioteke i realne sisteme. Na zvaničnoj stranici projekta Kyber navodi se nekoliko primera takve upotrebe: Cloudflare je integrisao Kyber, zajedno sa drugim postkvantnim algoritmima u biblioteku CIRCL; Amazon

je u okviru AWS Key Management Service podržao hibridne režime koji uključuju Kyber; a IBM je još 2019. godine predstavio prototip kvantno-bezbednog uređaja za skladištenje podataka koji koristi Kyber.

9.5 Pitanja

1. Kako izgleda protokol javnog ključa zasnovan na problemu učenja sa greškama?
2. Objasniti geometrijsku intuiciju veze između problema dekodiranja na ograničenoj udaljenosti (BDD) i problema učenja sa greškama (LWE).
3. Koje nedostatke osnovne šeme zasnovane na učenju sa greškama (LWE) ublažava njena varijanta učenja sa greškama u prstenu (Ring-LWE)?

Glava 10

Potpuno homomorfna enkripcija

10.1 Uvod

Potpuno homomorfna enkripcija omogućava delegiranje vršenja operacija nad podacima, bez kompromitovanja privatnosti samih podataka [12].

Klijent može na bezbedan način delegirati izračunavanje proizvoljne funkcije nad privatnim podacima serveru kome ne veruje. Podatke šalje enkriptovane, server homomorfno vrši operacije nad enkriptovanim podacima, i šalje rezultat klijentu, koji klijent treba samo da dešifruje.

Navikli smo da dobrim enkripcionim šemama smatramo one u kojima šifrat liči na slučajan šum, ali sada nam je poželjno da enkriptovani podaci čuvaju određene strukture otvorenog teksta - one strukture potrebne za vršenje aritmetičkih operacija.

10.2 Istorija

Ideju homomorfne enkripcije kao nešto što bi bilo dobro da postoji su predložili Rivest, Adleman i Dertouzos još 1978 [21]¹. Tek preko 30 godina kasnije, 2009. godine, Gentry je u svojoj doktorskoj disertaciji predložio prvu praktično moguću šemu potpuno homomorfne enkripcije. Nakon toga je koncipirano i implementirano još puno šema, i računski efikasnijih i lakših za razumevanje.

¹Strana 1, pasus 1

10.3 Multiplikativno ili aditivno homomorfna enkripcija

Enkripcione šeme koje omogućavaju vršenje samo množenja ili samo sabiranja nad šifrovanim tekstom su nam već odavno sa ovog kursa poznate - RSA i ElGamal! Obe ove enkripcione šeme su multiplikativno homomorfne. Postoje i aditivno homomorfne, jedan primer je Paillierov kriptosistem, ali njega nismo radili na ovom kursu.

U RSA, šifrat poruke m je $c = m^e \pmod{N}$. Recimo da su poruke m_1 i m_2 šifrovane sa $c_1 = m_1^e \pmod{N}$ i $c_2 = m_2^e \pmod{N}$. Množenjem ova dva šifrata dobijamo $m_1 \cdot m_2 = m_1^e \cdot m_2^e = (m_1 \cdot m_2)^e \pmod{N}$. Ovo jeste šifrat poruke $m_1 \cdot m_2$.

Slično, u ElGamal enkripciji poruka m je šifrovana sa $c = (g^r, m \cdot h^r)$. Želimo da pomnožimo poruke m_1 i m_2 čiji su šifrat $c_1 = (g^{r_1}, m_1 \cdot h^{r_1})$ i $c_2 = (g^{r_2}, m_2 \cdot h^{r_2})$. Operacija množenja otvorenog teksta $\cdot$ se homomorfno slika u operaciju pookoordinatnog množenja šifrata. Tako $c_1 \odot c_2 = (g^{r_1} \cdot g^{r_2}, m_1 \cdot h^{r_1} \cdot m_2 \cdot h^{r_2}) = (g^{r_1+r_2}, m_1 \cdot m_2 \cdot h^{r_1+r_2})$. Upravo se dobije ElGamal enkripcija poruke $m_1 \cdot m_2$ sa slučajnim parametrom $r_1 + r_2$.

Ipak, za potpuno homomorfnu enkripciju želimo da predstavimo bilo koju aritmetičku funkciju, a za to je dovoljno da imamo homomorfizme za sabiranje i za množenje.

10.4 Notacija i pojmovi

Notacija preuzeta iz [21]².

Definicija 10.4.1. Neka je $R(+, \cdot)$ prsten. Funkcija f je *aritmetička* ako se može zapisati kao kompozicija sabiranja i množenja.

Kao i do sad, enkripciona šema sa javnim ključem radi nad porukama $m \in R$, neka su E i D operacije za šifrovanje i dešifrovanje, neka je pk javni ključ, a sk tajni. Ukratko, imamo

$$D(E(m, pk), sk) = m$$

Za homomorfnu enkripciju kao novost imamo *evaluacioni algoritam*. Neka su $m_1, \dots, m_n \in R$, neka su im šifrat $c_i = E(m_i, pk)$, $i \in \{1, \dots, n\}$ i neka je $f : R^n \rightarrow R$. Evaluacioni algoritam Eval kao argumente prima *evaluacioni* ključ ek , funkciju f , šifrate $c_1, \dots, c_n$, a kao rezultat daje šifrat c_f , takav da se dešifrovanjem dobija

²Strana 6, sekcija Definitions and Basic Notions

$f(m_1, \dots, m_n)$. Drugim rečima, za Eval važi

$$c_f = \text{Eval}(f, (c_1, \dots, c_n), ek)$$

$$D(c_f, sk) = f(m_1, \dots, m_n)$$

Primetimo da se $c = E(f(m_1, \dots, m_n), pk)$ i c_f dešifruju u isti broj, ali su drugačiji u konstrukciji i u opštem slučaju $c \neq c_f$. Potpuno homomorfna enkripcija kakva je poznata danas koristi probabilističku enkripciju, koja se oslanja na slučajan šum. c i c_f imaju različite šumove.

Definicija 10.4.2. Enkripciona šema je *potpuno homomorfna* ako za proizvoljno n , proizvoljnu aritmetičku funkciju $f : R^n \rightarrow R$, proizvoljne poruke $m_1, \dots, m_n \in R$ i njihove šifrate $c_i = E(m_i, pk)$, $i \in \{1, \dots, n\}$ važi

$$D(\text{Eval}(f, (c_1, \dots, c_n), ek)) = f(m_1, \dots, m_n)$$

Potpuno homomorfnu enkripciju kraće nazivamo FHE (od njenog engleskog naziva *fully homomorphic encryption*).

Intuitivno, želimo da postoje operacije $\oplus$ i $\odot$ takve da za sve $m_1, m_2 \in R$ važi

$$D(E(m_1, pk) \oplus E(m_2, pk), sk) = m_1 + m_2$$

$$D(E(m_1, pk) \odot E(m_2, pk), sk) = m_1 \cdot m_2$$

Algoritam $\text{Eval}(f, (c_1, \dots, c_n), ek)$ vršimo tako što svaki $+$ iz f zamenimo za $\oplus$, i svako $\cdot$ za $\odot$.

Ukoliko enkripciona šema ima ovakav homomorfizam iz $(+, \cdot)$ u $(\oplus, \odot)$, ali algoritam D uspešno dešifruje samo ako je primenjeno dovoljno malo ovih operacija, u pitanju je *delimično homomorfna* enkripcija, ili SHE (eng. *somewhat homomorphic encryption*).

Definicija 10.4.3. Enkripciona šema je *delimično homomorfna* ako postoji ograničena klasa „dovoljno jednostavnih” aritmetičkih funkcija $\mathcal{F}$ takva da za proizvoljnu $f \in \mathcal{F}$ proizvoljnog reda n , proizvoljne poruke $m_1, \dots, m_n \in R$ i njihove šifrate $c_i = E(m_i, pk)$, $i \in \{1, \dots, n\}$ važi

$$D(\text{Eval}(f, (c_1, \dots, c_n), ek)) = f(m_1, \dots, m_n)$$

Sve homomorfne šeme enkripcije koje su u upotrebi se zasnivaju na probabilističkim

algoritmima, koji pri šifrovanju dodaju slučajan šum. Vršanjem aritmetičkih operacija šum raste, i nakon dovoljnog broja operacija, šifrat se više ne može dešifrovati. Upravo je zbog ovoga potrebno razmatrati klasu delimično homomorfnihi šema (odnosno SHE).

Prva potpuno homomorfna (FHE) šema je nastala kada je Gentry 2009. našao način da smanji akumulirani šum u SHE šemi.

Dakle, FHE se vrši ponavljanjem sledeća dva koraka:

1. Izvršiti dozvoljen broj operacija polaznom SHE šemom
2. Smanjiti šum

10.5 DGHV

Rad koji opisuje DGHV je izašao 2010. godine, godinu dana nakon Gentryjeve doktorske disertacije, a napisali su ga Dijk, Gentry, Halevi i Vaikuntanatha (otuda naziv šemi DGHV). Ova enkripciona šema je računski vrlo skupa i nije nigde u upotrebi, ali je razumljiva, i navodi se iz pedagoških razloga. O DGHV se može čitati u [21]³, ali blog [25] daje čitkiji opis. Pre uvođenja DGHV, posmatramo jedan uvodni primer.

10.5.1 Uvodni primer

Definišimo najpre jednu simetričnu enkripcionu šemu koja radi sa porukama $m \in \{0, 1\}$

Generisanje ključa: ključ je slučajan neparan broj p .

Enkripcija: Poruka $m \in \{0, 1\}$ se enkriptuje sa $c = m + 2 \cdot r + p \cdot q$, gde je r dovoljno mali šum, a q veliki slučajan ceo broj.

Dekripcija: Šifrat c se dekriptuje sa $(c \pmod{p}) \pmod{2}$. Uverimo se da ovo radi:

$$(c \pmod{p}) \pmod{2} = (m + 2 \cdot r + p \cdot q \pmod{p}) \pmod{2} \quad (10.1)$$

$$= (m + 2 \cdot r \pmod{p}) \pmod{2} \quad (10.2)$$

$$= m + 2 \cdot r \pmod{2} \quad (10.3)$$

$$= m \quad (10.4)$$

³Strana 9, sekcija First Generation: FHE based on the AGCD problem

Zbog (10.3) smo tražili da r bude dovoljno malo. Ako je $2 \cdot r < p$ (10.3) zaista sledi iz (10.2).

Dakle, enkripcija i dekripcija rade. Kako se ponašaju aritmetičke operacije? Neka je m_1 šifrovano sa $c_1 = m_1 + 2 \cdot r_1 + p \cdot q_1$ i m_2 sa $c_2 = m_2 + 2 \cdot r_2 + p \cdot q_2$.

Sabiranje: Lako se vidi da važi $c_1 + c_2 = (m_1 + m_2) + 2 \cdot (r_1 + r_2) + p \cdot (q_1 + q_2)$. Dakle, ako su r_1 i r_2 bili dovoljno mali na početku poruka se i dalje može dešifrovati, ali već sad se vidi da, koliko god mali bili, imamo samo konačno mnogo sabiranja pre nego što šum postane preveliki.

Množenje $c_1 \cdot c_2 = m_1 \cdot m_2 + 2 \cdot (r_2 \cdot m_1 + r_1 \cdot m_2 + 2 \cdot r_1 \cdot r_2) + p(q_1 m_2 + 2q_1 r_2 + 2q_1 q_2 + 2q_2 r_1 + q_2 m_1)$. Velika zagrada koja stoji pomnožena sa p može biti novo veliko slučajno q , tu nema problema. Šum je sada približno velik kao $r_1 \cdot r_2$ (podsetnik da je m je samo jedan bit).

Iako ova enkripcija jeste homomorfna, jasno je da je nedopustivo serveru otkriti tajni ključ p . Osnovni zahtev je sakrivanje podataka serveru. Uvodni primer naveden je jer DGHV funkcioniše jako slično, ali ima javni i tajni ključ, i javni ključ sakriva p od servera.

10.5.2 DGHV

Generisanje ključa Tajni ključ je slučajno izabran veliki neparan broj p . Javni ključ je niz $(x_0, \dots, x_n)$ takav da je x_0 najveći i neparan, i za sve $i \in 1, \dots, n$ važi $x_i = p \cdot q_i + r_i$ gde su q_i, r_i slučajni celi brojevi, s tim što r_0 mora da bude paran.

Enkripcija Poruka $m \in \{0, 1\}$ se šifruje u

$$c = m + 2r + 2 \sum_{i \in S} x_i \pmod{x_0}$$

gde je r slučajan ceo broj, a S slučajan podskup od $\{1, \dots, n\}$. Deo $\pmod{x_0}$ se tu nalazi da šifrat ne bi bio besmisleno veliki. Hajde da sredimo sabirke:

$$\begin{aligned} m + 2r + 2 \sum_{i \in S} x_i \pmod{x_0} &= m + 2r + 2 \sum_{i \in S} (p \cdot q_i + r_i) \pmod{x_0} \\ &= m + 2(r + \sum_{i \in S} r_i) + p \cdot 2 \sum_{i \in S} q_i \pmod{x_0} \\ &= m + 2R + pQ \pmod{x_0} \end{aligned}$$

Jednačina šifrata je dobila skoro isti oblik kao u uvodnom primeru (deluje da samo

$(\text{mod } x_0)$ smeta).

Dekripcija Šifrat c se dešifruje sa

$$(c \cdot (\text{mod } p)) \cdot (\text{mod } 2)$$

Jedino je potrebno pokazati da $(\text{mod } x_0)$ ne smeta. Ako je $c = m + 2R + pQ$ $(\text{mod } x_0)$ možemo naći k takvo da $m + 2R + pQ = c + kx_0$. Kako je $x_0 = pq_0 + r_0$, imamo

$$\begin{aligned} c &= m + 2R + pQ - k(pq_0 + r_0) \\ &= m + 2(r + \sum_{i \in S} r_i) + p \cdot 2 \sum_{i \in S} q_i - k(pq_0 + r_0) \\ &= m + (2r + 2 \sum_{i \in S} r_i - kr_0) + p \cdot (2 \sum_{i \in S} q_i - kq_0) \end{aligned}$$

Kako smo r_0 odabrali tako da bude paran, vidimo da je c opet oblika $m + 2R' + pQ'$, i dekripcija radi identično kao u uvodnom primeru.

Sabiranje i množenje se ponašaju identično kao u uvodnom primeru.

10.6 Bootstrapping

Tehnika bootstrappinga ⁴ omogućava da se od šifrata koji šifruje poruku m uz veliku količinu šuma (nastalu kao posledica izvođenja homomorfni računavanja) dobije novi šifrat koji šifruje istu poruku m , ali sa malom količinom šuma. Tako, kada šum dobijen kao posledica aritmetičkih operacija postane preveliki, šifrat se osveže, i nad njima server nastavi da vrši homomorfna izračunavanja. Tako se povremenim osvežavanjem od delimično homomorfne enkripcije dobija potpuno homomorfna.

Intuicija Po definiciji, algoritam za dešifrovanje $D(c, sk)$ uklanja šum iz šifrata c . Ako bi server imao tajni ključ sk , mogao bi da izvrši algoritam $D(c, sk)$ koji uklanja šum, i da ponovo šifruje i dobije šifrat c' sa malim, svežim šumom. Ipak, nedopustivo je serveru otkriti tajni ključ sk .

Ovde dolazimo do Gentryjeve ključne ideje: i algoritam $D(c, sk)$ za dešifrovanje i uklanjanje šuma je aritmetička funkcija, koja se može izvršiti homomorfno u šifrovanom domenu. Klijent serveru šalje tajni ključ sk šifrovano. Ovo je zapravo evaluacioni ključ ek . Evaluacioni ključ je zapravo šifrat tajnog ključa $ek = E(sk, sk)$. Ako server izvrši $D(c, sk)$ homomorfno u šifrovanom domenu, rezultat je enkripcija

⁴Beleške sa predavanja [15] odlično objašnjavaju bootstrapping

iste polazne poruke, ali drugi šifrat c' sa manjim šumom.

Algoritam D po definiciji uklanja šum, tako da će jedini šum prisutan u c' biti onaj koji proizilazi od osnovnog šuma potrebnog za šifrovanje, i operacija koje čine D.

Tako se od delimično homomorfne enkripcione šeme dobija potpuno homomorfna. c je šifrat poruke m u polaznoj delimično homomorfnoj enkripcionoj šemi, a c' u potpuno homomorfnoj enkripcionoj šemi koju smo od polazne šeme dobili procesom bootstreping-a.

Da bi ova konstrukcija bila moguća, funkcija D mora biti „dovoljno jednostavna”, odnosno mora pripadati familiji funkcija kojima polazna delimično homomorfna enkripciona šema barata.

Primetimo da se bootstrapping oslanja na još jednu pretpostavku o bezbednosti. Ona se zove pretpostavka o *kružnoj bezbednosti* i podrazumeva da je bezbedno objaviti šifrat tajnog ključa. Ovo nije bezbedno u svakoj enkripcionoj šemi. Za neke šeme se zna da nisu kružno bezbedne i one se, naravno, ne koriste za FHE. Za šeme koje se koriste (npr. GSW) se ne zna da li su kružno bezbedne. Pretpostavlja se da jesu, jer nijedan napad nije objavljen, ali trenutno ne postoji dokaz da jesu bezbedne. Sve FHE šeme koje trenutno postoje se oslanjaju na pretpostavku o kružnoj bezbednosti. Naći neku koja se ne oslanja predstavlja jedno otvoreno pitanje za dalji razvoj.

10.7 Primene

10.7.1 Podaci u oblaku

Ovo je sveobuhvatna primena, glavni razlog zašto je FHE koncipiran: korisnik želi da delegira izračunavanje neke funkcije moćnom ali nepoverljivom serveru. Učestvovanje u oblaku tradicionalno podrazumeva da server u oblaku ima pristup podacima jer je potreban za vršenje izračunavanja. Nedovoljno pravne regulacije kao i kršenje postojeće regulacije daje vlasniku servera moć da proda korisničke podatke trećoj strani. Osim toga, serveri su ranjivi na zlonamerne napade [17].

10.7.2 Mašinsko učenje

Homomorfna enkripcija nalazi primene u mašinskom učenju - veoma prirodno sa obzirom na to da se svaki model mašinskog učenja svodi na kompoziciju aritmetičkih

operacija, i da je mnoge modele potrebno trenirati na osjetljivim podacima [21]⁵. Od značaja su i šifrovanje podataka pri upotrebi gotovog modela, i treniranje modela na šifrovanim podacima. Koristiti gotov model na šifrovanim podacima može biti korisno na primer za sakrivanje osjetljivih upita postavljenih velikom jezičkom modelu. Treniranje modela na šifrovanim podacima između ostalog omogućava bezbednu kolaboraciju. Recimo, nekoliko bolnica želi zajedno da učestvuju u treniranju modela, ali svaka bolnica je dužna da sakrije osjetljive medicinske podatke svojih pacijenata. Uz FHE, svaka bolnica doprinosi enkriptovanim podacima, a model se trenira tako da nijedna institucija nije videla podatke druge.

Isti princip bezbedne kolaboracije primenjuje se šire od mašinskog učenja — na primer, banka može da organizuje aukciju u kojoj čak ni ona kao organizator ne saznaje pojedinačne ponude učesnika i cene prodavaca [15].

10.7.3 Internet stvari

Uređaji povezani u internet stvari kontinuirano prikupljaju lične podatke — o zdravlju, lokaciji, svakodnevnom navikama — koji se tipično šalju na server radi obrade. FHE omogućava da ta obrada ostane u potpunosti u šifrovanom domenu, i time korisnicima pruža nivo privatnosti koji tradicionalne arhitekture ne garantuju [17].

10.8 Otvorena pitanja

Iako implementacije FHE postoje (HElib, Microsoft SEAL, OpenFHE i druge), upotreba je trenutno ograničena zbog ekonomisanja računarskim resursima [12]. Sve implementacije se oslanjaju na vršenje operacija uz gomilanje šuma i redovno bootstrapovanje. Ovaj proces je računarski zahtevan. Bilo bi revolucionarno naći šemu koja ne zahteva bootstrapping, ali svaki pronalazak šeme sa računski manje zahtevnim bootstrappingom je značajan pomak.

Osim ovoga, postoji prostor za napredak u nalaženju FHE šema koje se ne oslanjaju na rešetke [15]. Sve moderne implementacije se oslanjaju na LWE ili prstenasti LWE. U kriptografiji je uvek dobro konstruisati primitive na razne načine, tako da se oslanjanju na razne nezavisne pretpostavke o težini. Ako bi pretpostavke o težini LWE i prstenastog LWE bile srušene, najbolje poznate konstrukcije FHE takođe padaju sa njima.

⁵Strana 22, sekcija FHE FOR MACHINE LEARNING

10.9 Pitanja

- Šta je homomorfna enkripcija?
- Šta je bootstrapping u homomorfnoj enkripciji?
- Navesti neku primenu homomorfne enkripcije.

Glava 11

Secret sharing

11.1 Uvod

Deljenje tajni (engl. *secret sharing*) je kriptografska tehnika kojom se tajna distribuira među n učesnika tako da samo određene grupe mogu da je rekonstruišu, dok sve ostale grupe ne znaju ništa o njenoj vrednosti.

Posmatrajmo naredni primer, koji ćemo koristiti za uvođenje osnovnih pojmova:

Primer 11.1.1. Generalni štab čuva šifru za aktivaciju vojnog stanja u zemlji. Skup učesnika je $P = \{M, N, K, O\}$, gde su:

- M : Ministar odbrane,
- N : Načelnik štaba,
- K : Komandant kopnenih snaga,
- O : Obaveštajni direktor.

Pritom važe sledeća pravila:

1. Obaveštajni direktor ne sme sam da aktivira vojno stanje.
2. Ministar i obaveštajni direktor zajedno mogu.
3. Načelnik, komandant i obaveštajni direktor zajedno mogu.

Dakle, minimalni skupovi koji mogu da aktiviraju vojno stanje su $\{M, O\}$ i $\{N, K, O\}$. Svaki njihov nadskup takođe može rekonstruisati tajnu.

Deo tajne koji je dodeljen svakom učesniku se naziva **udeo** (engl. *share*). Svaki podskup članova koji može da rekonstruiše tajnu ćemo nazvati **kvalifikujući skup**, a skup svih kvalifikujućih skupova naziva se **struktura pristupa** (engl. *access structure*). Za primer iznad, struktura pristupa je

$$\Gamma = \{\{M, O\}, \{M, O, N\}, \{M, O, K\}, \{N, K, O\}, \{M, N, K, O\}\}.$$

Svaki skup u strukturi pristupa je skup sa kojim možemo da rekonstruišemo tajnu.

Definicija 11.1.2. Monotona struktura na skupu P je kolekcija Γ podskupova od P takva da važi:

- $P \in \Gamma$,
- ako je $A \in \Gamma$ i $A \subset B \subset P$, onda je i $B \in \Gamma$.

Primetimo da je struktura pristupa iz primera monotona. Ovo svojstvo važi za sve strukture pristupa svih šema deljenja tajni. Skupove koji nisu sadržani ni u jednom manjem kvalifikujućem skupu nazivamo **minimalni kvalifikujući skupovi** i označavamo ih sa $m(\Gamma)$. Za primer 11.1.1: $m(\Gamma) = \{\{M, O\}, \{N, K, O\}\}$.

Definicija 11.1.3. Šema deljenja tajni za monotonu strukturu pristupa Γ nad skupom članova P je par algoritama **Share** i **Recombine** za koje važe:

- **Share**(s, Γ) za svakog učesnika $A \in P$ određuje vrednost s_A , koja predstavlja njegov udeo tajne.
- **Recombine**(H) na osnovu udela učesnika iz skupa H vraća tajnu ako je H kvalifikujući, inače ne vraća ništa

Stranu koja izvršava algoritam **Share** i tajno raspodeljuje udele učesnicima nazivamo **diler** (engl. *dealer*). Pretpostavljamo da je diler poverljiv, odnosno da ne odaje tajnu ni jednom nekvalifikujućem skupu učesnika.

Šema deljenja tajni se smatra sigurnom ako nijedan nekvalifikujući skup učesnika ne može dobiti nikakvu informaciju o tajni.

Posebno važan slučaj monotone strukture pristupa je **pragovna struktura pristupa** (engl. *threshold access structures*). Za skup učesnika $P = \{p_1, \dots, p_n\}$ i ceo broj $t \leq n$, pragovna struktura pristupa definiše se kao:

$$\Gamma_{t,n} = \{B \subseteq P : |B| \geq t\},$$

odnosno svaki skup od bar t učesnika može rekonstruisati tajnu. Takvu šemu nazivamo *t*-od-*n* šemom deljenja tajni. [29]¹

11.1.1 Način predstavljanja struktura

Strukturu pristupa možemo predstaviti preko formula Bulove algebre u disjunktivnoj normalnoj formi (DNF) - kao disjunkciju konjunkcija učesnika iz minimalnih kvalifikujućih skupova:

$$\bigvee_{O \in m(\Gamma)} \bigwedge_{A \in O} A.$$

Za primer 11.1.1 ova formula glasi:

$$(M \wedge O) \vee (N \wedge K \wedge O).$$

Ovu reprezentaciju koristićemo u nastavku prilikom konstrukcije pojedinih šema deljenja tajni.

11.2 Opšte metode deljenja tajni

U ovom poglavlju ćemo prikazati dve opšte metode deljenja tajni koje su prilično neefikasne, ali pokazuju da je uvek moguće konstruisati šemu za proizvoljnu strukturu. Iako postoje znatno efikasnije konstrukcije za pojedine klase struktura pristupa, problem konstruisanja optimalnih šema za proizvoljne strukture pristupa i dalje je predmet istraživanja. [1]²

11.2.1 Ito-Niizeki-Saito šema

Šema Ito-Niizeki-Saito(INS) direktno koristi DNF reprezentaciju strukture pristupa. Svako “ili” iz DNF formule odgovara nezavisnom deljenju tajne, a svako “i” odgovara XOR dekompoziciji (oznaka $\oplus$). Algoritam deljenja tajni u ovom slučaju je sledeći:

1. Zapiši strukturu pristupa kao DNF formulu
2. Za svaki minimalni kvalifikujući skup O sa l članova, generiši nasumične vrednosti $s_1, s_2, \dots, s_l$ takve da važi $s_1 \oplus s_2 \oplus \dots \oplus s_l = s$.

¹Strana 1

²Strana 110

3. Svaki učesnik dobija odgovarajući udeo iz svake klauzule u kojoj se pojavljuje: Učesnik A dobija vrednost s_i ako se pojavljuje na i -toj poziciji u izrazu za skup O .

Primer 11.2.1 (INS za primer 11.1.1). DNF formula: $(M \wedge O) \vee (N \wedge K \wedge O)$. Generišemo vrednosti za svaku klauzulu:

$$\begin{aligned} s &= s_1 \oplus s_2 \quad (\text{za klauzulu } M \wedge O), \\ s &= s_3 \oplus s_4 \oplus s_5 \quad (\text{za klauzulu } N \wedge K \wedge O). \end{aligned}$$

Udeli su:

$$s_M = s_1, \quad s_N = s_3, \quad s_K = s_4, \quad s_O = s_2 \parallel s_5.$$

Proveravamo:

- $\{M, O\}$: imaju s_1 i s_2 , pa $s_1 \oplus s_2 = s$.
- $\{N, K, O\}$: imaju s_3, s_4 i s_5 , pa $s_3 \oplus s_4 \oplus s_5 = s$.
- $\{M, N\}$: imaju s_1 i s_3 , pa ne mogu rekonstruisati tajnu.

11.2.2 Replicirano deljenje tajni

Drugi pristup ide u suprotnom smeru: umesto minimalnih kvalifikujućih skupova, gledamo maksimalne nekvalifikujuće skupove. Maksimalni nekvalifikujući skup je skup kome nedostaje tačno jedna osoba da bi bio kvalifikujući, i ne postoji drugi maksimalni nekvalifikujući skup čiji je on podskup.

Algoritam deljenja tajni u ovom slučaju je sledeći:

1. Nađi sve maksimalne nekvalifikujuće skupove $A_1, A_2, \dots$
2. Izračunaj njihove komplemente $B_i = P \setminus A_i$.
3. Generiši udele $s_1, \dots, s_k$ takve da $s_1 \oplus \dots \oplus s_k = s$, gde je k broj maksimalnih nekvalifikujućih skupova.
4. Učesnik dobija udeo s_i ako se nalazi u skupu B_i .

Primer 11.2.2 (Replicirano deljenje za primer 11.1.1). Maksimalni nekvalifikujući skupovi i njihovi komplementi:

$$A_1 = \{M, N, K\} \Rightarrow B_1 = \{O\}, \quad A_2 = \{N, O\} \Rightarrow B_2 = \{M, K\}, \quad A_3 = \{K, O\} \Rightarrow B_3 = \{M, N\}.$$

Udeli ($s = s_1 \oplus s_2 \oplus s_3$):

$$s_O = s_1, \quad s_N = s_3, \quad s_K = s_2, \quad s_M = s_2 || s_3.$$

Proveravamo:

- $\{M, O\}$: imaju $s_2 || s_3$, s_1 — zajedno imaju sve tri vrednosti, pa $s_1 \oplus s_2 \oplus s_3 = s$.
- $\{N, K, O\}$: imaju s_3, s_2, s_1 — $s_1 \oplus s_2 \oplus s_3 = s$.
- $\{N, O\}$: imaju s_3 i s_1 — nedostaje s_2 .

Obe opisane šeme su sigurne i mogu se primeniti na proizvoljnu monotonu strukturu pristupa. Međutim, njihova efikasnost je često nezadovoljavajuća, naročito kod pragovnih struktura sa velikim brojem učesnika. Zbog toga ćemo u nastavku razmatrati Šamirovu šemu deljenja tajni, koja predstavlja jednostavno i efikasno rešenje za t -od- n šeme. [31]³

11.3 Rid-Solomonovo kodiranje

Da bi se razumela Šamirova šema u nastavku, najpre treba da uvedemo Rid-Solomonove kodove (engl. *Reed-Solomon codes*) jer je ona suštinski zasnovana na njima. Veza između ova dva koncepta je direktna: Šamirova šema je suštinski Rid-Solomonov kod u kome tajna odgovara vrednosti polinoma u nuli, a udeli su vrednosti tog polinoma u ostalim tačkama.

Kod za korekciju grešaka je mehanizam za prenos podataka koji omogućava ispravljanje grešaka nastalih u toku prenosa. Međutim, za razliku od grešaka u teoriji kodiranja, gde greška predstavlja uglavnom slučajni šum, u kriptografiji greške uvodi napadač. U kontekstu deljenja tajni, učesnici koji daju pogrešne udele su upravo takvi napadači. [31]⁴

Radimo nad konačnim poljem $\mathbb{F}_q$ (skupom celih brojeva mod q , gde je q prost ili stepen prostog broja). Biramo parametre n (dužina kodne reči), t (stepen polinoma), i skup od n različitih tačaka $X = \{x_1, \dots, x_n\} \subset \mathbb{F}_q \setminus \{0\}$.

Posmatramo skup polinoma stepena $\leq t$:

$$\mathcal{P} = \{f_0 + f_1X + \dots + f_tX^t : f_i \in \mathbb{F}_q\}.$$

³Strana 407

⁴Strana 407

Kodna reč (engl. *code-word*) nastaje evaluacijom polinoma $f \in \mathcal{P}$ u svim tačkama iz X :

$$c = (f(x_1), f(x_2), \dots, f(x_n)).$$

Primer 11.3.1. Posmatrajmo Rid-Solomonov kod sa parametrima $q = 101$, $n = 7$, $t = 2$ i $X = \{1, 2, \dots, 7\}$. Neka je polinom $f(X) = 20 + 57X + 68X^2$. Računamo vrednosti $f(i) \pmod{101}$ za $i = 1, \dots, 7$ i dobijamo kodnu reč:

$$c = (44, 2, 96, 23, 86, 83, 14).$$

11.3.1 Rekonstrukcija podataka - Lagranžova interpolacija

Ključna matematička činjenica je da je polinom stepena $\leq t$ jednoznačno određen svojom vrednošću u $t + 1$ tačaka. Na primer, prava linija (stepen 1) je određena dvema tačkama, a parabola (stepen 2) trima tačkama.

Definišemo **Lagranžove bazne polinome**:

$$\delta_i(X) = \prod_{\substack{x_j \in X \\ j \neq i}} \frac{X - x_j}{x_i - x_j}, \quad 1 \leq i \leq n.$$

Primetimo da važi $\delta_i(x_i) = 1$ i $\delta_i(x_j) = 0$ za $j \neq i$.

Rekonstrukcija polinoma vrši se formulom:

$$f(X) = \sum_{i=1}^n c_i \cdot \delta_i(X).$$

Iz činjenice iznad možemo zaključiti da važi $f(x_i) = c_i$. [31]⁵

11.3.2 Detekcija greške

Grešku ćemo jednostavno uočiti proverom stepena polinoma. Ako primljena kodna reč sadrži grešku, Lagranžova interpolacija daje polinom stepena većeg od t .

Primer 11.3.2. Primamo kodnu reč $c' = (44, 2, 25, 23, 86, 83, 14)$ umesto ispravne kodne reči iz Primera 11.3.1, treća pozicija je pogrešna (25 umesto 96). Lagranžova interpolacija nad svih 7 tačaka daje polinom stepena 6, a ne stepena $t = 2$, čime zaključujemo da postoji greška u kodnoj reči.

⁵Strana 409

11.3.3 Ispravljanje greške: Berlekemp-Velč algoritam

Nakon što je greška otkrivena, potrebno ju je i ispraviti. Za ovo postoji efikasan algoritam, poznat kao Berlekemp-Velč algoritam.

Pretpostavimo da broj grešaka e i broj brisanja s zadovoljavaju uslov $n > t + 2e + s$, gde je s broj pozicija za koje uopšte nije primljena vrednost. Tražimo polinom sa dve promenljive $Q(X, Y) = f_0(X) - f_1(X) \cdot Y$, gde je $\deg(f_0) \leq 2t$, $\deg(f_1) \leq t$ i $f_1(0) = 1$. Namećemo uslov da Q interpolira primljene tačke, tj. $Q(x_i, y_i) = 0$ za sve (x_i, y_i) . Supstitucijom primljenih tačaka (x_i, y_i) dobijamo linearan sistem jednačina po koeficijentima f_0 i f_1 , čijim rešavanjem dobijamo:

$$f(X) = \frac{f_0(X)}{f_1(X)}.$$

Da bismo se uverili u ovo, definišimo $P(X) = Q(X, f(X))$, gde je $f(X)$ traženi (originalni) polinom. Važi $\deg(P) \leq 2t$, ali $P(X)$ se poništava u svim ispravnim tačkama, kojih je bar $n - s - e$. Pošto je ovaj broj veći od $2t$ pod našom pretpostavkom, polinom P ima više nula nego što mu stepen dozvoljava, pa mora biti identički jednak nuli. Odatle sledi $f_0(X) = f_1(X) \cdot f(X)$, što daje traženo rešenje. [31]⁶

Primer 11.3.3. Za primljenu kodnu reč $c' = (44, 2, 25, 23, 86, 83, 14)$ sa $t = 2$, tražimo polinom $Q(X, Y) = f_0(X) - f_1(X) \cdot Y$, gde je $\deg(f_0) \leq 4$ i $\deg(f_1) \leq 2$, pri čemu važi $f_1(0) = 1$. Supstitucijom svih primljenih tačaka (x_i, y_i) dobijamo linearan sistem jednačina za koeficijente polinoma f_0 i f_1 . Rešavanjem sistema dobijamo:

$$f_0(X) = 20 + 84X + 49X^2 + 11X^3, \quad f_1(X) = 1 + 67X.$$

Odavde:

$$f(X) = \frac{f_0(X)}{f_1(X)} = 20 + 57X + 68X^2,$$

što je upravo polinom iz Primera 11.3.1. Vidimo da je greška uspešno korigovana.

11.4 Šamirova šema deljenja tajni

Šamir u svom originalnom radu motiviše problem sledećim primerom: jedanaest naučnika radi na tajnom projektu i želi da zaključa dokumenta u sef tako da se on može otvoriti ako i samo ako je prisutno bar šest naučnika. Pokazuje se da naivno rešenje pomoću brava i ključeva vodi do velikog broja kombinacija i ne

⁶Strana 411

skalira dobro sa veličinom grupe. [29] ⁷Cilj je generalizovati problem na deljenje proizvoljnih podataka kroz matematičke metode. Šamir predlaže rešenje zasnovano na polinomijalnoj interpolaciji i $(t + 1) - od - n$ šemi praga, a kasnije je pokazano da se ono može sagledati i kao poseban slučaj Rid-Solomonovog kodiranja opisanog u prethodnom poglavlju. [31]⁸

11.4.1 Algoritam deljenja

Neka je $\mathbb{F}_q$ konačno polje sa $q > n$ i neka je tajna $s \in \mathbb{F}_q$. Diler bira nasumičan polinom stepena t takav da je $f(0) = s$:

$$f(X) = s + f_1X + f_2X^2 + \dots + f_tX^t,$$

gde su $f_1, \dots, f_t$ nasumični elementi iz $\mathbb{F}_q$. Učesnici se identifikuju elementima $\{1, \dots, n\} \in \mathbb{F}_q \setminus \{0\}$; pošto je $q > n$, polje $\mathbb{F}_q$ sadrži bar n ne-nula elemenata, pa možemo jednostavno uzeti $\alpha_j = j$ za $j = 1, \dots, n$. Učesniku j se dodeljuje udeo:

$$s_j \leftarrow f(j).$$

Primetimo da je vektor $(s, s_1, \dots, s_n)$ tačno kodna reč Rid-Solomonovog koda.

11.4.2 Algoritam rekonstrukcije

Ako se okupi podskup $Y \subset X$ od bar t učesnika, oni mogu rekonstruisati originalni polinom Lagranžovom interpolacijom evaluiranom u nuli:

$$s = f(0) = \sum_{i \in Y} s_i \cdot \delta_i(0),$$

gde je:

$$\delta_i(0) = \prod_{\substack{j \in Y \\ j \neq i}} \frac{-j}{i - j}.$$

Koeficijenti $\delta_i(0)$ zavise samo od skupa Y i mogu se izračunati unapred. [1]⁹

⁷Strana 1

⁸Strana 413

⁹Strana 12

11.4.3 Korekcija pogrešnih udela

Korekciju potencijalno pogrešnih udela možemo izvršiti primenom Berlekemp-Velč algoritma, pod uslovom da je broj netačnih udela e ograničen sa: [31]¹⁰

$$e < t < \frac{|Y|}{3}.$$

11.4.4 Pseudoslučajno deljenje tajni (PRSS)

Šamirova šema se može pretvoriti u **pseudoslučajnu šemu deljenja tajni** (engl. *pseudo-random secret sharing*, PRSS), koja omogućava generisanje deljenja nasumične vrednosti uz minimalnu interakciju, ograničenu na fazu inicijalizacije [31].¹¹

Učesnike obeležavamo skupom $X = \{1, \dots, n\}$, sa pragom $t + 1$. Za svaki podskup $A \subset X$ veličine $n - t$ definišemo polinom $f_A(X)$ stepena t takav da je $f_A(0) = 1$ i $f_A(i) = 0$ za sve $i \in X \setminus A$. U fazi inicijalizacije, svakom podskupu A se dodeljuje tajna vrednost r_A , bezbedno distribuirana svim učesnicima iz A , zajedno sa javnom pseudoslučajnom funkcijom $\psi(r_A, a)$.

Kada učesnici žele da generišu deljenje nove nasumične vrednosti, biraju javnu nasumičnu vrednost a i definišu polinom:

$$f(X) = \sum_{A \subset X, |A|=n-t} \psi(r_A, a) \cdot f_A(X).$$

Učesnik i tada može izračunati svoj udeo:

$$s_i \leftarrow \sum_{i \in A \subset X, |A|=n-t} \psi(r_A, a) \cdot f_A(i),$$

a deljena tajna je

$$s = f(0) = \sum_{A \subset X, |A|=n-t} \psi(r_A, a) \cdot f_A(0) = \sum_{A \subset X, |A|=n-t} \psi(r_A, a).$$

[31]¹²

¹⁰Strana 413

¹¹Strana 413

¹²Strana 414

11.5 Zaključak

U radu su predstavljeni osnovni pojmovi i metode deljenja tajni, sa posebnim osvrtom na Šamirovu šemu. Pokazana je njena veza sa Rid-Solomonovim kodovima, kao i mogućnost proširenja na pseudoslučajno deljenje tajni. Deljenje tajni predstavlja važan kriptografski mehanizam koji omogućava bezbednu distribuciju osetljivih podataka i ima značajnu primenu u distribuiranim sistemima i savremenim kriptografskim protokolima.

11.6 Pitanja

1. Opisati problem deljenja tajni i objasniti kako se tajna distribuira među učesnicima.
2. Kako se tajna rekonstruiše u Šamirovoj šemi i koji matematički postupak se koristi?
3. Objasniti vezu između stepena polinoma i minimalnog broja učesnika potrebnih za rekonstrukciju tajne u Šamirovoj šemi.

Glava 12

Zakljucak

12.1 Rezime rezultata

U radu su predstavljeni osnovni pojmovi kriptografije i formalni zapisi. Posebno je istaknuta veza izmedju funkcija enkripcije i dekripcije.

12.1.1 Dalji rad

U nastavku je moguće proširiti analizu na savremene standarde i protokole.

Napomena 12.1.1. Ovo poglavlje služi kao kratak rezime i smernice za dalji rad.

Primer 12.1.2. Primer otvorenog problema je formalna analiza bezbednosti u realnim sistemima.

Bibliografija

- [1] Amos Beimel. Secret-sharing schemes for general access structures: An introduction. *Cryptology ePrint Archive*, Paper 2025/518, 2025.
- [2] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *Cryptology ePrint Archive*, (2018/046), 2018.
- [3] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for R1CS. *Cryptology ePrint Archive*, (2018/828), 2019.
- [4] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. DEEP-FRI: Sampling outside the box improves soundness. In *11th Innovations in Theoretical Computer Science Conference (ITCS 2020)*, volume 151 of *Leibniz International Proceedings in Informatics*, pages 5:1–5:32. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2020.
- [5] Daniel J. Bernstein, Johannes Buchmann, and Erik Dahmen, editors. *Post-Quantum Cryptography*. Springer Berlin, Heidelberg, 2009.
- [6] Manuel Blum. Coin flipping by telephone: A protocol for solving impossible problems. *SIGACT News*, 15(1):23–27, 1983.
- [7] Michael Burrows, Martin Abadi, and Roger M. Needham. A logic of authentication. *ACM Transactions on Computer Systems*, 8(1):18–36, 1990.
- [8] Vitalik Buterin, Dankrad Feist, Diederik Loerakker, George Kadianakis, Matt Garnett, Mofi Taiwo, and Ansgar Dietrichs. EIP-4844: Shard blob transactions. Ethereum Improvement Proposal, 2022.
- [9] David Cooper, Stefan Santesson, Stephen Farrell, Sharon Boeyen, Russ Housley, and W. Polk. Internet x.509 public key infrastructure certificate and certificate revocation list profile. RFC 5280, Internet Engineering Task Force, 2008.

- [10] Ethereum Foundation. Merkle patricia trie. Ethereum Developer Documentation, 2026.
- [11] Chaya Ganesh et al. Demystifying the role of zk-snarks in zcash. *arXiv preprint arXiv:2008.00881*, 2020.
- [12] Craig Gentry. Computing arbitrary functions of encrypted data. *Communications of the ACM*, 53(3):97–105, 2010.
- [13] Oded Goldreich. *Computational Complexity: A Conceptual Perspective*. Cambridge University Press, Cambridge, 2008.
- [14] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof-systems. 1985.
- [15] Alexandra Henzinger. Lecture 9: Fully homomorphic encryption (part 2). Technical report, MIT CSAIL, 2024.
- [16] IBM. Digital certificates. IBM Db2 11.5 Documentation, 2026. Available online.
- [17] IEEE Digital Privacy. Homomorphic encryption use cases. IEEE Digital Privacy, 2026. Accessed: 2026-06-17.
- [18] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Advances in Cryptology — ASIACRYPT 2010*, volume 6477 of *Lecture Notes in Computer Science*, pages 177–194. Springer, 2010.
- [19] Jeffrey C. Lagarias and Andrew M. Odlyzko. Solving low-density subset sum problems. *Journal of the ACM*, 32(1):229–246, 1985.
- [20] Ryan Lavin, Xuekai Liu, Hardhik Mohanty, Logan Norman, Giovanni Zaarour, and Bhaskar Krishnamachari. A survey on the applications of zero-knowledge proofs. 2024.
- [21] Chiara Marcolla, Victor Sucasas, Marc Manzano, and Riccardo Bassoli. Survey on fully homomorphic encryption, theory, and applications. *Proceedings of the IEEE*, 110(10):1572–1609, 2022.
- [22] Alexander Meyer. Post-quantum cryptography: An analysis of code-based and lattice-based cryptosystems, 2025.
- [23] Andrei Nitulescu. zk-snarks: A gentle introduction. 2023.

- [24] National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Federal Information Processing Standards Publication 203, National Institute of Standards and Technology, 2024.
- [25] João Vítor Oliveira Paliare and João Martinelli. Explaining the dghv encryption scheme. Medium, June 2024.
- [26] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Advances in Cryptology — CRYPTO 1991*, volume 576 of *Lecture Notes in Computer Science*, pages 129–140. Springer, 1991.
- [27] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing*, STOC '05, pages 84–93, New York, NY, USA, 2005. Association for Computing Machinery.
- [28] Seyed Mohsen Rostamkolaei Motlagh, Claus Pahl, Hamid R. Barzegar, and Nabil El Ioini. A comparative evaluation of zero knowledge proof techniques. In *Proceedings of the 10th International Conference on Internet of Things, Big Data and Security*, pages 237–244. SCITEPRESS, 2025.
- [29] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
- [30] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, October 1997.
- [31] Nigel P Smart. *Cryptography made simple*. Springer, 2016.
- [32] William Stallings. *Cryptography and Network Security: Principles and Practice*. Prentice Hall, 5 edition, 2011.
- [33] StarkWare Team. ethSTARK documentation – version 1.2. Technical report, StarkWare Industries Ltd., July 2023.
- [34] Stateless Consensus. Verkle replay - ethereum stateless book. Online, 2025.
- [35] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. Technical report, Ethereum Foundation, 2014. Ethereum Yellow Paper.
- [36] Andrew C. Yao. Protocols for secure computations. In *23rd Annual Symposium on Foundations of Computer Science (SFCS 1982)*, pages 160–164. IEEE, 1982.